DFSha — Sistema de Archivos Distribuido por Bloques

Informe técnico · Universidad EAFIT · SI3007 / ST0263 Sistemas Distribuidos · 2026-2

9 de octubre de 2026

Índice

Resumen ejecutivo 4

1. Objetivo y marco teórico 5

1.1 El problema 5

1.2 Objetivos 5

1.3 Marco teórico 6

1.4 Posición de DFSha frente al enunciado 8

2. Descripción del servicio y requisitos 9

2.1 Qué entrega DFSha 9

2.2 Requisitos funcionales 9

2.3 Requisitos no funcionales 10

2.4 Alcance por hitos 11

3. Arquitectura 12

3.1 Vista general 12

3.2 Las cuatro decisiones estructurales 13

3.3 Dos direcciones por DataNode 14

3.4 CQRS: por qué separar comandos de consultas desde el primer día 15

3.5 Estructura del código 16

4. Protocolos y APIs 17

4.1 Por qué dos protocolos y no uno 17

4.2 ControlNode /api/v1 — lo consume el cliente 17

4.3 La escritura en tres fases, y por qué tiene que ser así 21

4.4 El plano de control sobre gRPC 22

4.5 Por qué /blocks/{id}/stored se quedó en REST 23

4.6 DataNode /api/v1 23

4.7 Logging estructurado como contrato observable 24

5. Particionamiento y distribución..... 26
5.1 Bloques de tamaño fijo..... 26
5.2 La política: *power of d choices* con dominio de falla..... 26
5.3 Dominios de falla..... 26
5.4 El recuento dentro de la misma llamada..... 27
5.5 Divergencia: detectar sin borrar..... 27
5.6 Resultados medidos..... 28
6. Alta disponibilidad..... 29
6.1 Qué había que eliminar..... 29
6.2 Tres ControlNodes sin estado..... 29
6.3 La época: un lease no basta..... 30
6.4 El patrón de la etapa: exigir en vez de preguntar..... 32
6.5 Detección de fallos de los DataNodes..... 34
6.6 El balanceador: tres puertos, dos niveles..... 35
6.7 Lo que sigue siendo punto único de fallo..... 37
7. Replicación y consistencia..... 38
7.1 R=3 en cadena: el cliente sube una vez..... 38
7.2 Verificar antes de reenviar, escribir y reenviar a la vez..... 39
7.3 El interbloqueo que costó encontrar..... 39
7.4 El quórum W=2..... 39
7.5 Re-replicación: tres frenos..... 40
7.6 Lectura: relevo entre réplicas..... 41
7.7 Sobre-replicación tras una reincorporación: estado esperable..... 42
7.8 Consistencia del metadato: CQRS y read-your-writes..... 42
7.9 El RF3 y el bloqueo de archivos..... 43
7.10 write es append, y el WORM obliga a elegir dónde se paga..... 44
8. Seguridad..... 46
8.1 El modelo, en una frase..... 46
8.2 Autenticación..... 46
8.3 mTLS en el plano interno..... 46
8.4 TLS de cara al cliente..... 47
8.5 Cifrado extremo a extremo..... 48

8.6 ACLs: cuatro reglas, una función, y ningún «denegar» 50
8.7 Token de bloque: lo que hace que las ACLs sean ciertas 51
9. Entorno de ejecución y reproducibilidad 54
9.1 Requisitos previos 54
9.2 Despliegue local, paso a paso 54
9.3 Verificar las propiedades del sistema, una por una 58
9.4 Encender y apagar el TLS de cliente 59
9.5 Despliegue en AWS 62
9.6 Problemas frecuentes y qué significan 65
9.7 Puertos 67
10. Pruebas y análisis de resultados 68
10.1 Estrategia 68
10.2 La suite 68
10.3 Verificación por mutación: cómo sabemos que las pruebas prueban algo 69
10.4 Resultados medidos en el despliegue local 71
10.5 Resultados medidos en AWS 72
10.6 Veintidós casos en siete capas: el modo de fallo de este proyecto 73
10.7 Un intermitente conocido, declarado 80
11. Límites conocidos y trabajo futuro 81
11.1 Límites del sistema entregado 81
11.2 Trabajo futuro, en orden de valor 82
11.3 Qué se llevaría el equipo a un proyecto siguiente 82
Anexo A — Modelo de metadatos 84
Anexo B — Variables de configuración 87
Anexo C — Glosario 89
Anexo D — Dónde está cada cosa 90

Resumen ejecutivo

DFSha es un sistema de archivos distribuido basado en bloques, de arquitectura tipo HDFS y opción cliente/servidor, construido a lo largo de cuatro hitos para el curso SI3007 / ST0263. Parte archivos en bloques de tamaño fijo, los reparte entre varios DataNodes según la carga real y el dominio de falla de cada uno, y guarda el metadato en un ControlNode replicado. **Los bytes nunca atraviesan el ControlNode**: este decide *dónde* va cada bloque y el cliente habla directamente con los DataNodes, de modo que añadir un nodo de datos suma ancho de banda en lugar de saturar un nodo central.

El sistema entregado cumple los tres requisitos funcionales del enunciado —espacio de nombres (RF1), transferencia de archivos (RF2) y acceso a nivel de archivo con bloqueo (RF3)— y los no funcionales que el curso pide demostrar: particionamiento, replicación, alta disponibilidad, tolerancia a fallos y seguridad.

Lo medido, no lo afirmado:

Propiedad	Evidencia medida
Particionamiento	50 bloques repartidos 11/13/14/12 entre cuatro nodos; 150 réplicas repartidas 36/38/39/37 en AWS
Replicación	`FULLY_REPLICATED (3 de 3)` con quórum de commit W=2 y subida en cadena: el cliente envía los bytes **una sola vez**
Tolerancia a fallos	Caída de un DataNode con **43 bloques sub-replicados restaurados en ~80 s sin intervención**
Alta disponibilidad	Relevo de líder con la época subiendo de 1 a 2 en ~10 s, **sin interrumpir el servicio**: un `put` de 50 MB confirmó con un solo ControlNode vivo
Integridad	SHA-256 idéntico en la ida y la vuelta de un archivo de 50 MB, con cifrado extremo a extremo activo
Seguridad	mTLS en el plano interno, cifrado AES-256-GCM en el cliente, ACLs con grupos y autorización por bloque firmada con RSA-PSS
Verificación	640 pruebas en verde (4 se saltan sin PostgreSQL) y 14 mutaciones de verificación, las 14 atrapadas

El despliegue se validó dos veces: **en local con `docker compose`** (once servicios) y **en AWS sobre seis instancias `t3.micro` en dos zonas de disponibilidad reales**, el 9 de octubre de 2026.

Este informe no se limita a describir lo construido. Cada decisión se presenta frente a la alternativa que se descartó y con el motivo por el que se descartó, porque en un sistema distribuido la diferencia entre dos diseños rara vez es que uno funcione y el otro no: casi siempre es qué falla deja abierto cada uno, y eso solo se ve si se escribe. La sección 10 va más lejos y documenta **veintidós fallos propios en siete capas distintas**, con lo que cada uno enseñó. Esa sección es deliberada: un informe que solo cuenta lo que salió bien no dice si el equipo sabe por qué salió bien.

1. Objetivo y marco teórico

1.1 El problema

Un archivo que no cabe en una máquina, o que no puede permitirse desaparecer con ella, obliga a resolver cuatro cosas a la vez, y ninguna se puede resolver sin tocar las otras tres:

1. **Dónde se parte y dónde va cada parte.** Si las partes se reparten mal, un nodo se satura y los demás están ociosos; si se reparten bien pero sin tener en cuenta qué se cae junto, perder un rack pierde las tres copias de un mismo bloque.
2. **Quién sabe dónde está cada parte.** Ese conocimiento es metadato, y el metadato es justo lo que no se puede perder: sin él, los bytes siguen en disco y nadie sabe de qué archivo son.
3. **Qué pasa cuando una máquina deja de contestar.** No "si" deja de contestar: a escala, que una máquina falle es el caso normal, no la excepción.
4. **Quién puede leer qué.** Y, en un sistema donde los bytes viajan directos entre cliente y nodo de datos, cómo llega una decisión de permisos tomada sobre rutas hasta un nodo que no sabe qué es una ruta.

DFSha existe para resolver esas cuatro cosas de una forma que se pueda demostrar y defender, no para ser rápido. Cuando hubo que elegir entre rendimiento y poder explicar por qué algo es correcto, se eligió lo segundo; y cuando se eligió lo primero, está escrito y argumentado.

1.2 Objetivos

Objetivo general. Diseñar, implementar y desplegar un sistema de archivos distribuido por bloques con particionamiento, replicación, alta disponibilidad y seguridad, y demostrar cada una de esas propiedades con evidencia medida sobre un despliegue real en varias máquinas.

Objetivos específicos.

1. Implementar el espacio de nombres (RF1) con semántica POSIX-like, donde las operaciones de movimiento y renombrado sean metadato puro y de coste constante.
2. Implementar la transferencia de archivos (RF2) partiendo en bloques, con verificación de integridad por bloque y sin que los bytes atraviesen el plano de control.
3. Implementar el acceso a nivel de archivo (RF3): apertura, lectura por rango, escritura incremental y bloqueo con vencimiento.
4. Distribuir los bloques entre N nodos de datos con una política que tenga en cuenta la carga real y el dominio de falla, no solo un hash.
5. Replicar cada bloque R=3 veces de forma que el cliente envíe los bytes una sola vez, y recuperar automáticamente las copias perdidas cuando un nodo muere.
6. Eliminar los puntos únicos de fallo del plano de control mediante varias instancias sin estado y elección de líder con un mecanismo de aislamiento correcto.
7. Garantizar confidencialidad, integridad y control de acceso en un modelo donde el servidor es, deliberadamente, no confiable para el contenido.
8. Desplegar el sistema en una infraestructura real de varias máquinas y medir su comportamiento ante fallos inducidos.

1.3 Marco teórico

Bloque como unidad de distribución

La decisión fundacional de un sistema como este es qué se distribuye. Las dos opciones clásicas son **el archivo** y **el bloque**.

Distribuir por archivo es más simple: un archivo vive en un nodo, y mover un archivo es mover un archivo. El problema es que el tamaño máximo de un archivo queda acotado por el disco más grande del clúster, y que el reparto de carga depende de que los archivos sean de tamaños parecidos, cosa que nunca ocurre.

Distribuir por bloque —la decisión de GFS, HDFS y, por tanto, de DFSha— desacopla el tamaño del archivo del tamaño del disco y convierte el reparto en un problema con grano fino. El precio es que hace falta una capa de metadato que sepa reconstruir el archivo a partir de sus bloques, y esa capa pasa a ser el componente crítico del sistema. Todo el diseño del ControlNode es consecuencia de haber aceptado ese precio.

Tamaño de bloque. Un bloque grande (64 MB por defecto, configurable) amortiza el coste de metadato: un archivo de 1 GB son 16 bloques y no 262 144. El coste es amplificación en operaciones pequeñas, que en DFSha se paga concretamente en el `append` del RF3 (sección 7). Es el mismo compromiso que HDFS resuelve igual, y por el mismo motivo.

WORM y copy-on-write

Un bloque cerrado en DFSha es **inmutable**: nunca se sobrescribe (*write once, read many*). Esto no es una limitación heredada, es la base sobre la que se apoyan después tres propiedades independientes:

- La **replicación** se simplifica: tres copias de algo que no cambia no pueden divergir. Todo el problema de reconciliar escrituras concurrentes sobre la misma réplica desaparece por construcción.
- La **integridad** se vuelve comprobable: un SHA-256 calculado al escribir vale para siempre.
- La **unicidad del nonce criptográfico** (sección 8) descansa en que un par `(file_id, index)` no se cifre dos veces con contenidos distintos, lo cual es exactamente lo que WORM garantiza.

Hacia el usuario la API se ve como un CRUD completo. Un “update” es *copy-on-write*: se escriben bloques nuevos y el metadato pasa a apuntar a ellos en una sola transacción. La fachada es mutable; el almacenamiento no.

Consistencia: qué se promete y qué no

DFSha ofrece **consistencia fuerte sobre el metadato** y **read-your-writes** sobre las consultas. No ofrece linealizabilidad global sobre lecturas de terceros, y conviene decirlo con precisión porque la diferencia es lo que permite tener réplicas de lectura:

- Todo comando (crear, borrar, mover, confirmar) se ejecuta contra el primario de PostgreSQL, en una transacción. No hay escrituras eventualmente consistentes.
- Las consultas se sirven por defecto desde una réplica de lectura, que va por detrás del primario por un retardo de replicación asíncrona.
- Un cliente **nunca** ve una réplica atrasada respecto de sus **propias** escrituras: el mecanismo de LSN de la sección 7.8 lo redirige al primario cuando hace falta.

- Un cliente **sí** puede ver, durante ese retardo, un estado anterior de una escritura **ajena**. Esa es la concesión explícita que compra el escalado de lectura.

Detección de fallos: el problema del fallo indistinguible

En un sistema asíncrono no se puede distinguir un proceso caído de uno lento. Esto no es un detalle de implementación: es un resultado teórico, y cualquier detector de fallos real es una heurística con dos errores posibles —declarar muerto a quien vive y creer vivo a quien murió—, y solo se puede elegir cuál de los dos se prefiere.

DFSha usa un detector por *timeout* con un estado intermedio deliberado, `ALIVE → SUSPECT → DEAD`, porque las consecuencias de los dos errores no son simétricas: sacar a un nodo de la colocación es barato y reversible, mientras que dar sus réplicas por perdidas dispara copias de disco que pueden saturar la red. El estado `SUSPECT` es, literalmente, el sitio donde se paga el error barato para no tener que pagar el caro.

Leases y *fencing tokens*

Un **lease** es un permiso con vencimiento: el titular puede actuar mientras no caduque, y si no lo renueva, otro puede tomarlo. Resuelve el problema de un titular que muere sin avisar, pero **no** resuelve el de un titular que se congela y despierta.

La respuesta clásica —y la que DFSha implementa en tres sitios distintos— es el **token de cercado** (*fencing token*): un número que **solo sube** y que viaja con cada operación, de modo que quien recibe la operación puede rechazarla si trae un número viejo. Es la idea que Lamport formula para los relojes lógicos y que los sistemas de almacenamiento modernos aplican a la exclusión mutua distribuida.

El mecanismo completo, y el argumento de por qué un lease sin token no basta, está en la sección 6.3. Es la pieza teórica más importante del proyecto, porque aparece tres veces —liderazgo del ControlNode, bloqueo de archivos del RF3, y por extensión en la autorización por bloque— y porque es el tipo de error que **no rompe nada visible** cuando falta.

Power of d choices

Repartir N elementos entre M nodos eligiendo cada destino al azar deja un desequilibrio que crece con N. Elegir el nodo **menos cargado de todos** reparte perfectamente, pero exige una foto global consistente y produce *efecto manada*: todas las peticiones concurrentes convergen sobre el mismo nodo “más vacío” y lo saturan.

El resultado conocido como *power of two choices* (y su generalización a d) dice que muestrear d candidatos al azar y quedarse con el menos cargado reduce el desequilibrio máximo de forma exponencial respecto a la elección puramente aleatoria, con un coste de consulta constante. DFSha usa d=3 (`DFSHA_PLACEMENT_D`), que es donde el rendimiento ya está cerca del óptimo y la ventana sigue siendo pequeña.

Quórum de escritura

Con R réplicas, exigir que las R confirmen antes de dar una escritura por buena convierte la disponibilidad de la escritura en el producto de las disponibilidades de todos los nodos: cuantas más copias, **menos** disponible. Exigir un quórum W < R rompe esa paradoja: se gana durabilidad sin perder disponibilidad, a cambio de que el sistema tenga que completar la copia que falta por su cuenta.

DFSha usa R=3 y W=2. Un archivo confirmado con 2 de 3 todavía tolera perder un nodo, y el hueco lo cierra el reconciliador. La argumentación completa está en la sección 7.4.

Arquitecturas de referencia consideradas

Referencia	Qué se tomó	Qué no, y por qué
GFS / HDFS	Bloques grandes, NameNode de metadato, bytes directos cliente ↔ DataNode, replicación en cadena, heartbeats con block report	El *append* atómico multi-escritor de GFS: añade un modelo de consistencia relajado que el enunciado no pide
BitTorrent / DHT	La idea de que el ancho de banda de subida del emisor es el recurso escaso, que es lo que justifica el pipeline	El enrutamiento por DHT y el descubrimiento P2P: la opción elegida es cliente/servidor, y una DHT añade indirección sin resolver el metadato
Sistemas de almacenamiento con quórum (Dynamo-like)	El quórum W < R y la reconciliación en segundo plano	El modelo de consistencia eventual con resolución de conflictos: con WORM no hay conflictos que resolver

1.4 Posición de DFSha frente al enunciado

El enunciado ofrecía dos opciones de arquitectura. DFSha implementa la **opción 1 (cliente/servidor con distribución servidor-a-servidor)**: el cliente obtiene un plan del plano de control y transfiere contra los nodos de datos, y la replicación entre nodos la hacen los propios nodos.

La **opción 2 (P2P puro)** se descartó con un argumento concreto y no por preferencia: el objetivo del curso incluye demostrar alta disponibilidad **del metadato**, y en un diseño P2P puro el metadato o se replica en todos los pares —y entonces el problema difícil es la consistencia, no el P2P— o vive en una DHT, y entonces hay que demostrar la disponibilidad de la DHT, que es el mismo problema con una capa más. La opción 1 permite atacar el problema del metadato de frente, que es donde están las decisiones interesantes.

2. Descripción del servicio y requisitos

2.1 Qué entrega DFSha

DFSha presenta al usuario un sistema de archivos jerárquico con un árbol propio por usuario, manipulable desde una CLI (`dfsha`) con comandos de semántica POSIX-like. Por debajo, cada archivo se parte en bloques de tamaño fijo, cada bloque se cifra en el cliente, se coloca en tres nodos de datos de dominios de falla distintos y se verifica con SHA-256 en cada salto.

Lo que el usuario ve:

```
dfsha register ana
dfsha login ana
dfsha mkdir -p /datos/pruebas
dfsha put /tmp/original.bin /datos/pruebas/original.bin
dfsha stat /datos/pruebas/original.bin
dfsha get /datos/pruebas/original.bin /tmp/bajado.bin
dfsha share /datos/pruebas beto read
```

Lo que ocurre por debajo de un solo `put`: resolución de permisos sobre el directorio destino, reserva del plan de escritura con elección de tres destinos por bloque, derivación de una clave de archivo, cifrado AES-256-GCM bloque a bloque, subida directa al primer nodo de cada cadena, reenvío encadenado a los otros dos con verificación de checksum en cada salto, confirmación síncrona de cada copia al ControlNode y, solo entonces, un commit transaccional que comprueba el quórum.

2.2 Requisitos funcionales

RF1 — Espacio de nombres

Operación	Comando	Implementación
Listar	`dfsha ls [ruta]`	`GET /fs/ls`, consulta pura, se sirve desde la réplica de lectura
Cambiar de directorio	`dfsha cd`, `dfsha pwd`	Estado **del lado del cliente**, en `~/.dfsha/session.json`
Crear directorio	`dfsha mkdir [-p]`	`POST /fs/mkdir`
Borrar directorio	`dfsha rmdir [-r]`	Borrado **lógico** (`deleted_at`); falla si no está vacío salvo `-r`
Borrar archivo	`dfsha rm`	Borrado lógico; los bloques quedan para el recolector
Mover / renombrar	`dfsha mv`	**Metadato puro, O(1)**: no se toca un solo byte en ningún DataNode
Metadatos	`dfsha stat`	Incluye estado de replicación por archivo

Dos decisiones merecen justificarse aquí:

`cd` y `pwd` viven en el cliente. El ControlNode no guarda sesión. Esto no es pereza: guardar el directorio de trabajo en el servidor obligaría a sesiones pegajosas en el balanceador, y entonces matar al ControlNode que atiende a un usuario le rompería la sesión. Como el estado vive en el cliente, cualquier ControlNode puede atender cualquier petición, que es la precondición de toda la alta disponibilidad de la sección 6.

`mv` es O(1) por diseño, no por casualidad. Es consecuencia directa de que el DataNode almacene por `block_id` y no conozca rutas lógicas. Si el DataNode guardara los bloques en una jerarquía que imitara el árbol del usuario, mover un directorio con mil archivos sería mover los bloques de mil archivos en cuatro nodos.

RF2 — Transferencia

Operación	Comando	Garantía
Subir	`dfsha put [--block-size N] [--parallel N]`	Particionado en bloques, cifrado en el cliente, R=3, commit con quórum W=2
Bajar	`dfsha get [--parallel N]`	Verificación SHA-256 por bloque, relevo automático entre réplicas

La escritura es **en tres fases** —reservar, subir, confirmar— y esa forma es consecuencia directa de WORM. La justificación está en la sección 4.3.

RF3 — Acceso a nivel de archivo

Operación	Comando	Nota
Abrir	`dfsha lock` / `open` interno	Abrir y bloquear ocurren **en la misma transacción**
Leer por rango	`dfsha read --offset N --length N`	Descarga **solo los bloques que intersectan** el tramo
Escribir	`dfsha append <remoto> <local>`	Append con reescritura del bloque de cola
Bloquear	`dfsha lock [--shared]`, `unlock`, `locks`	Lease de 120 s con **época**

RF4 — Seguridad (transversal)

Autenticación con JWT, autorización con ACLs y grupos, confidencialidad con cifrado extremo a extremo, identidad mutua entre servicios con mTLS, y autorización por bloque con firma asimétrica. Sección 8 completa.

2.3 Requisitos no funcionales

Requisito	Cómo se satisface	Dónde se demuestra
Particionamiento	Bloques de tamaño fijo con colocación *power of d choices* (d=3) y restricción de dominio de falla	§5, figura 8

Requisito	Cómo se satisface	Dónde se demuestra
Replicación	R=3 en cadena, quórum W=2, re-replicación automática	§7, figura 3
Alta disponibilidad	Tres ControlNodes sin estado tras balanceador, lease con época	§6, figura 5
Tolerancia a fallos	`ALIVE/SUSPECT/DEAD`, re-replicación con tres frenos, relevo entre réplicas en lectura	§6.5, §7.5, figura 9
Escalabilidad de lectura	CQRS con réplica de PostgreSQL y read-your-writes por LSN	§7.8
Escalabilidad de ancho de banda	Los bytes no atraviesan el plano de control	§3.4
Seguridad	Cinco capas independientes	§8
Observabilidad	Logging estructurado JSON desde el primer commit	§4.7
Reproducibilidad	Despliegue local y en AWS documentados paso a paso	§9

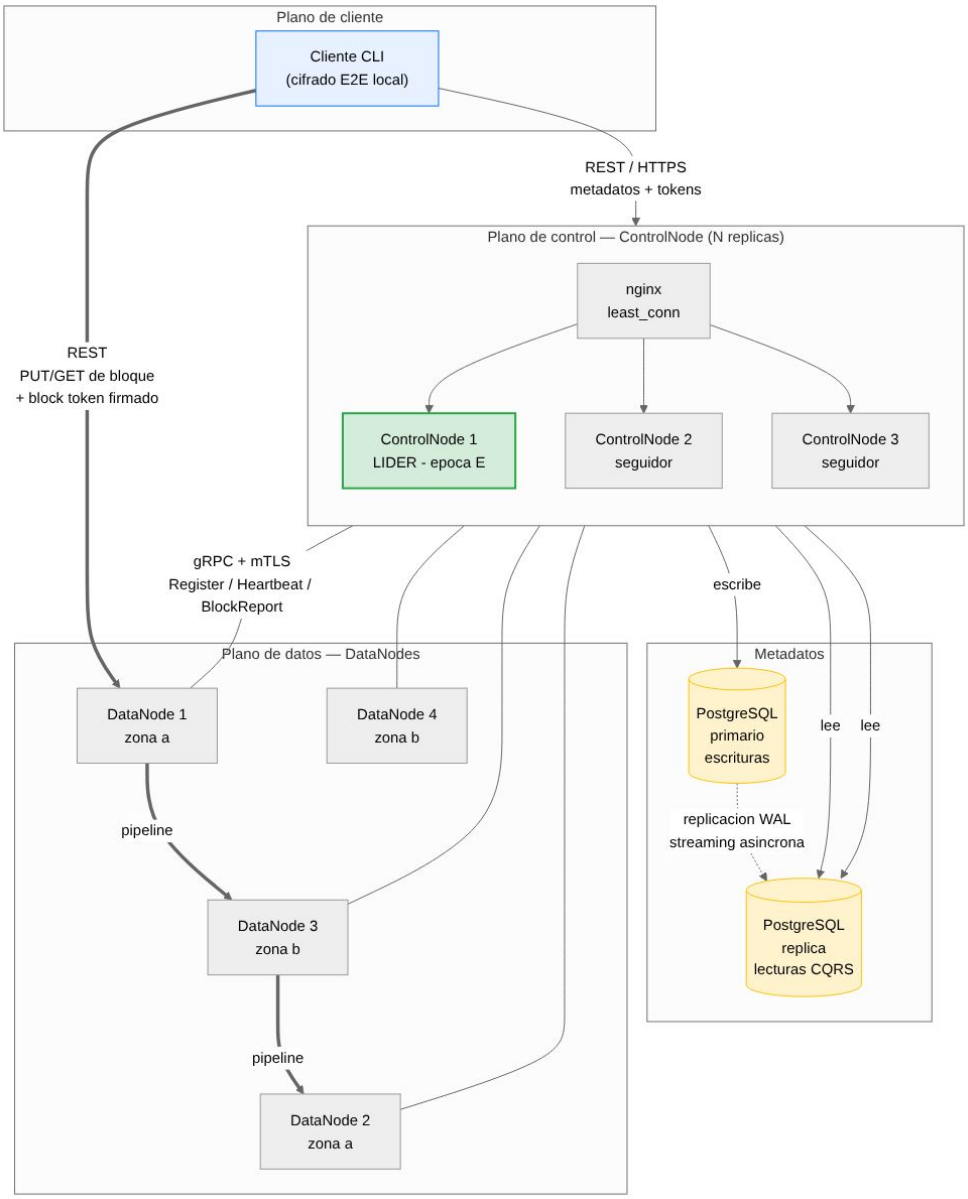
2.4 Alcance por hitos

Hito	Qué entregó
1	RF1 + RF2 con un ControlNode y un DataNode. JWT, SHA-256 por bloque, recolector manual, logging estructurado
2	N DataNodes reales. Plano de control sobre gRPC, heartbeats de 3 s, `ALIVE/SUSPECT/DEAD`, *power of d choices* con dominios de falla, detección de divergencia sin borrado automático
3	Bloque A: PostgreSQL con réplica, Alembic, tres ControlNodes y liderazgo con época. Bloque B: R=3 con pipeline, W=2, re-replicación. Bloque C: mTLS, cifrado extremo a extremo, ACLs, token de bloque, RF3, TLS de cliente
4	Despliegue en AWS validado, informe y video

Esta progresión no es cronología: es la estructura del argumento. Cada hito elimina un supuesto del anterior, y las "costuras" que cada etapa dejó abiertas —el stream gRPC declarado bidireccional en el hito 2 aunque solo se usara en un sentido, la interfaz de colocación con el factor de replicación ya en la firma, la separación `commands/` / `queries/` desde el hito 1— son las que permitieron que el hito 3 fuese ampliación y no reescritura.

3. Arquitectura

3.1 Vista general



Arquitectura general de DFSha. La línea gruesa es el camino de los datos; las finas, el plano de control.

Cuatro componentes y una base de datos:

Componente	Responsabilidad	Estado propio
Cliente (CLI)	Partir, cifrar, subir y bajar bloques; resolver rutas relativas; guardar la sesión	Token JWT, cwd, clave maestra, LSN de escritura — todo en `~/.dfsha/`
ControlNode (N instancias)	Namespace, colocación, autorización, emisión de tokens de bloque, pertenencia del clúster	**Ninguno.** Todo en PostgreSQL

Componente	Responsabilidad	Estado propio
DataNode (N instancias)	Almacenar bloques por `block_id`, reenviar en cadena, latir, ejecutar órdenes	Sus bloques en disco y su `boot_id`
nginx	Balancear tres puertos con tres políticas distintas	Ninguno
PostgreSQL	Metadato: primario de escritura y réplica de lectura	Todo el estado durable del sistema

3.2 Las cuatro decisiones estructurales

Estas cuatro decisiones no se revisan en ninguna etapa posterior: todo lo demás se construyó encima de ellas, y cambiarlas sería reescribir el sistema.

Decisión 1 — WORM con fachada CRUD

Ya argumentada en §1.3. Lo que importa aquí es la consecuencia arquitectónica: **ninguna réplica puede divergir de otra**, porque no hay escrituras concurrentes sobre un bloque cerrado. Todo el aparato de reconciliación que necesitaría un sistema mutable —vectores de versión, resolución de conflictos, *last-writer-wins*— simplemente no existe en DFSha, y no existe por una decisión tomada en la primera semana.

Decisión 2 — El DataNode no conoce rutas lógicas

El DataNode almacena en un *layout* plano con sharding por los dos primeros caracteres del `block_id`:

```
$DFSHA_DATA_DIR/blocks/<2 primeros chars>/<block_id>.blk
$DFSHA_DATA_DIR/blocks/<2 primeros chars>/<block_id>.meta
```

El mapeo `ruta → [block_id]` vive **solo** en el ControlNode.

Qué compra: `mv` y `rename` son O(1) y no tocan un byte de datos; un DataNode se puede añadir, quitar o reemplazar sin que el árbol de nadie se entere; y el sharding por prefijo evita el directorio con cien mil entradas que degrada cualquier sistema de archivos anfitrión.

Qué cuesta, y esto es lo interesante: crea una obligación de seguridad en otro sitio. Como el DataNode no sabe de rutas ni de usuarios, **no puede** comprobar un permiso. Si no se hace nada más, `GET /blocks/{id}` sirve a cualquiera que sepa el identificador, y las ACLs del ControlNode son ciertas solo a medias. Esa deuda se paga en la sección 8.8 con el token de bloque, y conviene decir que la deuda la **creó esta decisión**: es el mejor ejemplo del proyecto de que una decisión de arquitectura no se evalúa sola.

Decisión 3 — Colocación explícita, no por hash

El ControlNode elige el DataNode destino de cada bloque y **registra la elección** en `block_replicas`. La ubicación jamás se recalcula aplicando una función al `block_id`.

La alternativa —hash consistente, como en una DHT— tiene una propiedad atractiva: no hay que guardar dónde está nada, se calcula. Y tres problemas que la descartan para este sistema:

1. **No puede tener en cuenta la carga real.** Un hash reparte uniformemente por construcción, lo cual suena bien hasta que los nodos tienen capacidades distintas o uno se llena por otro motivo. La política de DFSha consulta el espacio libre real, reportado por heartbeat.
2. **No puede respetar dominios de falla.** Garantizar que las tres copias de un bloque caigan en tres dominios distintos con un hash exige o bien un hash por dominio —y entonces ya se está llevando una tabla— o bien aceptar que a veces no se cumple.
3. **Cualquier cambio en el conjunto de nodos remapea bloques.** El hash consistente acota cuántos, pero no evita que entrar o salir un nodo mueva datos. Con colocación explícita, añadir un nodo no mueve nada: simplemente empieza a recibir bloques nuevos.

El precio de la decisión 3 es que hay que **acordarse de registrar la elección siempre**, y ese precio se cobró literalmente: la primera versión de la re-replicación elegía el nodo destino y no insertaba la fila `PENDING`, con el resultado de que el nodo copiaba el bloque correctamente, avisaba, y recibía un `404 bloque desconocido en el metadato`. Está contado en §7.5.

Decisión 4 — Routing híbrido: el plano de control no está en el camino de los datos

El ControlNode es *routing tier* **solo para metadatos**. Los bytes viajan directos entre cliente y DataNode.

Es la decisión que determina la escalabilidad del sistema. Si los bytes pasaran por el ControlNode, añadir DataNodes no añadiría ancho de banda: el ControlNode sería el techo, y sería además el componente que no se puede replicar fácilmente porque es el que tiene el estado. Con el routing híbrido, cada DataNode que entra suma su enlace al total.

Las consecuencias se notan en todo el resto del informe:

- El cliente necesita un **plan** antes de transferir, lo que fuerza la escritura en tres fases (§4.3).
- El permiso tiene que viajar **firmado** hasta el DataNode (§8.8).
- El DataNode tiene que anunciar una dirección **alcanzable por el cliente**, lo que en un despliegue con redes distintas se convierte en dos direcciones (§3.3).
- El `append` del RF3 lo tiene que hacer el cliente, porque el servidor no podría aunque quisiera: concatenar exige descifrar, y la clave no sale del cliente (§7.10).

3.3 Dos direcciones por DataNode

Hasta el hito 2, **solo el cliente hablaba con los DataNodes**, así que una dirección anunciada por nodo bastaba. El hito 3 introdujo el pipeline de replicación y la re-replicación, y con ellos **los DataNodes empezaron a hablar entre sí**. El conjunto de "quién tiene que alcanzar este nodo" creció, y los dos grupos están en redes distintas.

El síntoma, descubierto al validar en Docker y no en las pruebas: `dfsha put` fallaba con `409 no alcanzan el quórum de escritura (W=2)` y `dfsha cluster` mostraba 50 bloques repartidos pero **50 réplicas en total en vez de 150**. En los logs del DataNode:

```
{"next_hop": "http://localhost:8002", "error": "ConnectError",
 "error_detail": "[Errno 111] Connection refused"}
```

El plan llevaba la dirección válida **desde el host** (`localhost:800N`) por un camino **entre contenedores**, donde `localhost` resuelve al propio contenedor que reenvía.

La solución: el DataNode anuncia dos direcciones.

Variable	Alcanzable por	Si está vacía
`DFSHA_DATANODE_ADVERTISE_URL`	El **cliente**	— (obligatoria)
`DFSHA_DATANODE_PEER_URL`	Otros **DataNodes**	La misma que la anterior

Esto no contradice la decisión del hito 2 de no anunciar dos direcciones, y la diferencia es fina pero es la que hay que defender. Lo que el hito 2 rechazó fue que el ControlNode **infiriera** la dirección según el **origen de la petición**: eso mete en el plano de control una suposición sobre la topología de red del cliente, que se rompe con NAT, túneles o varias interfaces. Aquí **no hay ninguna inferencia**: las dos direcciones son estáticas y su destinatario se sabe por la **estructura del mensaje**, no por quién llama. El plan del cliente lleva siempre `advertise_url`; la cadena del pipeline lleva siempre `peer_url`. El ControlNode no elige: manda las dos, y cada una va en su sitio.

Inventario completo de qué dirección va en cada camino —es completo porque **el ControlNode no llama nunca a un DataNode**: todo su tráfico hacia ellos viaja por el stream gRPC que el nodo abre:

Camino	Dirección	Por qué
Plan de escritura y de lectura (`replicas[].base_url`)	**cliente**	Los bytes van directos del cliente al DataNode
Cadena del pipeline (`pipeline[]`)	**par**	La recorre un DataNode reenviando a otro
`source_base_url` de una orden de re-replicación	**par**	El nodo destino descarga del origen
`dfsha cluster`, mensajes de error	**cliente**	Es la que una persona puede probar con `curl`
GC por REST	**cliente**	Lo consume el script, que corre donde el cliente
GC por el canal de control	*ninguna*	La orden viaja por el stream del propio nodo

3.4 CQRS: por qué separar comandos de consultas desde el primer día

La separación entre `commands/` (mutan el namespace) y `queries/` (lo leen) existe desde el hito 1, cuando ambos golpeaban la misma base SQLite y la separación no compraba nada funcional.

Se hizo igualmente porque **el coste de introducirla después habría sido reescribir todos los casos de uso**, y porque es la condición previa de poder enrutar las lecturas a otra base. En el hito 3 se cobró: añadir una réplica de PostgreSQL consistió en enrutar un conjunto de funciones ya identificado, no en auditar el código buscando cuáles leían y cuáles escribían.

Es un argumento general que vale la pena enunciar: **una separación arquitectónica que no compra nada hoy puede ser exactamente lo que hace barata una decisión futura**, y la forma de saber si vale la pena es si se puede nombrar la decisión futura concreta. Aquí se podía: estaba en la hoja de ruta desde el principio.

3.5 Estructura del código

```
src/dfsha/
├── common/              DTOs, checksum, errores, logging
│   ├── crypto.py        las tres capas de clave del cifrado extremo
a extremo
│   ├── blocktoken.py    autorización por bloque que el DataNode
verifica solo
│   ├── serve.py         arranque con TLS de cliente opcional
│   ├── tls.py           ÚNICO sitio donde se construye un contexto
TLS de cliente
│   └── proto/           control.proto (el código generado no se
versiona)
├── control_node/
│   ├── api/             routers FastAPI: solo traducción HTTP ↔
casos de uso
│   ├── commands/        lado escritura (CQRS)
│   ├── queries/         lado lectura (CQRS)
│   ├── domain/          Path, File, Block, User, Lease, acl,
filelock
│   ├── repositories/    modelos SQLAlchemy, interfaces, unidad de
trabajo
│   └── services/        auth, placement, leadership, read_routing,
access, shared
├── data_node/           storage.py, heartbeat.py (cliente gRPC),
runtime.py, api/
└── client/              cli.py, session.py, chunker.py,
transfer.py, tls.py
alembic/versions/        ocho migraciones del metadato
docker/nginx/            nginx.conf, dfsha.conf, dfsha-stream.conf,
dfsha-tls.conf
scripts/                 gc.py, gen_certs.py, gen_proto.py,
gen_testfile.py,
                         verificar_pruebas.py, demo/
deploy/                  material de despliegue en AWS + RUNBOOK-
postgres.md
tests/                   unit/, integration/
```

Tres reglas transversales que el código respeta sin excepción:

1. **Acceso a datos detrás de repositorios.** No hay SQL suelto en los routers.
2. **Los routers solo traducen HTTP ↔ casos de uso.** Las reglas de dominio viven en `domain/`, probadas sin base de datos.
3. **Un solo sitio por decisión transversal.** `common/tls.py` es el único lugar donde se construye un contexto TLS de cliente, y `client/tls.py` el único que decide cómo verificar HTTPS. No es estética: cuando apareció la trampa de httpx descrita en §10.6, el arreglo fue una función y no una búsqueda por todo el repositorio.

4. Protocolos y APIs

4.1 Por qué dos protocolos y no uno

DFSha usa **gRPC solo entre ControlNode y DataNode**. Cliente ↔ ControlNode y cliente ↔ DataNode siguen en REST. Es una decisión razonada, no una migración a medias:

	Tráfico de control	Tráfico de cliente
Qué transporta	Mensajes pequeños de esquema fijo	Bytes crudos de hasta 64 MB
Frecuencia	Uno cada 3 s **por nodo**, para siempre	Esporádico, por sesión
Conexión	Conviene mantenerla abierta durante horas	Peticiones cortas e independientes
Quién lo depura	El desarrollador, con el código generado delante	Cualquiera, con `curl`
Elección	**gRPC**: Protobuf y HTTP/2 pagan en bytes, handshakes y tipos revisados por el compilador	**REST**: legible, herramientas estándar, documentación interactiva

Meter la transferencia de bloques en gRPC añadiría una capa de opacidad sobre un `PUT` de bytes sin ganar nada: el cuerpo sería igual de grande y ya no se podría reproducir un problema con `curl`.

Solo migran tres RPC: `Register`, `Heartbeat` y `BlockReport`.

4.2 ControlNode `/api/v1` — lo consume el cliente

```
POST /auth/register   {username, password}
     -> 201

POST /auth/login      {username, password}
     -> 200 {access_token, expires_in, token_type, kdf_salt}
     -> 401 con el MISMO mensaje para usuario inexistente y
contraseña
        incorrecta: distinguirlos permitiría enumerar usuarios
```

`kdf_salt` viaja en la respuesta del login **a propósito, y es la única vía**: el cliente la necesita para derivar la clave maestra en cada sesión nueva (§8.5). No es secreta —su trabajo es que dos usuarios con la misma contraseña no compartan clave— y el servidor nunca ve lo que se deriva de ella.

Todo lo demás exige `Authorization: Bearer <token>`.

Namespace (RF1):

Método	Ruta	Cuerpo / parámetros	Respuesta
`GET`	`/fs/ls`	`?path=/a/b`	`{entries: [{name, type, size, created_at}]}`

Método	Ruta	Cuerpo / parámetros	Respuesta
GET	/fs/stat	?path=/a/b/c	{path, type, size, block_size, block_count, created_at, replication_state, min_replicas, max_replicas, replication_factor}
POST	/fs/mkdir	{path, parents}	201
DELETE	/fs/rmdir	?path=&recursive=	204
DELETE	/fs/rm	?path=	204
POST	/fs/mv	{src, dst}	204; 409 si dst es un archivo existente

mv nunca sobrescribe en silencio y nunca crea el directorio padre implícitamente. Las dos cosas son decisiones: un sistema de archivos que borra datos porque el usuario escribió mal un destino es peor que uno que falla.

Los cuatro últimos campos de stat son null en un directorio. replication_state es FULLY_REPLICATED, UNDER_REPLICATED o UNAVAILABLE, y se deriva del bloque **peor** replicado en el momento de la consulta; min_replicas y max_replicas son los que hacen visible el rango 3-4 de 3 de la sobre-replicación de §7.7, que un número plano escondería.

Transferencia (RF2):

```
POST /files/create   {path, size, block_size?, cipher_overhead?}
     -> 201 {file_id, block_size, expires_at,
             blocks:[{block_id, index, size,
                      replicas:[{data_node_id, base_url}],
                      pipeline:[peer_url, ...]}]}

POST /files/{file_id}/commit  {wrapped_key?, key_algo?}
     -> 200 {path, size, block_count}
     -> 409 si algún bloque no alcanza el quórum
     -> 410 si la reserva venció

POST /files/{file_id}/abort   -> 204

GET  /files/open?path=/a/b/c
     -> {file_id, size, block_size, wrapped_key, key_algo,
```

```
         blocks:[{block_id, index, size, checksum_sha256,
                  replicas:[{data_node_id, base_url}]}]}
```

RF3 — apertura, bloqueo, lectura por rango y escritura incremental:

```
POST /fs/open     {path, mode, lock, holder}
     -> 200 {file_id, path, size, block_size, mode,
             wrapped_key, key_algo, lock}
        lock = un LockResponse, o null si no se pidio bloqueo
     -> 400 ruta invalida
     -> 404 no existe o no es tuyo
     -> 409 file_locked

POST /fs/lock     {path, mode, holder}     mode: shared | exclusive
     -> 200 {file_id, path, holder, epoch, mode,
             expires_at, lease_seconds}
     -> 400 · 404 · 409 file_locked

POST /fs/unlock   {path, mode, holder}
     -> 204 · 400 · 404

GET  /fs/locks?path=/a/b
     -> 200 {path, holders:[{holder, mode, epoch, expires_at}]}
     -> 400 · 404

GET  /files/read?path=&offset=0&length=
        length ausente = hasta el final del archivo
     -> 200 {file_id, offset, length, size,
             blocks:[{block_id, index, size, checksum_sha256,
                      replicas:[...], token, skip, take}]}
     -> 400 · 404

POST /files/{file_id}/append
        {size, cipher_overhead, lock_holder, lock_epoch}
     -> 201 {file_id, block_size, expires_at,
             tail, tail_plain_size,
             blocks:[{block_id, index, size,
                      replicas, pipeline, token}]}
        tail = plan de lectura del bloque de cola, o null
     -> 404
     -> 409 invalid_state | file_locked | stale_lock
     -> 503 sin DataNode disponible

POST /files/{file_id}/append/commit
        {block_ids, new_size, replaces, lock_holder, lock_epoch}
     -> 200 {path, size, block_count}
     -> 404 · 409 blocks_not_stored | stale_lock
```

El reparto de prefijos no es intuitivo y conviene señalarlo: **el handle y los bloqueos viven bajo `/fs`** —son operaciones sobre el espacio de nombres— mientras que **la lectura por rango y el `append` viven bajo `/files`**, porque son transferencia. Un `curl` contra `/files/lock` devuelve 404.

Seis detalles del contrato que solo se ven leyendo el código y que sostienen lo que argumenta §7.9:

- **Adquirir y renovar son la misma ruta.** `/fs/lock` renueva si el `holder` ya lo tenía, porque un cliente que reintenta tras un timeout de red no puede saber si el primer intento llegó.
- **Un `holder` vacío cae al `user_id`**, en `open`, `lock` y `unlock`.
- **`unlock` exige READ, no WRITE, y es idempotente**: soltar lo que ya no tienes también es `204`.
- **`/fs/locks` se lee sin cerrojo** y filtra los vencidos al responder, porque el vencimiento se evalúa al consultar y no con un barrido.
- **`skip` y `take` se aplican al bloque ya descifrado.** El bloque se descarga entero, por el motivo criptográfico de §7.9.
- **Un `409` de lock trae `holder`, `mode` y `retry_after_seconds` en `details`**, que es exactamente el dato que el cliente descartaba hasta que una prueba lo exigió (§10.6).

ACLs, compartición y grupos — todo bajo el prefijo `/acl`, no bajo `/fs`:

```
POST /acl/share     {path, principal, permission}
     -> 204 · 400 · 404 · 409

POST /acl/unshare   {path, principal}
     -> 204 · 400 · 404

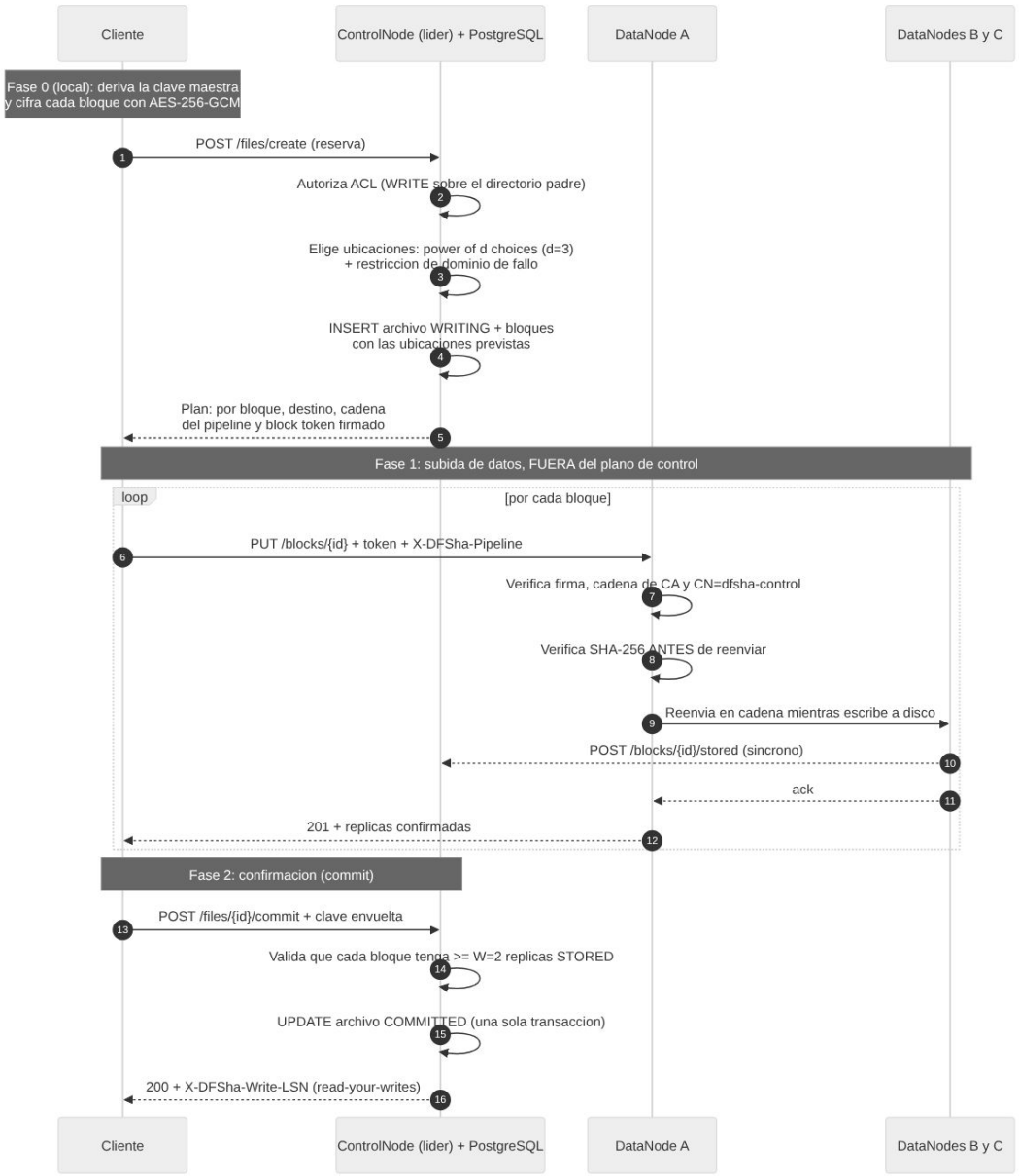
GET  /acl/show?path=/a/b
     -> 200 {path, effective, source, inherited_from,
             grants:[{principal, principal_type, permission,
                      granted_by, granted_at}]}

GET  /acl/shared-with-me
     -> 200 {entries:[{owner, name, permission, via_group, path}]}

POST /acl/groups                {name}
     -> 201 · 409 ya existe
GET  /acl/groups
     -> 200 {groups:[{name, members}]}
POST /acl/groups/members        {name, username}
     -> 204 · 404
POST /acl/groups/members/remove  {name, username}
     -> 204 · 404
```

`effective` es `READ`, `WRITE`, `ADMIN` o `null`; `inherited_from` es la ruta del ancestro del que sale el permiso, y `via_group` dice por qué grupo llega la concesión o `null` si es directa. Esos dos campos son la herencia hacia arriba de §8.6 hecha inspeccionable: sin ellos, un usuario vería *qué* permiso tiene pero no *de dónde* le viene, y un modelo sin denegaciones en el que no se puede ver el origen de una concesión deja de ser explicable.

4.3 La escritura en tres fases, y por qué tiene que ser así



Secuencia completa de una escritura: reservar, subir, confirmar.

Un `PUT` de archivo en una sola llamada sería más simple de usar. No se puede, y el motivo encadena tres decisiones anteriores:

1. Por la **decisión 4**, los bytes no pasan por el ControlNode. Luego el cliente necesita saber **a qué nodos** enviar cada bloque antes de enviar nada: eso es la fase de reserva.
2. Por la **decisión 3**, esa elección la toma y la registra el ControlNode, no se deduce. Luego la reserva tiene que escribir metadato, y ese metadato describe un archivo que todavía no existe: de ahí el estado `WRITING`.
3. Por la **decisión 1** (WORM), el archivo no puede hacerse visible hasta que todos sus bloques estén escritos y comprobados. Luego hace falta un momento explícito en el que el sistema valide el conjunto y lo publique: eso es el commit.

Las tres fases no son un protocolo elegido entre varios: son lo que queda cuando se respetan las tres decisiones a la vez.

Lo que cada fase garantiza:

- **Reserva.** Autoriza (WRITE sobre el directorio destino), elige destinos con la política de colocación, escribe el archivo en `WRITING` con `expires_at`, y devuelve un plan que incluye un **token de bloque firmado por bloque**. Un cliente que muera aquí no bloquea el nombre: la reserva vencida se marca `DELETED` de forma perezosa cuando otro `create` pide la misma ruta.
- **Subida.** Fuera del plano de control por completo. Cada bloque se sube a un solo nodo y se propaga en cadena. El nodo avisa al ControlNode **de forma síncrona y antes de responder 201**, de modo que cuando el cliente puede pedir el commit, el ControlNode ya conoce esas copias.
- **Commit.** Valida que cada bloque tenga al menos W réplicas en estado `STORED`, almacena la clave envuelta, y publica el archivo **en una sola transacción**. Si había un archivo anterior en esa ruta, pasa a `DELETED` en esa misma transacción: hacerlo en dos pasos dejaría, ante una caída intermedia, una ruta sin archivo visible.

4.4 El plano de control sobre gRPC

Definido en `control.proto`; el código se genera con `python scripts/gen_proto.py` y **no se versiona**, porque código generado en el repositorio envejece respecto de su fuente sin que nada lo detecte.

RPC	Tipo	Para qué
`Register`	unario	Alta del nodo: las dos URLs, dominio de falla, `boot_id`, capacidad
`Heartbeat`	**stream bidireccional**	Latido cada 3 s con métricas de carga y block report incremental. El ControlNode responde `Ack` y, por el mismo stream, empuja `FullReportReq`, `ReplicateBlock` y `DeleteBlock`
`BlockReport`	unario	Report completo, fuera de banda porque puede ser grande

El stream se declaró bidireccional en el hito 2 aunque entonces el ControlNode solo empujara `FullReportReq`. Esa previsión es lo que hizo que las órdenes de re-replicación del hito 3 fueran *un caso más en el `oneof`* y no un transporte nuevo.

Cuatro reglas del protocolo de pertenencia que merecen argumentarse:

1. **Un report incremental NUNCA marca `MISSING`.** El nodo solo manda lo que cambió, así que la ausencia de un bloque no significa nada. Concluir "falta" desde un incremental daría por perdido el disco entero en cada latido. Solo un report **completo** autoriza esa conclusión. Blindado en `test_report_incremental_nunca_marca_missing`.

2. **Un report completo por cada reconexión.** El contador de "cada N latidos" vive en el DataNode; un stream que se rompe antes del latido N lo deja a medias, y con una red inestable podría no mandarse un completo nunca. El ControlNode pide `FullReportReq` en el primer latido de cada stream.
3. **El estado es derivado, no almacenado.** `SUSPECT` se calcula del último latido cada vez que se consulta. La columna persistida solo existe para detectar la transición y emitir el evento; si las dos discrepan, manda lo derivado. Así no puede haber deriva silenciosa entre lo que el sistema cree y lo que es.
4. **El `boot_id` vive dentro del volumen de datos**, junto a los bloques. Perder el volumen pierde el `boot_id`, que es justo lo que el ControlNode necesita saber para dar las réplicas por perdidas. Guardarlo fuera mentiría exactamente en ese caso.

4.5 Por qué `/blocks/{id}/stored` se quedó en REST

Es la excepción que confirma el criterio de §4.1, y la razón es de latencia, no de estilo.

El DataNode llama a este endpoint **de forma síncrona antes de responder 201 al cliente**. Si esa confirmación viajara en el block report incremental del siguiente heartbeat, un `commit` inmediato podría llegar hasta 3 segundos **antes** que la noticia y fallar con un 409 por una carrera que el usuario no puede ni entender ni evitar.

La latencia del commit no puede quedar atada al periodo del heartbeat. Ese es el argumento completo, y es el que explica por qué un endpoint del plano interno no migró con los otros tres.

4.6 DataNode `/api/v1`

Método	Ruta	Detalle
`PUT`	`/blocks/{block_id}`	Bytes crudos + `X-DFSha-Checksum` + `X-DFSha-Block-Token` + `X-DFSha-Pipeline`. `201`; `422` si el checksum no cuadra; `409` si el bloque ya existe
`GET`	`/blocks/{block_id}`	Bytes crudos + `X-DFSha-Checksum`. Exige token con operación `read`
`DELETE`	`/blocks/{block_id}`	`204`, idempotente. Exige token con operación `delete`
`GET`	`/health`	`{status, used_bytes, capacity_bytes, block_count, disk_free_bytes, data_node_id, fault_domain, boot_id}`

Tres detalles que parecen menores y no lo son:

- **Los bloques son inmutables**: reescribir un `block_id` existente es `409`, nunca una sobrescritura silenciosa.
- **`used_bytes` y `block_count` se calculan del estado real en disco**, no de un contador en memoria, porque la política de colocación depende de esos números.
- **`disk_free_bytes` sale de `shutil.disk_usage`, nunca de `capacity_bytes - used_bytes`.** Si el disco se llena por logs, por la base de datos o por otro contenedor, la resta miente y la colocación mandaría bloques a un nodo que no puede recibirlos.

`/health` es el único endpoint que no exige token de bloque: no revela contenido y es lo que consultan las sondas del despliegue.

4.7 Logging estructurado como contrato observable

Una línea JSON por evento a stdout, con `timestamp`, `level`, `service`, `event` y `duration_ms`. No es decorativo: es la evidencia con la que se miden los resultados de la sección 10, y es lo que permitió diagnosticar la mayoría de los fallos de §10.6.

Eventos por capa:

Capa	Eventos
Transferencia	`block.upload`, `block.download`, `file.put`, `file.get`
CQRS	`metadata.command`, `metadata.query` — la separación tiene que verse en las trazas
Pertenencia	`node.registered`, `node.state_changed`, `heartbeat.received`
Colocación	`placement.selected`, `placement.domain_relaxed`, `placement.insufficient_candidates`
Divergencia	`divergence.unknown_block`, `divergence.missing_block`
Liderazgo	`leadership.acquired`, `leadership.lost`, `leadership.renewed` (DEBUG), `leadership.epoch_rejected`
Enrutado	`query.routed_to_primary` con `reason`, `client_lsn`, `replica_lsn`
Replicación	`replication.pipeline`, `replication.partial`, `replication.quorum_met/failed`
Re-replicación	`rereplication.scheduled/ dispatched/completed/failed/ expired`, `block.replica_failed`
Seguridad	`block.token_rejected` con `reason`

Capa	Eventos
RF3	`lock.acquired/renewed/released`, `lock.epoch_rejected`, `file.append_planned`, `file.appended`

Dos decisiones de nivel que vale la pena justificar:

`leadership.renewed` va a DEBUG y no a INFO. Con un intervalo de renovación de 2 s y tres instancias son 1,5 líneas por segundo para siempre, y ahogarían los eventos que sí cuentan algo. Los sucesos de verdad —adquisición, pérdida, rechazo por época— van a INFO y WARNING.

El motivo del rechazo de un token va al log y no a la respuesta. Hacia fuera, todo rechazo es el mismo `403` con el mismo texto; quien prueba tokens no debería recibir pistas sobre cuál de sus intentos estuvo más cerca. Quien diagnostica tiene el motivo exacto en `block.token_rejected`.

5. Particionamiento y distribución

5.1 Bloques de tamaño fijo

Cada archivo se parte en bloques de `DFSHA_BLOCK_SIZE` bytes (64 MB por defecto), todos llenos salvo el último. El tamaño es configurable por despliegue y también por subida (`dfsha put --block-size N`), lo cual existe por un motivo práctico que conviene decir: **permite demostrar el particionamiento sin mover 3 GB**. Un archivo de 50 MB con bloques de 1 MB da 50 bloques y 150 réplicas, que es suficiente para que el reparto sea estadísticamente visible.

El invariante "todos llenos salvo el último" es lo que mantiene simples la colocación, la contabilidad de tamaños y la lectura por rango. Es también lo que obliga a que el `append` reescriba el bloque de cola, con el coste que se argumenta en §7.10.

5.2 La política: *power of d choices* con dominio de falla

La selección de destinos de un bloque, en orden:

```
1. Filtrar candidatos:
     - estado ALIVE (SUSPECT y DEAD quedan fuera de la colocación)
     - disk_free_bytes - tamaño_del_bloque >= DFSHA_MIN_FREE_BYTES
     - aún no elegido para ESTE bloque
2. Muestrear d = 3 candidatos al azar
3. Quedarse con el menos cargado de los d (por fracción de uso, no
por bytes absolutos)
4. Repetir R = 3 veces, con la restricción de dominio de falla
```

Por qué fracción de uso y no bytes libres absolutos. Un nodo con 500 GB libres de 1 TB y otro con 500 GB libres de 10 TB no están igual de cargados, y el que tiene el disco más grande debería recibir más. Comparar la fracción lo resuelve sin tratar los nodos heterogéneos como un caso especial.

Por qué d=3 y no "el menos cargado". Elegir el mínimo global exige una foto consistente de todos los nodos y produce **efecto manada**: todas las peticiones concurrentes convergen sobre el nodo que acaba de reportarse más vacío y lo saturan, precisamente porque estaba libre. Muestrear una ventana pequeña rompe esa convergencia con un coste de consulta constante, y el resultado teórico dice que la mejora respecto al azar puro es exponencial ya con d=2.

Por qué d=3 y no d=2. Con cuatro nodos, d=2 muestrea la mitad del clúster por decisión, que empieza a parecerse a mirar el global. d=3 mantiene la aleatoriedad suficiente con una ventana que sigue sin ser el conjunto entero. Es configurable (`DFSHA_PLACEMENT_D`) porque el valor correcto depende del número de nodos.

5.3 Dominios de falla

Cada DataNode declara un `DFSHA_DATANODE_FAULT_DOMAIN`: una **cadena opaca** de la que el sistema solo compara igualdad. En el compose local son `local-1..4`; en AWS, las zonas de disponibilidad reales.

Que sea opaca es deliberado. El sistema no necesita saber qué significa —rack, zona, bastidor, fuente de alimentación— y no debería: el operador sabe qué se cae junto en su infraestructura, y la política solo necesita saber qué no debe coincidir.

Dos reglas, y la asimetría entre ellas es la decisión:

1. **Se intenta siempre un dominio distinto por réplica.** Si no quedan dominios libres, se **relaja** la restricción y se registra `placement.domain_relaxed`. Negarse a colocar sería preferir no tener la tercera copia a tenerla en un dominio repetido, y eso es peor en todos los escenarios.
2. **Nunca dos réplicas en el mismo `data_node_id`.** Esta **no** se relaja jamás. Dos copias en el mismo disco se pierden juntas: no son dos copias.

5.4 El recuento dentro de la misma llamada

Los bloques de un `put` se colocan **todos en una única llamada** a `/files/create`, con una sola foto de carga. Si la política no contara lo que ya asignó dentro de esa misma llamada, con d=3 y cuatro nodos iguales el cuarto nodo **no entraría en la ventana ni una vez**: el efecto manada que el *power of d* evita entre peticiones reaparecería dentro de una.

Es un detalle de implementación con consecuencia medible, y es la diferencia entre el reparto 11/13/14/12 que se midió y un reparto donde un nodo se queda a cero.

5.5 Divergencia: detectar sin borrar

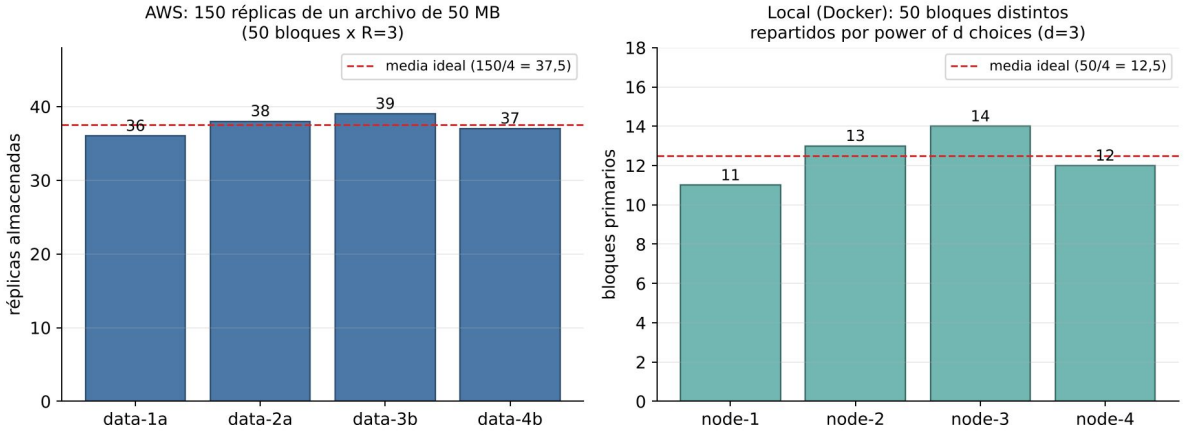
Cuando el block report de un nodo no coincide con lo que el metadato dice de él, hay dos casos y se tratan de forma **deliberadamente asimétrica**:

Caso	Evento	Acción
El nodo tiene un bloque que el metadato no le asocia	`divergence.unknown_block`	Queda para el recolector, que es **manual**
El metadato dice que el nodo tiene un bloque y el report completo no lo lista	`divergence.missing_block`	La réplica pasa a `MISSING`; **no se borra nada**

El ControlNode no borra datos por una divergencia. El argumento cabe en una línea: un bloque de más cuesta disco; uno de menos cuesta datos. Las dos cosas no son comparables, y cuando no son comparables no hay que buscar un equilibrio, hay que elegir el lado seguro siempre.

Esta decisión del hito 2 tiene consecuencias que reaparecen en §7.7: la sobre-replicación tras una reincorporación **no se limpia sola**, y es exactamente por esta regla.

5.6 Resultados medidos



Reparto de bloques y réplicas medido en los dos despliegues.

Despliegue	Qué se midió	Resultado	Desviación máxima
AWS, 4 DataNodes en 2 zonas	50 bloques × R=3 = 150 réplicas	**36 / 38 / 39 / 37**	1,5 réplicas sobre 37,5 (4 %)
Local, 4 DataNodes en 4 dominios	50 bloques (primarios)	**11 / 13 / 14 / 12**	1,5 bloques sobre 12,5 (12 %)

Lo que el número demuestra y lo que no: **demuestra que la política funciona y que ningún nodo queda fuera del reparto**; no demuestra optimalidad, porque con 50 bloques la varianza esperada de cualquier política razonable es de este orden. El criterio del enunciado —que ningún nodo se lleve más de una fracción desproporcionada— se cumple con holgura, y la prueba de integración lo fija en que **ninguno supere el 60 %**.

El reparto en AWS es más uniforme que el local, y tiene explicación: en AWS cada bloque genera tres decisiones de colocación en lugar de una, así que la ley de los grandes números tiene tres veces más muestras sobre las que operar.

6. Alta disponibilidad

6.1 Qué había que eliminar

Al cerrar el hito 2 quedaban dos puntos únicos de fallo:

1. **Un solo ControlNode.** Si se caía, el clúster entero quedaba sin namespace, sin autenticación y sin plan de escritura. Los bytes seguían en los discos y nadie podía pedirlos.
2. **El metadato en SQLite, en un fichero local.** Dos ControlNodes en máquinas distintas no pueden compartir un fichero, así que mientras el metadato viviera en SQLite no había forma de tener un segundo ControlNode. **Sin base compartida no hay alta disponibilidad**, y esa es la razón por la que la migración a PostgreSQL fue el requisito previo y no una mejora paralela.

SQLite sobrevive donde sigue siendo mejor: en las pruebas unitarias, en memoria, sin nada que conservar. La regla de una línea: **PostgreSQL se migra con Alembic, SQLite se crea con `create_all`**.

6.2 Tres ControlNodes sin estado

Todas las instancias son idénticas y ninguna guarda estado en memoria. Qué hace cada una:

Capacidad	Todas las instancias	Solo el líder
Servir consultas (`ls`, `stat`, `open`)	✔️	
Autenticación y emisión de JWT	✔️	
Planes de escritura y lectura	✔️	
Aceptar heartbeats por gRPC	✔️	
Entregar órdenes ya decididas por el stream	✔️	
Evaluar la pertenencia (marcar SUSPECT/DEAD)		✔️
Programar re-replicaciones		✔️
Recolectar		✔️

Que **todas** acepten heartbeats es lo que hace que matar al líder no deje a ningún DataNode sin camino: el heartbeat escribe en la base compartida, no en la memoria del proceso que lo recibe. Y la distinción entre **decidir** una orden y **entregarla** es deliberada: negarse a entregar una orden ya tomada porque la atiende una instancia que no es la líder dejaría la copia esperando a que el lease cambiara de manos. Cerrar una tarea cumplida tampoco es una decisión: es registrar un hecho que ya ocurrió.

El esquema lo migra Alembic, no el ControlNode. Con tres instancias arrancando a la vez contra la misma base, migrar desde cada una sería una carrera. Migra un servicio `migrate` de un solo uso; cada ControlNode **verifica** al arrancar que la base está en la última revisión y falla pronto con el comando exacto si no lo está.

Y hay una prueba que es lo que hace útil a Alembic: `test_migracion_deja_el_mismo_esquema_que_los_modelos` compara la base migrada contra `models.py` con `compare_metadata` y exige cero diferencias. Sin ella, añadir una columna y olvidar la revisión pasa todas las pruebas —que usan `create_all`— y falla solo al arrancar contra PostgreSQL, en el despliegue.

6.3 La época: un lease no basta

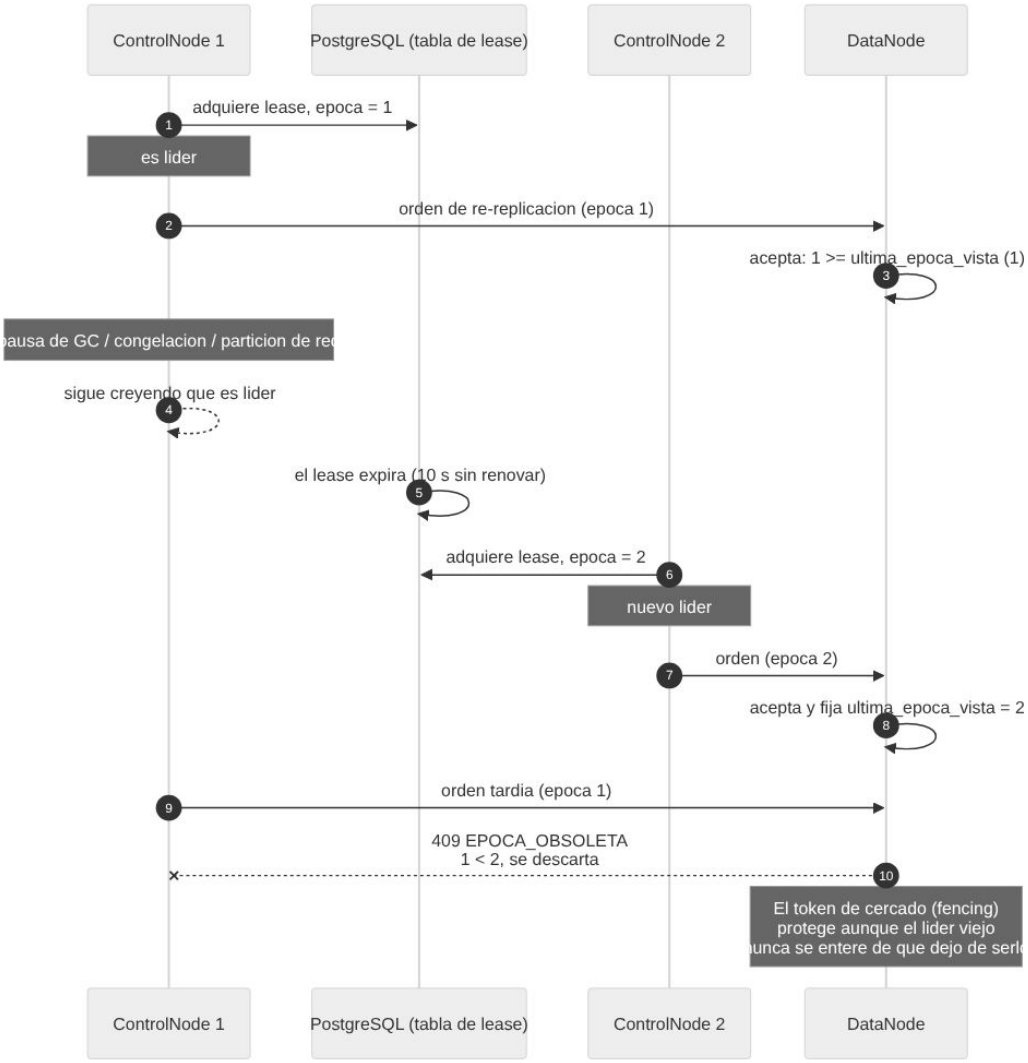
Esta es la pieza central del hito 3 y merece desarrollarse entera.

El mecanismo base es un **lease**: una fila en la tabla `leadership` que una instancia sostiene y renueva cada `DFSHA_LEASE_RENEW_MS` (2 s). Si no la renueva antes de `DFSHA_LEASE_TTL_MS` (6 s), otra la puede tomar.

El lease resuelve el caso del líder que **muere**. No resuelve el del líder que se **congela y despierta**:

```
t=0   A toma el lease (época 7) y empieza a evaluar la pertenencia
t=1   A se congela: pausa larga del recolector de basura,
      partición de red, contenedor sin CPU asignada
t=7   el lease de A vence sin que A se entere
t=8   B lo toma con época 8 y empieza a trabajar
t=9   A despierta EN MEDIO de su operación, convencido de que sigue
mandando
```

A no hizo nada mal según su propia información. El problema es que su información es de t=0, está actuando en t=9, y nada le obliga a volver a mirar. Si lo único que A comprobó fue “soy el líder” antes de empezar, en t=9 escribe, y hay **dos líderes marcando nodos muertos y programando la misma re-replicación dos veces** sobre un clúster que ya se está recuperando de una caída.



La época como token de cercado: el líder congelado despierta y es rechazado.

Las cuatro reglas que lo cortan

1. **La época viaja con la operación** como `Fencing(leader_id, epoch)`, no se consulta antes. **No existe ningún `soy_el_lider()`** consultable: esa función *es* el problema.
2. **Se verifica dentro de la misma transacción que la escritura**, con `SELECT ... FOR UPDATE` sobre la fila del lease. Comprobar en una transacción y escribir en otra deja una ventana, y por esa ventana se cuela A.
3. **La época solo sube.** Nunca baja ni se reinicia.
4. **Recuperar el propio lease vencido también sube la época.** Si un lease vencido propio se tratara como una renovación, A volvería con la época 7 intacta y validaría el trabajo que empezó antes de la pausa. Entre la 7 y la 9 pudo pasar cualquier cosa.

La separación entre `services/leadership.py`, que **propone** una época (vista local, puede estar obsoleta), y `commands/leadership.require_leadership`, que la **verifica**, es el diseño, no una capa de más.

Aviso para quien refactorice esto

En algún momento alguien mirará este código y pensará que falta algo obvio:

```python
# NO. Esto deshace el Bloque A entero.
if not leadership.soy_el_lider():
    return
hacer_el_trabajo()
```

Parece una simplificación evidente y es **la vulnerabilidad exacta** que la época existe para cerrar. Dos motivos, y el segundo es el que no se ve:

1. `soy_el_lider()` solo puede responder con la **vista local**, que es precisamente lo que está mal en el único caso que importa: un proceso congelado cree que sigue mandando.
2. **Aunque consultara la base**, entre esa consulta y el `UPDATE` hay una ventana. La única forma de cerrarla es que la comprobación y la escritura estén en la misma transacción bajo el mismo cerrojo de fila, y eso obliga a que la época entre como **parámetro** de la operación.

Por eso `evaluate_membership` recibe `fencing: Fencing` y llama a `require_leadership` *dentro* de su propio `with uow:`. **La incomodidad de pasar la época por todas partes no es un descuido de diseño: es la firma del tipo obligando a que nadie pueda escribir sin ella.** Si esa incomodidad desaparece en un refactor, la protección también. Los módulos afectados llevan un bloque «Aviso para quien refactorice esto» con el código tentador escrito y tachado: documentar la versión incorrecta es más útil que documentar la correcta, porque la correcta ya está ahí.

Dónde se prueba cada mitad, y por qué en dos sitios

SQLite no implementa `FOR UPDATE` y SQLAlchemy lo omite **en silencio** en ese dialecto. Es decir: la prueba de exclusión mutua corriendo en SQLite no probaría nada y estaría en verde.

Qué	Dónde	Cómo
Comparación de época	SQLite, pruebas unitarias	Reloj inyectado, sin esperas reales
Exclusión mutua	PostgreSQL de verdad	`tests/integration/test_leadership_postgres.py`: ocho hilos saliendo de una barrera común, gana exactamente uno

Esas cuatro pruebas se saltan sin `DFSHA_TEST_PG_URL` para que `pytest` siga corriendo en un portátil sin Docker, pero **no son un extra**: son justamente las que no se pueden fingir. El CI levanta un servicio PostgreSQL para ejecutarlas en cada push.

6.4 El patrón de la etapa: exigir en vez de preguntar

Cuatro problemas distintos del hito 3 —quién manda en el clúster, quién puede leer un archivo, quién puede pedir un bloque, quién puede escribir en un archivo abierto— acabaron teniendo **la misma forma y la misma solución**. No estaba planeado: apareció al resolver el primero y se reconoció en los otros tres.

La forma del problema. En los cuatro hay dos momentos separados: uno en que **se decide** si algo está permitido y otro en que **se hace**. Entre los dos cabe un cambio de estado, y por ahí se cuela el fallo.

La solución, en una frase:

Una función que responde «¿puedo?» es peor que una que hace «hazlo si puedes». La primera se puede ignorar; la segunda no.

En concreto, tres reglas:

1. **La autorización viaja como parámetro de la operación**, no se consulta antes. Si hay que pasarla por toda la pila hasta el fondo, mejor: esa incomodidad es la firma del tipo impidiendo que alguien escriba sin ella.
2. **Se verifica en el mismo sitio donde se hace el efecto** —la misma transacción, el mismo cerrojo de fila, la misma petición—, no en un paso previo.
3. **No existe ninguna función consultable** del tipo `soy_el_lider()`, `tengo_el_lock()` o `que_permiso_tengo()`. Existir es el problema: el camino nuevo que alguien añada dentro de seis meses se olvidará de llamarla, y **no fallará ninguna prueba**.

Los cuatro casos

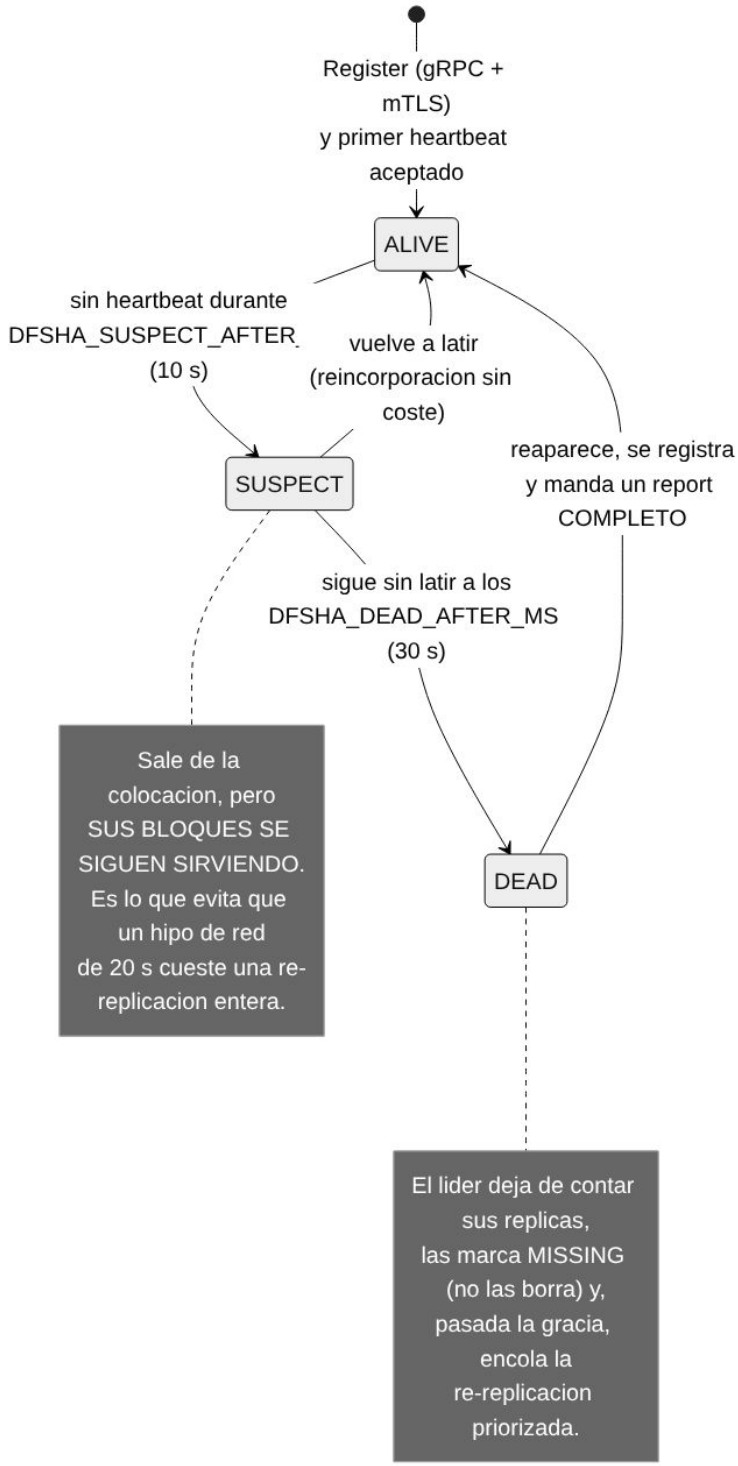
Problema	Qué viaja con la operación	Quién lo exige, y dónde	Qué pasaría sin esto
Liderazgo	`Fencing(leader_id, epoch)`	`require_leadership`, dentro de la transacción que escribe, con `SELECT … FOR UPDATE`	Dos ControlNodes marcando nodos muertos y duplicando el tráfico de recuperación de un clúster que ya se está recuperando
Permisos (ACLs)	el permiso **mínimo** exigido	`directory_for(uow, user, path, minimum)`, que resuelve y exige a la vez	Un `resolve()` devuelve el permiso, y el camino que alguien añada después se olvida de compararlo
Acceso a bloques	`block_id` y operación, **firmados**	`verify_token(token, block_id=…, operation=…)`, en el DataNode	Las ACLs se saltan pidiéndole el bloque directamente al DataNode, que no conoce ni rutas ni usuarios
Bloqueo de archivos (RF3)	`LockFencing(holder_id, epoch)`	`require_lock`, dentro de la transacción que escribe	Dos clientes escriben el mismo archivo **creyendo ambos** tener exclusión

Por qué tres de los cuatro llevan un par y no un número

- **El identificador solo no basta**: el mismo actor puede haber perdido y recuperado el permiso entre medias, y lo que hizo antes no vale después.
- **El número solo tampoco**: dice *cuándo*, no *quién*. En el RF3 esto se cobra literalmente, porque al soltar un lock su época puede reiniciarse y dos titulares distintos llegar a tener el número 1. Solo el par los distingue.

Que cuatro problemas que no se parecen en nada se resuelvan igual es lo que hace que el diseño sea coherente y no una colección de soluciones sueltas. Es el resultado conceptual del que el equipo está más satisfecho.

6.5 Detección de fallos de los DataNodes



Máquina de estados de un DataNode.

Transición	Umbral	Qué se hace
`ALIVE → SUSPECT`	`DFSHA_SUSPECT_AFTER_MS` (10 s)	Sale de la colocación. **Sus bloques se siguen sirviendo**
`SUSPECT → ALIVE`	Vuelve a latir	Reincorporación sin coste
`SUSPECT → DEAD`	`DFSHA_DEAD_AFTER_MS` (30 s)	Sus réplicas dejan de contarse como disponibles; empieza a correr la espera de gracia
`DEAD → ALIVE`	Se registra de nuevo	Reconciliación por `boot_id` y report completo

`SUSPECT` no toca las réplicas, y esa es la decisión. Un hipo de red de veinte segundos no debe costar una re-replicación entera. Sacarlo de la colocación es barato y reversible; dar sus copias por perdidas no lo es.

La distinción entre lo que hay y lo que se puede servir se ve en `dfsha cluster`, que muestra dos columnas aparentemente redundantes: `bloq.` es lo que el nodo dice tener en su disco y `repl.` lo que el ControlNode cuenta como servible. Al pasar a `DEAD`, `repl.` cae a 0 mientras `bloq.` se queda en su valor: los bytes siguen ahí, pero el sistema ya no cuenta con ellos. En la validación en AWS esa asimetría se observó literalmente (`bloq. 82 / repl. 0`), y es la decisión 5.5 hecha visible sin abrir un solo log.

6.6 El balanceador: tres puertos, dos niveles

Hay **un balanceador y tres puertos**, y no todos se proxean igual:

Puerto	Tráfico	Nivel	Por qué
`:8000`	REST del cliente	**7** (HTTP)	nginx lee la petición, reparte por `least_conn` y puede reintentar en otra instancia
`:8443`	Plano interno (mTLS)	**4** (TCP)	El handshake tiene que ser de punta a punta
`:9000`	gRPC del plano de control (mTLS)	**4** (TCP)	Lo mismo

Un proxy de nivel 7 termina TLS. Abre la conexión del que llama, la lee, y abre *otra distinta* hacia el ControlNode. El certificado que el DataNode presenta **muere en nginx**, y al ControlNode le llega una conexión anónima. Eso no es una pérdida de rendimiento: **destruye el propósito entero del mTLS**, que es que el ControlNode sepa quién está al otro lado y no solo que conoce un secreto.

Se podría reinyectar la identidad en una cabecera tipo `X-SSL-Client-CN`, y conviene decir explícitamente por qué **no**: la garantía pasaría a ser «confía en lo que dice el balanceador», que es **exactamente el secreto compartido del que el hito 3 se deshizo**, solo que ahora con un salto más y la falsa sensación de estar usando certificados. Quien pudiera hablar con nginx podría afirmar ser cualquiera.

Con `stream`, nginx reparte bytes sin mirarlos, ControlNode y DataNode se validan entre sí contra la misma CA, y **el balanceador no tiene ni necesita certificado**.

Lo que se pierde, dicho antes de que alguien lo descubra: el reparto pasa a ser por conexión y no por petición, y nginx ya no puede reintentar a mitad de una petición. Para estos dos puertos da igual, y para gRPC es **incluso más apropiado**: el stream de `Heartbeat` es una conexión larga que dura horas, así que el balanceo por petición nunca le aportó nada. El único camino que se beneficia de verdad del nivel 7 es el REST del cliente, y ese se queda donde estaba.

El plano interno también va por el balanceador, no contra una instancia fija, por el mismo motivo que el gRPC: así, matar al líder durante la demostración no deja a ningún DataNode sin camino.

Un límite conocido que se decidió no arreglar

nginx resuelve los nombres de `upstream` **al cargar la configuración** y se queda así. Si a un ControlNode le cambiara la IP dentro de la red de compose, el balanceador seguiría apuntando a la vieja. Se decidió no arreglarlo, con tres razones en orden de peso:

1. **En AWS el alcance se reduce a un nombre.** De los dos `upstream` solo uno es un nombre; el otro es la IP privada de `dfsha-control`, estable entre reinicios.
2. **El efecto está acotado, pero menos de lo que se creía.** Los `upstream` llevan `max_fails=2 fail_timeout=5s` y `proxy_next_upstream`: si **una** IP queda obsoleta, nginx lo detecta al conectar, pasa a la siguiente y la saca de la rotación.
3. **El arreglo cuesta justo lo que el balanceador aporta.** Hacer que nginx re-resuelva sin módulos de pago exige poner la dirección en una variable dentro de `proxy_pass`, y eso **se salta el bloque `upstream`**: adiós a `least_conn` y al reintento en otra instancia. En `:8000` eso es exactamente la razón de que sea nivel 7.

La decisión se mantiene: perder `least_conn` y el reintento en el camino principal para cerrar un caso con una salida de un comando es mal negocio. Pero **la segunda razón, tal como estaba escrita, era demasiado generosa, y hay que corregirla en voz alta** porque es justo el tipo de afirmación que este informe critica en otros sitios.

Al verificar los arreglos de documentación del hito 4 se observó el caso real, y no se parece al que la justificación describía:

Lo que decía la justificación	Lo observado
El disparador es raro: a un ControlNode le cambia la IP	El disparador es `docker compose up -d --build`, que **recrea los tres ControlNodes y no el `lb`**, porque su configuración no cambió. Es la operación más común del desarrollo
Se recupera solo: nginx saca de la rotación la IP obsoleta	**No se recupera**, porque las **tres** IP quedan obsoletas a la vez y no queda ninguna a la que pasar. Los cuatro DataNodes entran en bucle de reinicio con `datanode.register_retry` y `code: UNAVAILABLE` contra `lb:9000`

El síntoma es ruidoso e inmediato —ningún DataNode se registra— y la salida sigue siendo la de siempre, `docker compose restart lb`. Pero «raro y auto-recuperable» describía un caso distinto del que

ocurre en la práctica, y mientras lo dijera así nadie iba a buscar el disparador correcto. Ahora está escrito donde toca, en el apartado de problemas frecuentes (§9.6), que es donde lo va a buscar quien lo sufra, y con el aviso de siempre: **el balanceador no es el primer sospechoso, salvo justo después de un `--build`.**

6.7 Lo que sigue siendo punto único de fallo

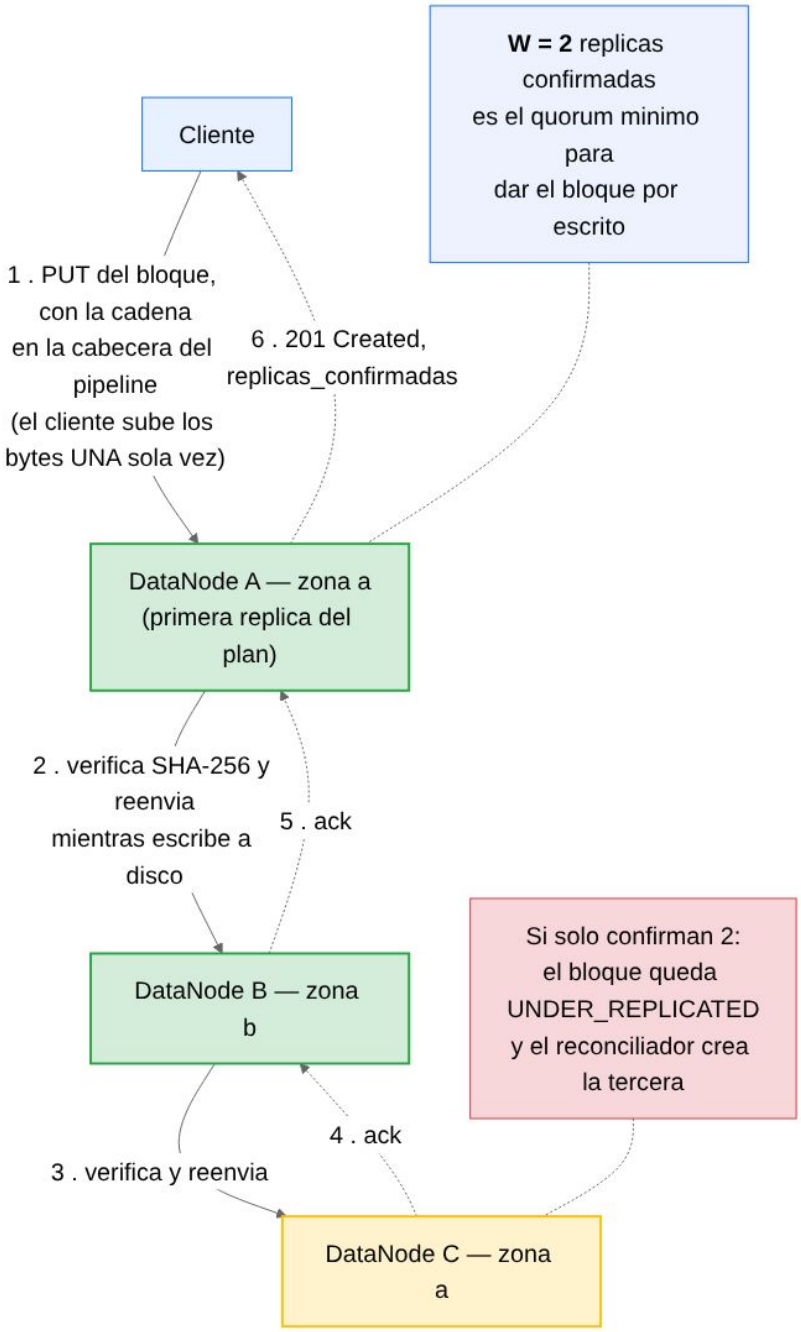
PostgreSQL. La promoción de la réplica es **manual**, documentada en `deploy/RUNBOOK-postgres.md`. Es una decisión, no un olvido: un failover de base de datos automático y correcto necesita un árbitro externo, protección contra doble promoción y una forma de avisar a los clientes. Es otro proyecto, y **uno a medias es peor que ninguno**, porque un failover que a veces promueve dos primarios pierde datos de una forma que no se detecta hasta mucho después.

No hay que confundirlo con el liderazgo del ControlNode, que **sí** es automático, y la diferencia es exactamente por qué: el ControlNode no tiene estado propio, así que promoverlo mal no corrompe nada; una base de datos con dos primarios sí.

En el despliegue de AWS esto se traduce en que **matar la instancia `dfsha-db` tumba el clúster**. Está escrito antes de la demostración, a propósito, para que sea un límite declarado y no un descubrimiento en la sustentación.

7. Replicación y consistencia

7.1 R=3 en cadena: el cliente sube una vez



Pipeline de replicación R=3 con quórum W=2.

Con tres copias, la alternativa ingenua es que el cliente suba el mismo bloque tres veces. DFSha lo sube **en cadena**: el cliente manda el bloque a la primera réplica del plan con las otras dos en la cabecera `X-DFSha-Pipeline`, y cada DataNode lo reenvía al siguiente quitándose de la lista.

El argumento es de recurso escaso. Con instancias pequeñas y un enlace doméstico, **el ancho de subida del cliente es el cuello de botella**, y multiplicarlo por tres es justo lo que no se puede permitir. La red

interna del clúster, en cambio, es abundante y simétrica. El pipeline mueve el coste de donde duele a donde no.

7.2 Verificar antes de reenviar, escribir y reenviar a la vez

El enunciado pide dos cosas que, leídas literalmente, son incompatibles: verificar el checksum **antes** de reenviar, y reenviar **mientras** se escribe. Un SHA-256 no se puede verificar sin ver el último byte, así que reenviar en *streaming* puro y verificar antes de reenviar no pueden ocurrir a la vez.

La lectura que satisface las dos: **recibir entero, verificar, y entonces escribir a disco y reenviar en paralelo**.

- **Verificar antes de reenviar** impide que una corrupción se propague por la cadena: lo que sale de un nodo ya está comprobado.
- **Escribir y reenviar en paralelo** evita que la latencia de la cadena sea la suma de las latencias de disco de tres nodos: el reenvío no espera al `fsync`.

Dicho de otro modo, y es la formulación que vale: **no se puede verificar y reenviar a la vez, así que hay que elegir qué se solapa con qué.** Solapar el reenvío con la *escritura* —no con la recepción— da las dos garantías y solo cuesta memoria: un bloque en RAM por subida concurrente, que ya era el comportamiento del DataNode desde el hito 1.

7.3 El interbloqueo que costó encontrar

La primera versión hacía la escritura y el reenvío **dentro** del handler asíncrono. Eso deja el bucle de eventos del nodo parado: mientras escribe o espera, **el nodo deja de aceptar peticiones**. Con dos subidas concurrentes eso es una espera circular:

```
bloque A:  cliente -> DN1 -> DN2 -> DN3
bloque B:  cliente -> DN2 -> DN1 -> DN4
```

DN1 espera a DN2, DN2 espera a DN1, y ninguno puede atender al otro. Las dos subidas mueren por *timeout*. Se reprodujo en las pruebas de integración, que pasaron de **248 s en timeouts a 50 s** al mover el trabajo bloqueante a un hilo del pool (`run_in_threadpool`).

Aviso para quien refactorice esto. `_escribir_y_reenviar` parece una función síncrona suelta que "podría" volver al handler para ahorrarse una indirección, y el `run_in_threadpool` parece ceremonia innecesaria alrededor de código que no es lento. Las dos lecturas son erróneas por el mismo motivo: **el problema no es que el trabajo tarde, es que mientras tarda el nodo no puede atender a nadie**, y quien le está llamando es un nodo que a su vez no puede atenderle a él. Un solo nodo con una sola subida no reproduce nada; hacen falta dos nodos y dos subidas cruzadas, que es por lo que un refactor puede deshacerlo y ver la suite en verde localmente.

7.4 El quórum W=2

Decisión	Argumento
W=2 y no W=3	Con W=R, la disponibilidad de la escritura es el producto de las disponibilidades: cuantas más copias, **menos** disponible. Es la paradoja que el quórum existe para romper

Decisión	Argumento
W=2 y no W=1	Con una sola copia confirmada, perder ese nodo antes de que el reconciliador actúe pierde el dato. Dos copias toleran un fallo desde el instante del commit
Un fallo aguas abajo no tumba la subida	Si el nodo escribió bien pero el siguiente falla, responde `201` con un `acked` menor. Fallar la petición convertiría W=3 en el mínimo de hecho, que es lo contrario de la decisión
El quórum se decide en el commit y solo ahí	El `X-DFSha-Replicas-Acked` que ve el cliente es informativo; la cuenta buena es la de `block_replicas`, que cada DataNode actualiza por su cuenta
W < R deja el archivo sub-replicado, no roto	Con 2 de 3 todavía tolera perder un nodo. Rechazar el commit pondría la durabilidad por encima de la disponibilidad, que es la elección contraria a la que hacen estos sistemas. Con menos de W el commit falla y el cliente reintenta: no hay medias tintas

Que no haya carreras en la cuenta del quórum depende de una decisión del hito 2: el aviso `POST /internal/v1/blocks/{id}/stored` es **síncrono y anterior al 201**, así que cuando el cliente puede pedir el commit, el ControlNode ya sabe de esas copias (§4.5).

7.5 Re-replicación: tres frenos

La dispara el hueco entre las copias que un bloque debería tener y las que tiene. **Solo la programa el líder, con época verificada dentro de la transacción**, por la razón del Bloque A agravada: dos líderes programando a la vez no duplicarían un evento de log, duplicarían el **tráfico de copia de un clúster que ya se está recuperando de una caída**.

Los tres frenos, y ninguno es opcional:

1. Espera de gracia (`DFSHA_REREPLICATION_GRACE_MS`, 5 min)

No se programa nada hasta que el nodo lleva ese tiempo muerto. Reiniciar un contenedor tarda segundos, y copiar su disco entero por un reinicio no solo es caro: **se encadena**. La copia satura la red, otro nodo deja de latir a tiempo, y se dispara otra copia.

Se mide desde el último latido: como un nodo pasa a `DEAD` en `dead_after`, exigir `dead_after + gracia` de silencio es exactamente «gracia desde que entró en DEAD».

Para el video y las demostraciones se baja a 30 s, que es un cambio de parámetro y no de comportamiento. La prueba `test_un_reinicio_rapido_NO_dispara_una_copia` fija la otra mitad: un nodo que se reinicia y vuelve enseguida no debe costar ni una copia; sin ese freno, cada despliegue rodante movería el disco entero.

2. Tope por destino (`DFSHA_REREPLICATION_MAX_PER_NODE`, 2)

Sin él, la recuperación se concentra en el nodo más vacío y lo tumba por saturación — el nodo que precisamente se ofreció como destino **por estar libre**.

3. Prioridad por copias restantes

Un bloque con **una sola** copia va antes que uno con dos. No es una optimización: uno está a un fallo de desaparecer y el otro todavía tolera una caída. **Si el clúster no da abasto, el orden en que se rinde decide si se pierden datos.** El desempate es por `block_id` y no por antigüedad, para que dos ControlNodes que miren el mismo estado decidan lo mismo.

El destino tira, el origen no empuja

La orden le llega a quien tiene que hacer el trabajo y puede negarse si no le cabe; el origen solo ve una descarga más, que es lo que ya sabe hacer, y no se le carga de escrituras mientras quizá sirve lecturas. Además no hay endpoint nuevo: se reutiliza `GET /blocks/{id}`.

El DataNode ejecuta las órdenes fuera del hilo del heartbeat. Copiar 64 MB tarda; hacerlo en el hilo del stream dejaría de mandar latidos mientras dura, el ControlNode daría por muerto justo al nodo que está haciendo el trabajo, y la respuesta a eso sería programar todavía más copias.

La regla que costó un 404

Al despachar una copia hay que **crear la fila `PENDING` en `block_replicas` para el nodo destino**. Es la decisión 3 de §3.2 aplicada también aquí: el ControlNode elige el destino y **registra la elección**.

La primera versión no lo hacía, y el síntoma fue este: el nodo destino copiaba el bloque correctamente, avisaba con `/blocks/{id}/stored`, y recibía `404 bloque desconocido en el metadato`, porque `mark_stored` solo sabe actualizar una fila que ya existe. Resultado: la copia quedaba en su disco como **huérfana**, la tarea no se cerraba nunca, y el bloque aparecía en los logs como `divergence.unknown_block` cada seis segundos. **Todo el mecanismo funcionaba salvo el registro de la decisión.**

7.6 Lectura: relevo entre réplicas

Leer prueba las réplicas **por orden**. Sin relevo, tener tres copias no servía para leer: perder el primer nodo del plan hacía fallar el `get` aunque los otros dos tuvieran los bytes.

Una réplica que devuelve bytes **corruptos** también se descarta y se pasa a la siguiente, que es exactamente el caso para el que existe tener más de una copia.

Lo que esto cuesta en diagnóstico, y conviene decirlo. El relevo captura el error de cada réplica, prueba la siguiente, agota la lista y reporta el fallo **agregado**. El mensaje que llega arriba cuenta cuántas réplicas se probaron, no **por qué** falló cada una, así que un fallo determinista que habría dado igual con cien copias se lee como un problema de tener pocas. Ocurrió literalmente durante el desarrollo (§10.6, caso `blocks.size`).

La consecuencia práctica, que vale la pena escribir: **al diagnosticar, el primer error de la cadena vale más que el último.** Un relevo mejora la disponibilidad y **empeora el diagnóstico**, y las dos cosas son ciertas a la vez.

7.7 Sobre-replicación tras una reincorporación: estado esperable

Observado en la validación y documentado antes de que nadie lo encontrara. `dfsha stat` puede mostrar `FULLY_REPLICATED (3-4 de 3 copias por bloque)`. **No es una anomalía.** La secuencia:

1. Un DataNode muere. El evaluador marca sus réplicas `MISSING` — **no las borra** (§5.5).
2. Pasada la gracia, la re-replicación pone una copia nueva en otro nodo. El bloque vuelve a tener 3 copias vivas, más la fila `MISSING` del caído.
3. El nodo **vuelve** con su disco intacto y manda un report completo. Un report completo es la verdad sobre ese disco, así que la fila `MISSING` vuelve a `STORED`.
4. El bloque se queda con **4 copias** y R=3.

El estado de replicación se deriva del bloque **peor** replicado, así que el archivo sigue siendo `FULLY_REPLICATED`: tener de más en un bloque no empeora a ninguno. El rango `3-4` se muestra a propósito — un número plano escondería que los bloques no están todos igual.

Y no se limpia solo. Esto corrige la intuición natural: parece que el recolector lo recogerá, y no lo hace, porque el recolector recoge bloques de archivos `DELETED` y de reservas vencidas, y este bloque pertenece a un archivo vivo. La cuarta copia se queda. Cuesta disco hasta que alguien decida quitarla; no cuesta corrección, y la durabilidad solo mejora.

Quitarla automáticamente violaría la regla de que el ControlNode no borra datos por una divergencia, y la violaría en el peor sitio posible: **para deshacer una redundancia que el propio sistema acaba de conseguir.**

Que la fila `MISSING` sobreviva es, además, lo que hace que una reincorporación normal **no cueste** una re-replicación. Esta sobre-replicación es la otra cara de esa misma propiedad: la reincorporación llegó tarde, cuando la copia ya se había repuesto. Está fijado en `tests/unit/test_sobrereplicacion.py`, que además blinda lo que **no** debe pasar: no se encola re-replicación, no se cuenta como sub-replicado ni crítico, no se registra divergencia, y el recolector no lo lista.

7.8 Consistencia del metadato: CQRS y read-your-writes

Las consultas se sirven por defecto desde la réplica de lectura de PostgreSQL, que va por detrás del primario por replicación asíncrona. Eso crea un problema concreto: un `mkdir /a` seguido de un `ls /` puede preguntarle a una réplica que aún no reprodujo el `mkdir`. **Eso no es un poco de retraso, es mentirle al cliente sobre su propia escritura.**

El mecanismo, en tres pasos:

1. Tras cada comando, un middleware del ControlNode devuelve `pg_current_wal_lsn()` en la cabecera `X-DFSha-Write-LSN`. Va en middleware y no en cada caso de uso porque tiene que medirse **después** del commit.
2. El cliente lo guarda en `session.json` —cada invocación de `dfsha` es un proceso nuevo, así que en memoria no serviría— y lo reenvía como `X-DFSha-Read-LSN`.
3. El ControlNode lo compara con `pg_last_wal_replay_lsn()` de la réplica. Si la réplica va por detrás, la consulta se atiende desde el primario y se registra `query.routed_to_primary`.

El coste cae solo sobre los clientes que acaban de escribir: quien no manda LSN va derecho a la réplica sin consulta adicional.

Tabla de enrutado completa:

Operación	Destino	Por qué
`ls`, `stat`, `open`, `cluster/status`	**réplica**	Consultas puras. Son el grueso del tráfico
Cualquiera de las anteriores con LSN por delante de la réplica	**primario**	Read-your-writes
Todos los comandos	**primario**	Escriben
`POST /internal/v1/blocks/{id}/stored`	**primario**	Escribe
`GET /internal/v1/gc/orphan-blocks`	**primario**	Es una consulta, pero su respuesta dispara un **borrado en disco**. Una réplica retrasada podría listar un bloque que ya no toca borrar
`auth/register`, `auth/login`	**primario**	`register` escribe; `login` lee credenciales recién creadas

De esa tabla, la fila del recolector es la que merece subrayarse porque enuncia una regla general: **una consulta cuya respuesta dispara una escritura destructiva no se sirve desde la réplica.**

Y todos los caminos de fallo caen del lado seguro: un LSN mal formado, una réplica inalcanzable o un `DFSHA_DB_REPLICA_URL` que por error apunta a un primario acaban sirviendo desde el primario. **`DFSHA_DB_REPLICA_URL` vacía es un modo soportado**, no una degradación: sin ella, el motor de lectura y el de escritura son el mismo y el sistema se comporta como en las etapas anteriores.

7.9 El RF3 y el bloqueo de archivos

El escenario del Bloque A, cambiando «líder» por «cliente» y sin cambiar nada más:

```
t=0   Ana toma el lock de /informe.txt (época 3) y empieza a
escribir
t=1   Ana se congela: su portátil suspende, se le va la red
t=7   el lease vence sin que Ana se entere
t=8   Beto lo toma con época 4 y escribe su versión
t=9   Ana despierta EN MEDIO de su escritura, creyendo que sigue
teniendo el lock
```

Si lo único que Ana comprobó fue «tengo el lock» antes de empezar, en t=9 escribe encima de lo de Beto y **los dos creen haber escrito con exclusión**. Un lock que falla así es peor que no tener lock: da una garantía que no cumple.

Misma solución, palabra por palabra: `LockFencing(holder_id, epoch)` viaja con la operación y se verifica dentro de la misma transacción que la escritura, con cerrojo sobre las filas del archivo. En t=9 la fila dice 4, Ana trae 3, y el `append` se rechaza entero con `stale_lock`.

Cuatro decisiones del lock que merecen argumentarse:

El titular es una SESIÓN, no un usuario. Ana desde dos máquinas son dos titulares, y tiene que ser así o el lock no excluiría nada entre sus propios procesos. Por eso `file_locks` no tiene clave foránea a `users` y el cliente guarda un `holder_id` en `session.json`: cada invocación de `dfsha` es un proceso nuevo, y uno aleatorio por proceso haría que `dfsha lock` y el `dfsha append` siguiente fueran dos titulares distintos.

El TTL es dos órdenes de magnitud mayor que el del liderazgo (120 s frente a 6 s), y la diferencia es la que hay entre los dos problemas: allí el relevo tiene que ser rápido porque un clúster sin líder no se repara solo; aquí lo que espera es una **persona** con el archivo abierto, y arrebatárselo a los seis segundos porque tardó en teclear sería inutilizable. El precio es que un cliente que muere deja el archivo bloqueado ese tiempo.

No hay ascenso de compartido a exclusivo. Dos clientes con lock compartido pidiendo ascender a la vez se esperarían el uno al otro para siempre. Detectarlo obliga a un grafo de espera y a elegir víctima, que es otro proyecto. Quien quiera escribir pide exclusivo desde el principio.

Adquirir y renovar son la misma llamada. Un cliente que reintenta tras un timeout de red no puede saber si el primer intento llegó; si «adquirir» fallara cuando ya lo tienes, ese reintento correcto sería un error.

`read` por rango: el bloque se baja entero

`plan_range` devuelve solo los bloques que intersectan el tramo, con cuánto recortar de cada uno. Lo que **no** hace es pedir tramos parciales al DataNode, y el motivo no es pereza:

Un bloque cifrado no se puede descifrar por partes. La etiqueta de AES-GCM cubre el bloque completo, así que para obtener cualquier byte con garantía de que no fue alterado hay que tener los demás. Pedir un `Range:` al DataNode daría bytes cifrados que no se pueden ni descifrar ni verificar.

Lo que el rango ahorra —y es casi todo el ahorro real— es **no bajar los bloques que no intersectan**: leer 1 KB del final de un archivo de 1 GB con bloques de 64 MB baja 64 MB en vez de 1 GB. El límite útil de la granularidad lo pone el tamaño de bloque, que es configurable. La alternativa sería cifrar en trozos más pequeños con su propia etiqueta cada uno: más etiquetas, más nonces que no repetir, y un diseño distinto. No se hizo, y queda escrito como lo que es: una decisión, no un olvido.

7.10 `write` es append, y el WORM obliga a elegir dónde se paga

Los bloques son inmutables. Si el último bloque está **a medias** —lo normal, salvo tamaño múltiplo exacto — añadir datos tiene que llenarlo, y llenarlo es reescribirlo. Dos salidas, y ninguna gratis:

	Coste	Veredicto
(a) Empezar siempre bloque nuevo	Mil `append` = mil bloques diminutos, cada uno con su colocación, sus R=3 filas de réplica y su sitio en la cola. 1000 bloques de 12 B son 3000 filas para 12 KB de datos	Descartada
(b) Reescribir el bloque de cola	Añadir 1 byte reescribe hasta un bloque entero (64 MB con el	**Elegida**

	Coste	Veredicto
	default)	

Se eligió (b), y el precio se dice entero: **amplificación de escritura acotada por el tamaño de bloque, no por el del archivo**. A cambio se conserva el invariante «todos llenos salvo el último», que es lo que mantiene la colocación, la contabilidad de tamaños y las lecturas por rango razonando sobre bloques homogéneos. Es copy-on-write, la decisión 1 aplicada a un bloque en vez de a un archivo. El evento `file.appended` registra `rewritten_bytes`, que es lo que hace la amplificación visible en los logs.

La cola la reescribe el CLIENTE, y no por comodidad. Con cifrado extremo a extremo el servidor **no podría aunque quisiera**: concatenar exige descifrar, y la clave no sale del cliente. El append encaja solo en la decisión 4 —los bytes no pasan por el ControlNode— sin que haya habido que forzar nada.

El metadato no cambia hasta el commit, y esto es lo importante. Los bloques nuevos nacen con `file_id` NULL —sueltos, sin pertenecer a nadie— y el archivo conserva su tamaño y su cola viejos. Un lector que pase por en medio ve **el archivo de antes, entero y legible**. En el commit del append ocurren las tres escrituras en una sola transacción: desligar la cola vieja, enganchar los bloques nuevos, actualizar el tamaño. El orden importa: primero desligar, porque al revés los dos bloques compartirían `(file_id, index)` un instante y saltaría la restricción de unicidad.

El lock se exige dos veces, al planificar y al confirmar. No es redundante: entre las dos el cliente sube bytes, lo que con bloques de 64 MB puede tardar minutos — tiempo de sobra para que su lease venciera y otro tomara el archivo.

8. Seguridad

8.1 El modelo, en una frase

El servidor es no confiable para el contenido y confiable para el metadato. Esa asimetría es la decisión, y de ella salen las cinco capas:

Capa	Qué protege	De quién
JWT	Quién es el usuario	De quien no tiene credenciales
ACLs con grupos	Qué rutas puede tocar cada usuario	De otros usuarios del sistema
Token de bloque	Que ese permiso alcance a los **bytes**, no solo al metadato	De quien conozca un `block_id`
mTLS interno	Que cada servicio sea quien dice ser	De quien pueda hablar con el plano interno
Cifrado extremo a extremo	El contenido	**Del propio servidor** y de quien le robe el disco

Las capas son independientes a propósito: romper una no entrega las demás. Quien robara toda la base de datos y todos los discos obtendría rutas, tamaños y bytes opacos.

8.2 Autenticación

Usuario y contraseña, contraseña almacenada con bcrypt, token JWT firmado con `DFSHA_JWT_SECRET` y vida de una hora. El ControlNode es *stateless*: la identidad viaja en el token y no hay sesión en el servidor, que es la precondición de que cualquier instancia pueda atender cualquier petición (§6.2).

`DFSHA_JWT_SECRET` **no tiene valor por defecto en el código** y debe tener al menos 16 caracteres. Un secreto por defecto en un repositorio público es un hallazgo de seguridad, y este proyecto se evalúa en seguridad.

8.3 mTLS en el plano interno

Una CA propia (`scripts/gen_certs.py`) y un certificado por rol: `control`, `data`, `client`. El plano interno (`:8443`) y el gRPC (`:9000`) exigen certificado de cliente (`CERT_REQUIRED`).

Esto sustituyó a `DFSHA_INTERNAL_SECRET`, que desapareció del código. El argumento: un secreto compartido protege contra quien no lo conoce, pero **no dice quién está al otro lado** — cualquiera que lo tenga es todos a la vez. Con mTLS cada rol presenta su propio certificado y el servidor sabe con quién habla. Esa diferencia es la que permite, en §8.8, exigir que el firmante de un token de bloque sea `dfsha-control` y no cualquier miembro del clúster.

Dos simplificaciones de práctica, declaradas:

1. En el compose local, `./certs` se monta entero y eso incluye `ca.key`, que **no debería salir de la máquina que firma**. En AWS cada instancia recibe únicamente sus tres ficheros y la clave de la CA se queda fuera.

2. Los cuatro DataNodes comparten un `data.crt`: revocar a uno obligaría a rotar los cuatro. Se eligió así porque el SAN ya cubre los cuatro nombres y el número de nodos es fijo; con nodos que entran y salen, cada uno necesitaría el suyo.

Y una consecuencia de §3.3 que hay que tener presente: los certificados tienen que cubrir **los nombres de par** (`data-node-N` en compose, las IP privadas en AWS), porque son los que el nodo que llama verifica contra el SAN. Sin ellos, el pipeline falla en el *handshake* en vez de en el `connect`: un fallo distinto con el mismo efecto.

8.4 TLS de cara al cliente

`DFSHA_CLIENT_TLS_CERT` y `DFSHA_CLIENT_TLS_KEY` vacías = HTTP plano, que es el **default**. Encenderlo es superponer `docker-compose.tls.yml` (§9.5). En AWS va encendido.

No es mTLS, y la diferencia es la decisión:

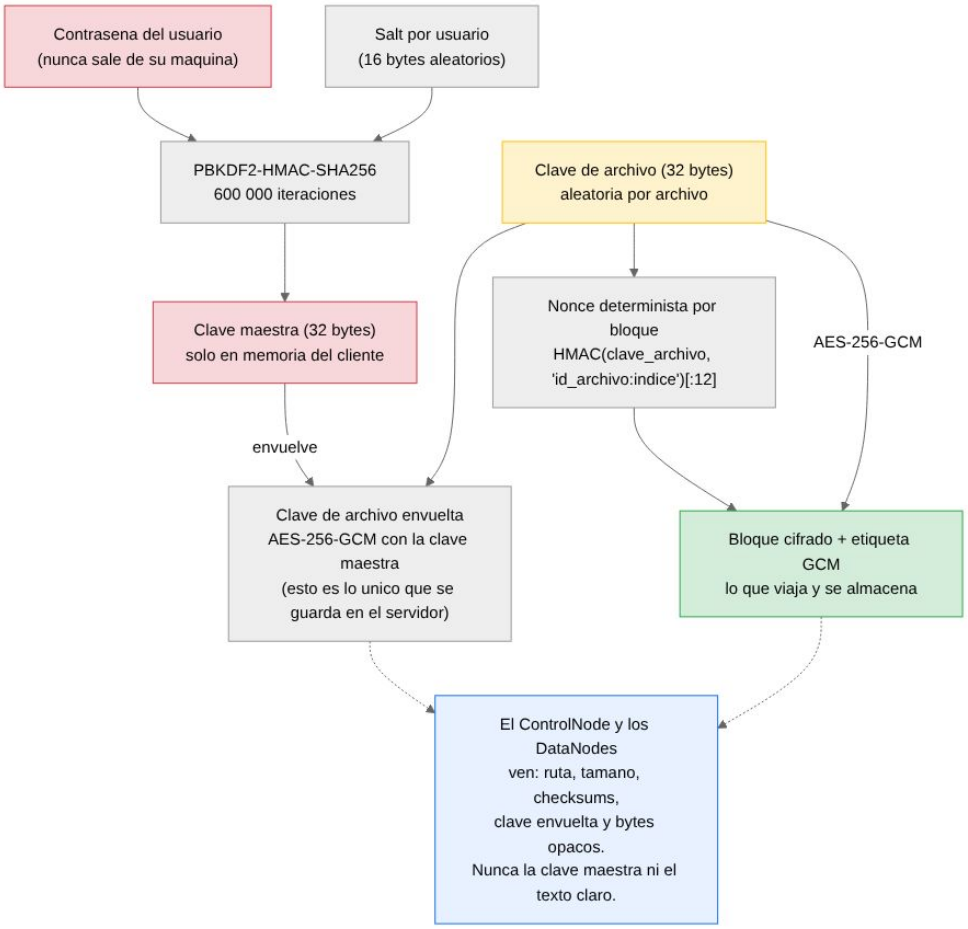
	Plano interno	Plano de cliente
Quién llama	un servicio nuestro	una **persona**
Cómo se identifica	su **certificado**	su **JWT**
El servidor exige certificado	**sí** (`CERT_REQUIRED`)	**no** (`CERT_NONE`)
Qué aporta TLS	cifrado **e identidad de los dos**	cifrado e identidad **del servidor**

Darle un certificado a cada usuario sería montar una PKI —emisión, distribución, revocación, rotación— para acabar sabiendo lo mismo que ya dice el token. Aquí TLS hace lo que hace bien: cifrar el canal y demostrar que el servidor es quien dice ser. **Quién es el usuario lo dice la capa de arriba, y eso no es un hueco.**

Qué protege de verdad, dicho en el orden correcto. Los bloques **ya viajaban cifrados** desde el Bloque C, así que esto **no** es lo que salva la confidencialidad del contenido: eso ya estaba. Lo que añade es proteger lo que quedaba en claro: el **token JWT**, con el que cualquiera podría suplantar al usuario durante toda su vida; el **metadato** —nombres, rutas, tamaños, con quién se comparte—; y los **tokens de bloque**.

`verify=False` no aparece en ningún sitio, y es deliberado. Es la salida que hace que todo funcione a la primera, y es exactamente el fallo que este proyecto lleva persiguiendo: algo que aparenta estar puesto y no lo está. Un TLS sin verificar cifra contra un observador pasivo y no protege de nada frente a quien pueda ponerse en medio, que es el atacante que importa — y el `https://` se ve igual. Como la CA es propia, hay que decirle al cliente en quién confiar, y de las dos formas posibles solo una es aceptable: instalarla en el almacén del sistema afectaría a **todo** el equipo; decírselo a este cliente y solo a él (`DFSHA_TLS_CA_CERT`) es lo que se hace. Si falta, **falla y dice dónde buscó**.

8.5 Cifrado extremo a extremo



Las tres capas de clave. El servidor nunca ve ninguna.

El servidor **nunca ve una clave**. Con el metadato entero, la base de datos y todos los bloques del disco, no puede descifrar un byte.

```
contraseña del usuario
    │  PBKDF2-HMAC-SHA256, 600 000 iteraciones, con users.kdf_salt
    ▼
clave maestra  ──────  no sale NUNCA del cliente
    │  AES-256-GCM, con el file_id como dato autenticado
    ▼
clave de archivo ────  una por archivo; lo que se guarda es su
ENVOLTURA
    │  AES-256-GCM, nonce = HMAC-SHA256(clave, "file_id:index")[:12]
    ▼
bloque cifrado en el DataNode
```

Lo que el servidor guarda son **dos columnas inofensivas por separado**: `users.kdf_salt`, que no es secreta —su trabajo es que dos usuarios con la misma contraseña tengan claves distintas, por eso puede viajar en el login— y `files.wrapped_key`, que sin la clave maestra es ruido.

Cinco decisiones del esquema:

Una clave por archivo y no una por usuario, aunque sea más trabajo: así compartir un archivo puede llegar a ser entregar su clave, sin dar acceso a todo lo demás.

El `file_id` entra en la envoltura como dato autenticado. Sin eso, alguien con acceso de escritura al metadato podría intercambiar envolturas y hacer que el cliente descifrara el archivo equivocado con una clave que sí valida. No rompe la criptografía; rompe la integridad de lo que el usuario cree estar leyendo.

La envoltura viaja en el commit y no en el create, y no es una preferencia: se envuelve con el `file_id`, que al crear todavía no existe.

`wrapped_key` vacía = archivo sin cifrar, y el cliente lo mira en vez de suponerlo. Es lo que permite que un metadato migrado desde el hito 2 siga siendo utilizable.

El checksum va sobre el texto CIFRADO. El cliente cifra y *luego* calcula el SHA-256. Así el DataNode verifica integridad sin tener la clave y —lo que importa para el pipeline— **puede seguir comprobando antes de reenviar**. Las dos comprobaciones no son redundantes: el SHA-256 lo verifica quien **no** tiene la clave y protege contra corrupción accidental; la etiqueta de GCM solo la puede verificar quien **sí** la tiene, y protege contra alteración deliberada.

El nonce es determinista, y eso es lo contrario de un descuido

`nonce = HMAC-SHA256(clave_archivo, "file_id:index")[:12]`. No se guarda en ninguna parte: se vuelve a derivar al descifrar.

Repetir un nonce con la misma clave en GCM no degrada la seguridad, la **elimina**: revela el XOR de los dos textos claros y permite falsificar mensajes. Un nonce aleatorio de 96 bits podría repetirse **sin que nada lo detecte**, que es lo peor de los dos mundos: el fallo es catastrófico y silencioso.

Dónde descansa la unicidad, que es lo que hay que destacar. No sobre un invariante nuevo inventado para el cifrado, sino sobre dos decisiones que ya estaban tomadas y probadas **antes de que el cifrado existiera**:

Lo que hace falta	Quién lo garantiza	Desde cuándo
Que `(file_id, index)` no se repita	`UNIQUE(file_id, index)` en `blocks`	Esquema del hito 1
Que un `(file_id, index)` no se cifre dos veces con contenidos distintos	**WORM**: un bloque cerrado no se reescribe nunca	Hito 1

Y esa es la parte que vale para el informe: **una propiedad criptográfica apoyada en una decisión de diseño previa es más fuerte que una apoyada en disciplina de implementación.** Si la unicidad dependiera de «acuérdate de no reutilizar un nonce», la rompe cualquier camino nuevo, en silencio, sin que falle ninguna prueba. Apoyada donde está, romperla exige **quitar una restricción de la base de datos** —que cazan la migración y su prueba de esquema— o **dejar de respetar el WORM**, que es una decisión de arquitectura que nadie toma por accidente.

Dicho de otro modo: **el cifrado no añadió ninguna obligación al resto del sistema.** Se colgó de obligaciones que el sistema ya tenía por otros motivos, y por eso no hay ningún sitio donde alguien tenga que “acordarse” de nada.

Pasa por HMAC en vez de concatenar los valores en claro para que quien mire el disco no pueda deducir la posición de un bloque a partir de su nonce. Y tiene un efecto útil: descifrar el bloque 1 con los bytes del 0 **falla**, en vez de devolver basura.

`files.size` es el claro; `blocks.size` es lo almacenado

GCM añade 16 bytes por bloque. Los dos tamaños dejan de coincidir, y hay que decidir qué guarda cada columna:

Columna	Significa	Por qué
`files.size`	bytes **claros**	Es el tamaño del archivo que el usuario subió, y el que `ls` y `stat` muestran
`blocks.size`	bytes **en disco**	Es contra lo que el cliente comprueba lo que descarga, y lo que cuentan la colocación, la cuota y el recolector

Por eso el cliente declara `cipher_overhead` en `/files/create`: el ControlNode planifica ahí los bloques y necesita saber cuánto ocupará cada uno **de verdad** antes de elegir destinos.

8.6 ACLs: cuatro reglas, una función, y ningún «denegar»

No hay reglas de denegación. Solo concesiones, y la ausencia de concesión es denegación. Es la decisión que mantiene el modelo explicable: en cuanto existe un «denegar» hay que definir qué gana cuando un ancestro permite y un descendiente niega, y en qué orden se evalúan los grupos, y ahí es donde estos sistemas dejan de poder contarse en un párrafo.

Tres permisos **ordenados**: `READ < WRITE < ADMIN`. Que estén ordenados y no sean un conjunto de banderas es lo que permite que la comprobación sea un `>=` y no una tabla.

Las cuatro reglas viven en **una sola función**, `domain/acl.resolve`, pura y probada sin base de datos:

1. **La herencia va HACIA ARRIBA.** Se sube por el árbol hasta la primera regla aplicable. No se propagan permisos a los hijos al conceder, porque eso se corrompe en cuanto se mueve un directorio: el hijo llevaría permisos de un padre que ya no es el suyo. Y `mv` es metadato puro, así que mover tiene que seguir siendo O(1).
2. **Gana la regla MÁS CERCANA, aunque conceda MENOS.** Es la única forma que tiene un modelo sin denegaciones de acotar el alcance: si se tomara el máximo de todo el camino, una concesión amplia arriba haría imposible dar menos permiso abajo.
3. **El MÁXIMO entre lo concedido al usuario y a sus grupos**, que es la consecuencia directa de no tener denegaciones: los permisos solo pueden sumar.
4. **El propietario es ADMIN de su árbol, y NO como un caso especial cosido aparte**: entra por la misma función. Eso es lo que hace que todas las pruebas de los hitos 1 y 2 —cada usuario en su propio árbol— sigan describiendo el mismo comportamiento sin tocar ni una.

Los grupos son **planos**: un grupo no contiene a otro, y el nombre es único por dueño, no globalmente. `/compartido-conmigo` es un directorio **virtual**: no existe como fila, se compone al vuelo con las

concesiones que apuntan a este usuario. No es una fila porque no tiene dueño ni padre, y darle uno obligaría a inventar reglas para renombrarlo o borrarlo.

El permiso mínimo se pasa como argumento. La firma es `directory_for(uow, user_id, path, minimum)`: **resuelve y exige a la vez**. La alternativa que parece más natural —y que es la que uno escribe primero— sería:

```
# NO. Es el mismo agujero que soy_el_lider(), con otra ropa.
permiso = resolve(usuario, ruta)
if permiso < Permission.WRITE:
    raise Forbidden()
escribir()
```

Los dos defectos son **literalmente los mismos** que los del liderazgo: una consulta que devuelve un dato es una consulta que alguien puede no mirar, y entre la comprobación y el uso hay una ventana. Con el mínimo en la firma, **el tipo obliga**: no hay forma de llamar a `directory_for` sin decir para qué.

8.7 Token de bloque: lo que hace que las ACLs sean ciertas

Conviene enunciar el problema antes que la solución, porque es el mejor ejemplo del proyecto de una decisión de arquitectura que **crea una obligación de seguridad en otro sitio**:

- El ControlNode resuelve permisos sobre **rutas**.
- Los bytes van **directos** entre cliente y DataNode (decisión 4).
- El DataNode **no sabe nada de rutas ni de usuarios**: almacena por `block_id` (decisión 2).

Juntando las tres: **el único sitio donde se comprobaba el permiso no estaba en el camino de los datos.** `GET /blocks/{id}` servía a cualquiera que supiera el identificador.

«Un UUID no se adivina» no es una respuesta, y merece la pena decir por qué: un `block_id` aparece en todo plan que el ControlNode haya entregado alguna vez, en los logs, y en el metadato de un archivo que se dejó de compartir. El día que a alguien se le retira el acceso, **los ids que ya vio siguen siendo válidos para siempre**. Seguridad por desconocimiento del identificador es exactamente lo que un sistema con ACLs explícitas no debería necesitar.

Cómo funciona

El ControlNode **firma** una autorización corta y el DataNode la **verifica sin preguntarle a nadie**. No hay viaje extra: el token va en el mismo plan que ya se entregaba.

```
token = base64url(payload) . base64url(firma) .
base64url(certificado del firmante)
```

`payload` lleva `block_id`, operación (`read`/`write`/`delete`), expiración y sujeto. La firma es RSA-PSS SHA-256 **sobre los bytes ya codificados**, no sobre el JSON reserializado: firmar lo que se transmite evita depender de que dos versiones de Python ordenen las claves igual.

La tercera parte es la que no es obvia. **El certificado del ControlNode viaja dentro del token** para que el DataNode no necesite tenerlo configurado: le basta **la CA**, que ya tiene por el mTLS. Rotar el certificado del ControlNode no obliga a tocar ningún DataNode, y esa es la propiedad que hace esto mantenible. Es la misma idea que `x5c` en JWS.

Camino	Quién pide el token	Operación
`dfsha put` → DataNode	El plan de `/files/create`	`write`
Pipeline DN1 → DN2 → DN3	**Se reenvía el mismo**, no se genera otro	`write`
`dfsha get` → DataNode	El plan de `/files/open`	`read`
Re-replicación: destino → origen	La orden `ReplicateBlock`	`read`
Recolector por REST → DataNode	La respuesta de `/gc/orphan-blocks`	`delete`
Recolector por el canal de control	*ninguno*	—

Las dos filas raras, que son las que hay que saber defender:

El pipeline reenvía el token tal cual, no fabrica uno nuevo. No podría: la clave que firma vive solo en el ControlNode, y esa es justamente la propiedad que hace que un DataNode comprometido no pueda autorizarse nada.

`DeleteBlock` por el canal de control NO lleva token, y la asimetría es deliberada: ese borrado lo ejecuta el propio nodo en su disco, sin salir a la red, y la orden llegó por el stream de heartbeat, que ya está autenticado con mTLS. **El canal ES la autorización.** Un token ahí sería ceremonia: el nodo tendría que presentárselo a sí mismo.

No es un secreto compartido, y la diferencia es la decisión

Un HMAC con clave compartida obligaría a que el DataNode tuviera la misma clave con la que se firma, y entonces **el DataNode podría emitir tokens**: autorizarse cualquier bloque, y quien comprometiera un DataNode se llevaría la capacidad de firmar para todo el clúster. Con firma asimétrica el DataNode solo puede **verificar**. Lo fija `test_el_token_NO_lleva_la_clave_que_lo_firma`.

Firmado por la CA no basta. Esta es la trampa

La intuición natural al implementarlo es «el DataNode valida que el certificado del firmante encadena a nuestra CA», y **eso solo sería un agujero**: la misma CA firmó `dfsha-control`, `dfsha-data` y `dfsha-client`. Cualquier DataNode podría emitirse tokens con su propio certificado y el de al lado se los aceptaría. Se habría construido un sistema de autorización que autoriza justo a quien debía limitar. Y no es hipotético: el DataNode **ya tiene** ese certificado y su clave privada, porque los necesita para el mTLS.

Por eso la verificación exige **dos** cosas, las dos obligatorias:

1. Que el certificado del firmante encadene a nuestra CA.
2. Que su Common Name sea `dfsha-control`: que el firmante sea **el rol con derecho a decidir**.

Lo fijan `test_un_datanode_no_puede_firmar_sus_propios_tokens` y `test_el_cliente_tampoco_puede_firmar`.

Tres detalles que parecen menores y no lo son

1. **El PUT autoriza ANTES de leer el cuerpo.** Si no, un no autorizado haría que el nodo se tragara 64 MB por la red antes de rechazarlo. Autorizar primero convierte un abuso de ancho de banda en una respuesta corta.
2. **El DELETE autoriza ANTES de la idempotencia.** Un bloque que no existe también exige token. Si el `204` de «no estaba» llegara primero, el endpoint sería un **oráculo** —204 si no existe, 403 si existe— con el que se puede enumerar el contenido de un disco sin autorización ninguna.
3. **El motivo del rechazo va al log, no a la respuesta.** Hacia fuera es siempre un 403 con el mismo texto.

El límite: no hay revocación

Retirar un permiso corta la **emisión** de tokens nuevos; no invalida los ya emitidos. Un token que alguien guardara sigue sirviendo hasta que caduque, y por eso la vida por defecto son **10 minutos**: cubre de sobra la subida o bajada de un bloque de 64 MB y coincide con el TTL de las reservas de escritura, que es el otro reloj de la misma operación.

Es el compromiso clásico de una credencial sin estado, y la alternativa —consultar al ControlNode en cada petición de bloque— devolvería el plano de control al camino de los datos, que es justo lo que la decisión 4 evita. Queda fijado como comportamiento esperado en `test_al_retirar_el_permiso_deja_de_emitirse_token`, para que sea una decisión escrita y no un descubrimiento en la sustentación.

Las dos mitades se encienden juntas

Sin material TLS, el ControlNode no firma y el DataNode no exige. Es la **misma condición** en los dos lados —hay certificados o no los hay—, y está escrito así a propósito: lo que no puede pasar es que una mitad esté y la otra no.

9. Entorno de ejecución y reproducibilidad

Esta sección está escrita para que alguien que no haya tocado el proyecto pueda levantarlo entero desde cero en su propia máquina y verificar por sí mismo cada afirmación del informe. Los comandos son literales y están en el orden en que hay que ejecutarlos. Donde un paso puede fallar, se dice **cómo se manifiesta el fallo** y **qué significa**, porque un error cuyo síntoma no se reconoce cuesta más que el paso entero.

Dos avisos antes de empezar, los dos aprendidos a base de tropezar con ellos:

1. **Clonar indicando la rama.** La rama por defecto del repositorio en GitHub fue `etapa-1` hasta el hito 3. Clonar sin indicar rama daba el proyecto de la semana 8. Usa siempre `--branch main`.
2. **En Windows, usa PowerShell y no Git Bash.** MSYS reescribe los argumentos que empiezan por `/` y convierte `/datos/pruebas` en `C:/Program Files/Git/datos/pruebas` antes de que la CLI los vea. Con PowerShell, CMD o WSL no pasa. Si hay que usar Git Bash, la salida es anteponer otra barra (`dfsha cd //datos/pruebas`) o `MSYS_NO_PATHCONV=1`.

9.1 Requisitos previos

Requisito	Versión	Por qué esa
Docker con Compose v2	cualquiera reciente	Once servicios en el despliegue local
Python	**3.11 o superior**	El paquete declara `requires-python = ">=3.11"`. Con 3.10 el `pip install` del cliente falla
Espacio en disco	~3 GB	Imágenes, volúmenes y los bloques de las pruebas
RAM disponible para Docker	~4 GB	Dos PostgreSQL, tres ControlNodes, cuatro DataNodes y nginx

El cliente corre **en la máquina anfitriona**, no en un contenedor. Es el escenario soportado y el único verificado de punta a punta; las limitaciones del contenedor `client` están documentadas al final de esta sección.

9.2 Despliegue local, paso a paso

Paso 1 — Clonar

```
git clone --branch main https://github.com/tsepulvedf/DFSha-Telematica-.git
cd DFSha-Telematica-
```

Paso 2 — Crear el `.env` (obligatorio)

`docker compose up` **aborta** si no existe `.env` con los secretos rellenos:

```
falta DFSHA_JWT_SECRET; copia .env.example a .env
```

Es a propósito: los secretos no tienen valor por defecto en el código, porque uno por defecto en un repositorio público es un hallazgo de seguridad.

bash / zsh / WSL:

```
cp .env.example .env
sed -i "s|^DFSHA_JWT_SECRET=$|DFSHA_JWT_SECRET=$(python -c 'import
secrets;print(secrets.token_urlsafe(48))')|" .env
sed -i "s|^DFSHA_PG_PASSWORD=$|DFSHA_PG_PASSWORD=$(python -c 'import
secrets;print(secrets.token_urlsafe(24))')|" .env
sed -i "s|^DFSHA_PG_REPLICATION_PASSWORD=$|
DFSHA_PG_REPLICATION_PASSWORD=$(python -c 'import
secrets;print(secrets.token_urlsafe(24))')|" .env

grep -E '^DFSHA_(JWT_SECRET|PG_PASSWORD|PG_REPLICATION_PASSWORD)=.+'
.env   # deben salir tres líneas
```

En macOS el `sed -i` del sistema pide un argumento: usa `sed -i ''`.

PowerShell:

```
Copy-Item .env.example .env
$jwt = python -c "import secrets;print(secrets.token_urlsafe(48))"
$pg  = python -c "import secrets;print(secrets.token_urlsafe(24))"
$rep = python -c "import secrets;print(secrets.token_urlsafe(24))"
(Get-Content .env) `
  -replace '^DFSHA_JWT_SECRET=$', "DFSHA_JWT_SECRET=$jwt" `
  -replace '^DFSHA_PG_PASSWORD=$', "DFSHA_PG_PASSWORD=$pg" `
  -replace '^DFSHA_PG_REPLICATION_PASSWORD=$',
"DFSHA_PG_REPLICATION_PASSWORD=$rep" | Set-Content .env

Select-String -Path .env -Pattern '^DFSHA_(JWT_SECRET|PG_PASSWORD|
PG_REPLICATION_PASSWORD)=.+'
```

`.env` está en `.gitignore`. No se sube nunca.

> **Regla del `.env` que conviene conocer:** una línea vacía significa «no la he configurado», **nunca** «vale la cadena vacía». Los dos servicios lo tratan así (`env_ignore_empty=True`). Esto no era así al principio y costó que el DataNode no arrancara en AWS siguiendo las instrucciones del propio proyecto; está contado en §10.6.

Paso 3 — Generar la CA y los certificados (obligatorio)

El plano interno usa **TLS mutuo**, y los servicios no arrancan sin su material:

```
python scripts/gen_certs.py
```

Deja en `certs/` la CA y un certificado por rol (`control`, `data`, `client`). Nada de esto se versiona.

El script **se niega a regenerar una CA existente** sin `--force`, y explica por qué: volver a firmarla invalida todos los certificados emitidos, y un clúster a medio rotar deja de hablar consigo mismo.

Esto **sustituye a `DFSHA_INTERNAL_SECRET`**, que ya no existe en el código. Un `.env` viejo que todavía lo tenga no estorba —se ignora— pero le faltarán las tres variables de TLS, y entonces el servicio no arranca y lo dice.

Paso 4 — Levantar el clúster

```
docker compose up --build -d
docker compose ps
```

Once servicios, en este orden de arranque:

Servicio	Qué es
`postgres-primary`	El metadato. Todas las escrituras van aquí
`postgres-replica`	Réplica en streaming. Sirve el lado de consulta de CQRS
`migrate`	Aplica las ocho migraciones de Alembic y termina
`control-node-1..3`	Tres instancias idénticas. Una sostiene el lease de líder
`lb`	nginx. Publica 8000 (REST), 8443 (plano interno) y 9000 (gRPC)
`data-node-1..4`	8001–8004, cada uno con su volumen y su dominio de falla

Comprobación inmediata:

```
curl http://localhost:8000/health     # {"status":"ok","service":"control-node"}
curl http://localhost:8001/health     # el DataNode, con used_bytes y capacity_bytes
```

Los diez servicios de larga vida deben quedar `healthy` (el undécimo, `migrate`, termina con código 0). Que el stack **levante** ya es información: los ControlNodes esperan a que `migrate` acabe, y los DataNodes y el balanceador esperan a sus dependencias con `depends_on: service_healthy`.

> **Si esto NO es la primera vez que levantas el clúster**, y los DataNodes entran en bucle de reinicio quejándose de `lb:9000`, ejecuta `docker compose restart lb` y vuelve a mirar. Es un caso conocido de nginx, no un fallo de configuración tuyo: está explicado en §6.6 y en la tabla de §9.6. En un arranque limpio no ocurre.

Paso 5 — Instalar el cliente

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -e ".[dev]"
```

Paso 6 — Reproducir el ciclo completo

```
export DFSHA_CONTROL_URL=http://localhost:8000      # PowerShell: $env:DFSHA_CONTROL_URL="http://localhost:8000"

dfsha register ana                  # pide la contraseña dos veces
```

```
dfsha login ana

dfsha cluster                       # los cuatro DataNodes ALIVE, con
su dominio;
                                    # última línea: líder, época y
vida restante del lease

dfsha mkdir -p /datos/pruebas
dfsha ls /

python scripts/gen_testfile.py /tmp/original.bin --size 50MB
#   anota el SHA-256 que imprime

# Bloques de 1 MB para que el particionamiento sea visible: 50
bloques, 150 réplicas
dfsha put --block-size 1048576 /tmp/original.bin
/datos/pruebas/original.bin
dfsha stat /datos/pruebas/original.bin     # bloques: 50 ·
FULLY_REPLICATED (3 de 3)
dfsha cluster                              # el reparto entre los
cuatro nodos

dfsha get /datos/pruebas/original.bin /tmp/bajado.bin
sha256sum /tmp/original.bin /tmp/bajado.bin    # tienen que
coincidir
```

En Windows, el último comando es `Get-FileHash /tmp/original.bin, /tmp/bajado.bin -Algorithm SHA256`.

Con el tamaño de bloque por defecto (64 MB) ese archivo cabría en **un** bloque, y el reparto no se vería. De ahí el `--block-size`.

Paso 7 — Ver el borrado lógico y el recolector

```
dfsha rm /datos/pruebas/original.bin     # borrado lógico: los
bloques siguen en disco
curl http://localhost:8001/health        # used_bytes todavía alto,
block_count intacto

python scripts/gc.py --dry-run           # lista los huérfanos sin
borrar nada
python scripts/gc.py                     # borrarlos de verdad

curl http://localhost:8001/health        # used_bytes y block_count
de vuelta a cero
```

El recolector se corre **desde la raíz del repositorio**: encuentra los certificados en `certs/` por su cuenta. Desde otro directorio hay que decírselo:

```
python scripts/gc.py --tls-ca-cert certs/ca.crt --tls-cert
certs/client.crt --tls-key certs/client.key
```

Presenta el certificado de **cliente**, no el del ControlNode, y eso importa: el recolector pide la lista de huérfanos y recibe con ella un **token de borrado por bloque**, firmado. No puede fabricarlos él, así que quien decide qué es un huérfano sigue siendo el ControlNode.

9.3 Verificar las propiedades del sistema, una por una

Elección de líder y relevo

```
dfsha cluster            # anota el id del líder y la época

docker kill dfsha-control-node-2      # kill, NO stop: ver la nota
de abajo
sleep 8
dfsha cluster            # otro líder, y la época exactamente una
más alta
```

`docker kill` y no **`docker stop`**, y la diferencia es toda la demostración: una parada ordenada **suelta** el lease, y entonces lo que se ve es un traspaso limpio y no la caducidad. El escenario que hay que enseñar es el del líder que desaparece sin avisar.

La **época** es el número a mirar. Sube cada vez que alguien toma un lease vencido y no baja nunca.

Caída de un DataNode y recuperación automática

```
# Con la gracia por defecto (5 min) hay que esperar. Para verlo en
el video, bájala:
#   DFSHA_REREPLICATION_GRACE_MS=30000 docker compose up -d

dfsha stat /datos/pruebas/original.bin    # FULLY_REPLICATED (3 de
3)
docker kill dfsha-data-node-2

# A los ~10 s: SUSPECT. A los ~30 s: DEAD y el archivo pasa a
UNDER_REPLICATED
watch -n2 dfsha cluster                   # PowerShell: while(1)
{dfsha cluster; sleep 2; cls}
dfsha stat /datos/pruebas/original.bin    # UNDER_REPLICATED

# Pasada la gracia, vuelve solo:
dfsha stat /datos/pruebas/original.bin    # FULLY_REPLICATED (3 de
3), sin intervención
dfsha get /datos/pruebas/original.bin /tmp/tras-caida.bin   # el
SHA-256 sigue coincidiendo
```

Los guiones de demostración

Hay cuatro guiones que reproducen los escenarios del hito **y comprueban el resultado**. Cada uno lleva un **control positivo** junto a la comprobación negativa, porque «no aparece el texto claro» y «un `grep` mal escrito» se parecen demasiado.

```
python scripts/demo/replicacion_y_caida.py      # R=3, caída, DEAD,
re-replicación, vuelta
python scripts/demo/failover_del_lider.py       # relevo con época,
servicio no se interrumpe
python scripts/demo/permisos_y_token.py         # ACLs y token de
bloque: 403 por block_id
python scripts/demo/cifrado_en_reposo.py        # el .blk en disco
NO contiene el texto claro
```

Todos esperan a que el clúster esté listo antes de empezar (al menos R DataNodes `ALIVE`, con reintento), así que lanzarlos justo después de `docker compose up` ya no produce falsos fallos. Los cuatro aceptan `--help`.

Ejecutar las pruebas

```
pip install -e ".[dev]"
python scripts/gen_proto.py       # genera el código del .proto (no
se versiona)

pytest -q                         # toda la suite: 640 en verde, 4
saltadas
pytest tests/unit -q              # rápido, sin red
pytest tests/integration -q       # levanta ControlNode y DataNodes
reales en puertos reales

# Las cuatro que se saltan necesitan PostgreSQL de verdad:
DFSHA_TEST_PG_URL="postgresql+psycopg://dfsha:<clave>@localhost:5432
/dfsha" pytest -q
```

Y la verificación por mutación, que rompe a propósito cada protección de seguridad y exige que las pruebas que dicen fijarla **caigan**:

```
python scripts/verificar_pruebas.py            # las 14 mutaciones
python scripts/verificar_pruebas.py --solo 5   # una sola
```

Tarda unos minutos porque varias arrastran pruebas de integración. Restaura siempre los ficheros en un `finally`, pase lo que pase.

9.4 Encender y apagar el TLS de cliente

El TLS de cara al cliente viene **apagado por defecto**, y es una decisión: el modo por defecto tiene que ser el que encuentra quien clona el repositorio y hace `docker compose up`, y que `curl http://localhost:8000/health` conteste sin configurar nada vale más que tenerlo encendido de serie.

Se enciende con un **fichero que se superpone**, no con variables en el `.env`: encender esto cambia a la vez el certificado de siete servicios, las ocho direcciones anunciadas y la configuración de nginx, y tienen que moverse juntos. No toca el `.env` ni los volúmenes, así que encender y apagar **no borra datos**.

```
# 1. Encender
docker compose -f docker-compose.yml -f docker-compose.tls.yml up -d
--build
docker compose ps                  # los diez healthy; las sondas ya
preguntan en HTTPS

# 2. El cliente del host cambia de URL
$env:DFSHA_CONTROL_URL = "https://localhost:8000"
dfsha login ana                    # la CA la toma de certs/ca.crt

# 3. Comprobar que va por HTTPS y que verifica de verdad
python -c "import httpx, ssl;
print(httpx.get('https://localhost:8000/health',
verify=ssl.create_default_context(cafile='certs/ca.crt')).json())"
#   -> {'status': 'ok', 'service': 'control-node'}
python -c "import httpx; httpx.get('https://localhost:8000/health')"
#   -> ConnectError ... CERTIFICATE_VERIFY_FAILED: sin NUESTRA CA no
se confía
python -c "import httpx;
print(httpx.get('http://localhost:8000/health').status_code)"
#   -> 400: nginx contesta «plain HTTP request was sent to HTTPS
port»

dfsha cluster                      # las direcciones salen como
https://localhost:800N

# 4. Apagar
docker compose up -d
$env:DFSHA_CONTROL_URL = "http://localhost:8000"
dfsha login ana
Remove-Item Env:DFSHA_CONTROL_URL
```

Las dos comprobaciones que no devuelven `ok` son la mitad que importa: un TLS que acepta cualquier certificado se ve igual que uno que verifica, y solo el rechazo lo distingue.

No se usa `curl.exe` en Windows porque va con Schannel, que con una CA propia sin lista de revocación falla **por la revocación y no por la cadena**: el rechazo saldría, pero por un motivo que no es el que se quiere enseñar.

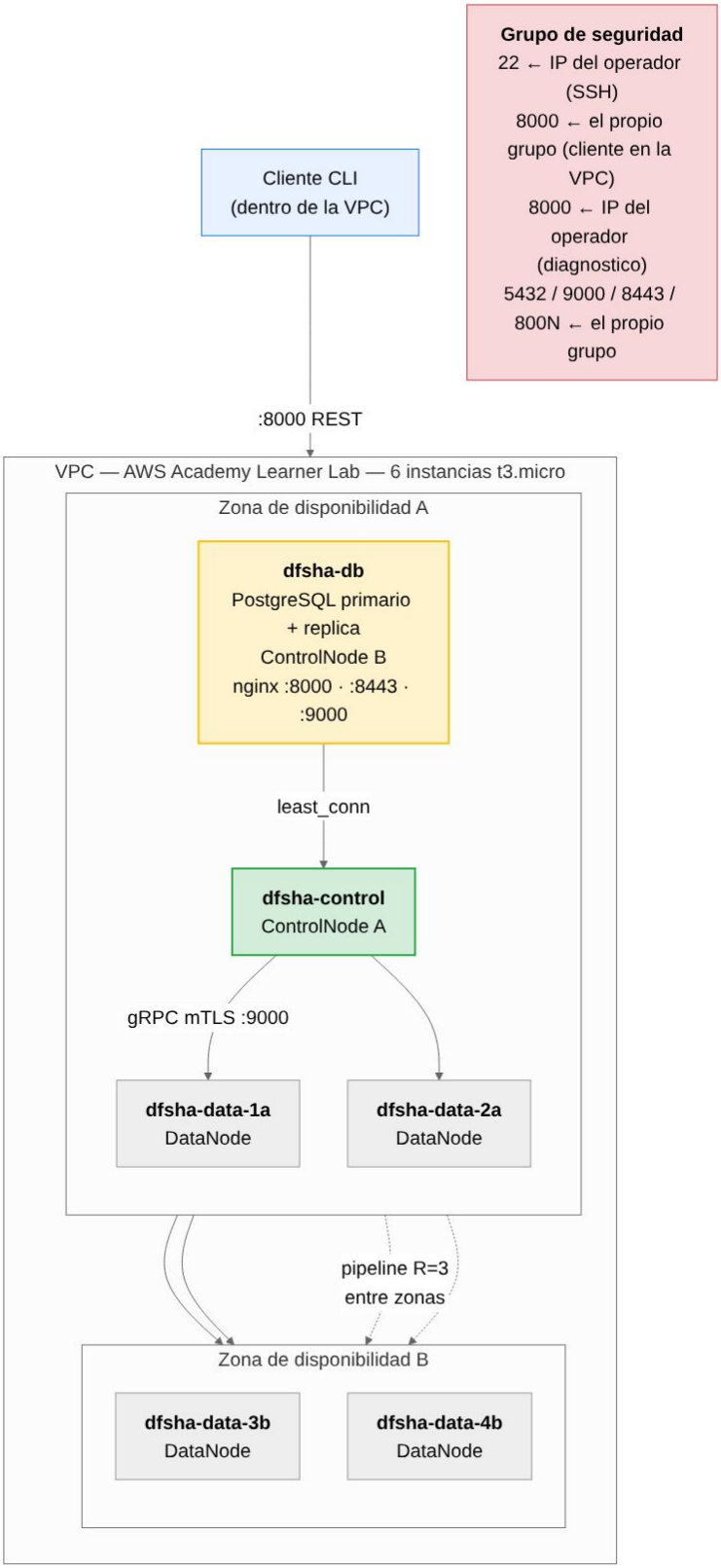
Qué hace el override, tramo a tramo:

Tramo	Con el override
cliente → nginx `:8000`	HTTPS con el certificado de control
nginx → ControlNode	HTTPS **verificado** contra la CA del proyecto

Tramo	Con el override
cliente → DataNode `:800N`	HTTPS con el certificado de datos
DataNode → DataNode (pipeline, re-replicación)	HTTPS **verificado** contra la CA
plano interno `:8443`, gRPC `:9000`	sin cambios: ya llevaban mTLS

nginx **termina** TLS en `:8000` y vuelve a cifrar hacia el ControlNode, al revés que en `:8443` y `:9000`. La razón es la de §6.6: en el plano interno el certificado es la identidad y no puede morir en el balanceador; en el de cliente la identidad es el JWT, que viaja dentro de la petición y llega intacto.

9.5 Despliegue en AWS



Topología del despliegue en AWS: seis instancias en dos zonas de disponibilidad.

Seis instancias `t3.micro` con Ubuntu **24.04** en dos zonas de disponibilidad de `us-east-1`. Los pasos exactos, con lo que cambia en cada instancia, están en **`deploy/README.md`**; esta sección da la secuencia y las decisiones.

Instancia	Zona	Qué corre
`dfsha-db`	A	PostgreSQL primario + réplica, **ControlNode B**, **nginx**
`dfsha-control`	A	ControlNode A
`dfsha-data-1a`, `dfsha-data-2a`	A	DataNode
`dfsha-data-3b`, `dfsha-data-4b`	B	DataNode

Por qué esta distribución y no otra

1. **Dos ControlNodes y nginx sin una séptima instancia.** Lo que faltaba no era un ControlNode: era **el balanceador**. Sin él, DataNodes y cliente apuntan a una IP, y matar al que atiende el tráfico enseñaría el traspaso del lease pero **no que el servicio continúa**. Medido en el compose local: ControlNode 90–95 MiB, PostgreSQL 62, nginx 20; `dfsha-db` queda en ~500–650 MB de ~950, con 1 GB de swap como red de seguridad.
2. **«Una instancia, un servicio» nació por los DataNodes** —un bloque en memoria por subida concurrente, y los dominios de falla— y **no aplica a un ControlNode**: no tiene estado propio y los bytes no lo atraviesan. Es la objeción que alguien planteará al ver tres servicios en una máquina, y tiene respuesta.
3. **El cliente corre dentro de la VPC**, en `dfsha-control`. Abrir el 8000 al exterior no bastaría: los bytes van directos a los DataNodes por IP privada, y la opción de túneles reintroduce la tensión de una sola dirección anunciada.
4. **El TLS de cliente va encendido.** El «apagado por defecto» es para quien clona; AWS es el despliegue real y ahí el tráfico cruza redes.
5. **Todo valor va escrito.** Los compose de `deploy/` **no interpolan** variables, y `DFSHA_DATANODE_PEER_URL` va explícita aunque coincida con la de cliente. El motivo está en §10.6: la interpolación leía el `.env` equivocado y producía una URL vacía que pisaba la correcta.

Reglas del grupo de seguridad

Puerto	Origen	Para qué
22	IP del operador	SSH
8000	**el propio grupo de seguridad**	**El cliente corre dentro de la VPC. Esta es la regla que hace funcionar el escenario soportado**
8000	IP del operador	Diagnóstico desde fuera. Solo para eso
8001–8004	el propio grupo + IP del operador	REST de los DataNodes
5432	el propio grupo	PostgreSQL

Puerto	Origen	Para qué
9000, 8443	el propio grupo	gRPC y plano interno

Las dos reglas del 8000 están aquí con su motivo escrito al lado, y eso es deliberado. La versión anterior de este material abría el 8000 **solo a la IP del operador**, mientras otra sección del mismo documento explicaba que el escenario soportado es el cliente corriendo dentro de la VPC. Las dos frases son defendibles por separado y juntas no funcionan: con esas reglas, el cliente que el documento recomienda **no puede alcanzar nginx**. El síntoma fue un `ConnectTimeout` sin llegar al *handshake*. Decir para qué es cada regla es lo que impide que alguien vuelva a quitar la que parece redundante. El caso completo está en §10.6.

Secuencia de despliegue

```
# ── En cada una de las seis instancias ──────────────────────────
sudo apt-get update && sudo apt-get install -y docker.io docker-compose-v2 git
sudo usermod -aG docker ubuntu
exit                      # hay que RECONECTAR para que el grupo tenga efecto
#   (saltarse esto es el error más común: todos los docker siguientes piden sudo)

ssh -i clave.pem ubuntu@<ip>
git clone --branch main https://github.com/tsepulvedf/DFSha-Telematica-.git
cd DFSha-Telematica-

# ── Certificados: se generan UNA VEZ, en una máquina, con TODOS los nombres ──
python3 scripts/gen_certs.py --hosts <lista de IP privadas y nombres>
#   los nombres de PAR no son opcionales: sin ellos el pipeline falla en el
#   handshake en vez de en el connect (§3.3, §8.3)

# A cada instancia se le copian ÚNICAMENTE sus tres ficheros.
# ca.key se queda fuera del despliegue.
chown 1000:1000 certs/*          # el uid del contenedor; NO confiar en que coincida
chmod 600 certs/*.key

# ── dfsha-db ────────────────────────────────────────────────────
cp deploy/.env.db.example deploy/.env       # y rellenar los secretos
docker compose -f deploy/docker-compose.db.yml up -d
docker compose -f deploy/docker-compose.db.yml run --rm migrate    #
```

```
las ocho migraciones

# ── dfsha-control ──────────────────────────────────────────────
cp deploy/.env.control.example deploy/.env  #
DFSHA_REPLICATION_FACTOR=3
docker compose -f deploy/docker-compose.control.yml up -d

# ── cada dfsha-data-N ──────────────────────────────────────────
cp deploy/.env.datanode.example deploy/.env
#   DFSHA_DATANODE_ADVERTISE_URL y DFSHA_DATANODE_PEER_URL con la IP
PRIVADA
#   DFSHA_DATANODE_FAULT_DOMAIN = la zona de disponibilidad real
docker compose -f deploy/docker-compose.datanode.yml up -d

# ── Verificación, desde dfsha-control ──────────────────────────
export DFSHA_CONTROL_URL=https://<ip-privada-db>:8000
dfsha register ana && dfsha login ana
dfsha cluster                                   # 4/4 ALIVE en dos
dominios reales
python scripts/gen_testfile.py /tmp/f.bin --size 50MB
dfsha put --block-size 1048576 /tmp/f.bin /f.bin
dfsha stat /f.bin                               # FULLY_REPLICATED (3
de 3)
dfsha get /f.bin /tmp/b.bin && sha256sum /tmp/f.bin /tmp/b.bin
```

Para la demostración del relevo del líder en AWS, `scripts/demo/failover_del_lider.py` **no sirve** porque usa `docker exec` local. La secuencia manual por SSH está en `deploy/README.md`, paso 9, y usa `docker kill` y no `stop` por el motivo de §9.3.

9.6 Problemas frecuentes y qué significan

Esta tabla es el resultado de haber tropezado con cada uno. El valor no está en la solución, sino en poder **reconocer el síntoma**: en un sistema con relevos y reintentos, el error de superficie casi nunca apunta a la causa.

Síntoma	Causa real	Solución
Los cuatro DataNodes en bucle de reinicio justo después de un `up -d --build`, con `datanode.register_retry` y `code: UNAVAILABLE` contra `lb:9000`	nginx resuelve los `upstream` **al cargar la configuración**. El `--build` recrea los tres ControlNodes y **no** el `lb`, cuya configuración no cambió: el balanceador se queda con las tres IP viejas, y como quedan obsoletas **las tres a la vez**, no hay ninguna a la que pasar	`docker compose restart lb`. Ver §6.6

Síntoma	Causa real	Solución
`falta DFSHA_JWT_SECRET` al hacer `up`	No hay `.env`, o la línea está vacía	Paso 2 de §9.2
El servicio no arranca y habla de `certs/`	No se ejecutó `gen_certs.py`	Paso 3 de §9.2
`DFSHA_DATANODE_CAPACITY_BYTES: Input should be a valid integer`	Una variable numérica puesta como `VAR=` en el `.env`	Dejarla **fuera** del fichero, no vacía. Desde el arreglo, una línea vacía ya cuenta como no puesta
`409 no alcanzan el quórum de escritura (W=2)`	**Casi nunca es capacidad del clúster.** Es un camino nodo-a-nodo que no conecta: `DFSHA_DATANODE_PEER_URL` mal puesta, o certificados sin los nombres de par	Revisar `peer_url` en `dfsha cluster` y el SAN de `data.crt`
`ConnectTimeout` contra el 8000 en AWS	Regla del grupo de seguridad: falta el 8000 abierto al **propio grupo**	§9.5
`Server disconnected without sending a response` en un cliente con certificado	**Es un fallo del cliente, no del servidor**: httpx descarta `cert=` si `verify` es una ruta	Usar `common/tls.client_ssl_context`
`ninguna de las N réplicas pudo servir el bloque`	**No es disponibilidad.** Suele ser un desajuste de tamaños o de checksum	Mirar `block.replica_failed` en los logs: trae el error real de cada réplica
`Connection refused` desde el contenedor `client`	El plan trae `localhost`, y ahí `localhost` es el propio contenedor	Usar el cliente del host, que es el escenario soportado
`no has iniciado sesion` tras un login correcto en el contenedor `client`	La sesión no sobrevive entre invocaciones de `docker compose run`	Encadenar los comandos en una sola invocación, o usar el cliente del host
`docker` pide `sudo` en AWS tras instalarlo	Falta reconectar la sesión SSH después del `usermod -aG`	`exit` y volver a entrar
`git pull` no trae nada y no dice nada	La rama local no tiene *upstream*	`git branch --set-upstream-to=origin/main main`
Rutas que se convierten en `C:/Program Files/Git/...`	Git Bash en Windows	PowerShell, o `MSYS_NO_PATHCONV=1`

9.7 Puertos

Puerto	Servicio	Protocolo	Quién lo usa
8000	nginx → ControlNode	REST (HTTP/HTTPS)	El cliente
8001–8004	DataNode	REST (HTTP/HTTPS)	El cliente y otros DataNodes
8443	nginx → ControlNode	Plano interno, **mTLS**, proxy TCP	El DataNode y el recolector
9000	nginx → ControlNode	gRPC, **mTLS**, proxy TCP	El DataNode
5432	PostgreSQL	—	Los ControlNodes

Documentación interactiva de las APIs REST: `http://localhost:8000/docs` y `http://localhost:8001/docs`.

10. Pruebas y análisis de resultados

10.1 Estrategia

Tres niveles, cada uno con un propósito que los otros dos no pueden cubrir:

Nivel	Qué prueba	Por qué no basta con el anterior
Unitarias	Reglas de dominio puras: resolución de rutas, ACLs, pertenencia con reloj inyectado, aritmética de colocación	—
Integración	ControlNode y DataNodes **reales** en puertos reales, con el cliente real	Un cliente simulado se salta justo el camino que importa: metadato contra el ControlNode, bytes contra el DataNode
Validación en despliegue	`docker compose` y AWS, a mano y con guiones	Las pruebas de integración corren los nodos como hilos en un proceso sobre `127.0.0.1`: **la dirección del cliente y la del vecino son la misma**, así que hay fallos de direccionamiento que no pueden distinguir

Ese tercer nivel no es una formalidad. La tercera columna de esa tabla es, literalmente, la razón por la que existe la sección 10.6.

10.2 La suite

640 pruebas en verde, 4 saltadas. Las cuatro que se saltan son `tests/integration/test_leadership_postgres.py`, que necesitan PostgreSQL de verdad porque la exclusión mutua del lease depende de `SELECT … FOR UPDATE`, que SQLite no implementa y SQLAlchemy **omite en silencio** en ese dialecto. Se saltan para que `pytest` siga corriendo en un portátil sin Docker; el CI levanta un servicio PostgreSQL y las ejecuta en cada push.

Qué cubren, por hito:

Hito 1 y 2. Round-trip de 50 MB en 50 bloques con SHA-256 idéntico; archivo vacío y de un byte; checksum incorrecto rechazado con `422` y commit fallando con `409`; aislamiento entre usuarios; `mv` sin tocar el DataNode; reserva vencida con `410`; ciclo completo del recolector con `used_bytes` restaurado. Con cuatro DataNodes reales: los cuatro registrados y `ALIVE` con su dominio, 50 bloques repartidos sin que ninguno se lleve más del 60 %, un nodo con menos capacidad recibiendo una fracción menor, el ciclo completo de caída, y un `.blk` borrado a mano que acaba en `MISSING` **sin que el ControlNode borre nada**.

Bloque A. Que la migración de Alembic y `models.py` describen *exactamente* el mismo esquema (`compare_metadata` con cero diferencias); el enrutado de consultas con sus cuatro caminos de fallo;

el lease con reloj inyectado; y la exclusión mutua contra PostgreSQL con ocho hilos saliendo de una barrera común.

Bloque B. Cada bloque en tres nodos y tres dominios; el cliente hablando con **un** nodo por bloque mientras los otros dos reciben su copia de la cadena; un nodo caído durante la subida que no impide el commit y deja el archivo `UNDER_REPLICATED`; el archivo legible con **dos de tres nodos caídos** y el SHA-256 intacto; el ciclo completo de recuperación; y `test_un_reinicio_rapido_NO_dispara_una_copia`, que es la mitad que no se ve si solo se prueba el camino feliz.

Bloque C. mTLS con el caso bueno **primero** —si ese falla, los de rechazo no prueban nada—; el token de bloque con sus dos consecuencias (un token de un bloque no abre otro; leer no autoriza a borrar); que un DataNode no puede firmar sus propios tokens; que con el metadato entero y el disco no se descifra sin la contraseña; las cuatro reglas de ACL sin base de datos; y el RF3 completo.

Los umbrales de tiempo van comprimidos en las pruebas de integración (1,5 s y 3 s) para que la suite no tarde minutos; la aritmética de los valores reales está cubierta por unitarias con reloj inyectado. Es una decisión consciente: una suite que tarda media hora no se ejecuta.

10.3 Verificación por mutación: cómo sabemos que las pruebas prueban algo

El problema

Una prueba que nunca has visto fallar no sabes si prueba algo.

Normalmente eso no preocupa: una prueba se escribe porque algo fallaba, se la ve en rojo, se arregla el código, se la ve en verde. Ha fallado al menos una vez y por el motivo correcto.

Las protecciones de seguridad del hito 3 no funcionan así. Se escribieron **antes** de que hubiera un fallo que arreglar, contra escenarios que hay que montar a propósito —un proceso congelado, un certificado de otro rol, un lock caducado— y comparten una propiedad incómoda: **quitarlas no rompe nada visible.** El sistema sigue funcionando, la suite sigue en verde salvo unas pocas pruebas, y el agujero solo se manifiesta con una pausa larga en el momento justo.

Con ese perfil, una prueba puede estar en verde **por el motivo equivocado** durante meses sin que nadie lo note.

Qué se hizo

Romper cada protección **a propósito** y exigir que las pruebas que dicen fijarla caigan. Si no caen, la prueba no cubre lo que su nombre anuncia.

Lo automatiza `scripts/verificar_pruebas.py`: aplica una mutación, corre las pruebas correspondientes, y **restaura el fichero en un `finally`** pase lo que pase. Catorce mutaciones: diez sobre las cuatro protecciones del patrón, una sobre el cifrado y tres sobre el TLS de cliente.

El hallazgo: tres pruebas pasaban por el camino equivocado

Prueba	Decía fijar	Pasaba en realidad porque	Cómo se vio
`test_el_lider_congelado_es_rec`	la comparación de **época**	A y B son instancias distintas: le bastaba el	Al quitar la comparación de época

Prueba	Decía fijar	Pasaba en realidad porque	Cómo se vio
`hazado_y_no_escribe_nada`		`leader_id`, y la época no se ejercitaba	**seguía en verde**
`test_el_token_de_Beto_no_abre_el_bloque_de_Ana`	que el **`block_id`** viaja firmado	usaba un token de **escritura** contra un endpoint que pide uno de **lectura**: rechazaba por la operación, no por el bloque	Al quitar la comprobación del `block_id` **seguía en verde**
`test_el_cliente_congelado_no_escribe_encima` (RF3)	el vencimiento del **lease**	simulaba la congelación **soltando** el lock, y soltar reinicia la época mientras que caducar la sube: dos caminos distintos	Al revisarla contra la prueba hermana

La primera es la más elocuente, y conviene decirlo sin suavizarlo: es la prueba con el nombre más rotundo del módulo de liderazgo, **se pidió explícitamente en el punto de control del Bloque A, y se dio por buena por las dos partes**. Describía el escenario correcto, afirmaba en su *docstring* que era «la razón de ser de la época», y la época no intervenía en su resultado.

Ninguna de las tres era un agujero de cobertura: entre las pruebas hermanas el caso estaba cubierto. Lo que eran es **confianza sin respaldo**, y eso es peor que un hueco conocido, porque un hueco se mira y una prueba en verde no.

La formulación, que es lo que vale

> **Una prueba que monta un escenario PARECIDO al que dice cubrir puede pasar por una razón distinta de la que su nombre anuncia.**
>
> Leerla no lo detecta —la prueba es correcta, el escenario es correcto, el nombre describe la intención—. Lo detecta **romper lo que dice proteger y comprobar que cae**.

Las tres se encontraron igual: **no leyéndolas**. Se habían leído varias veces, y en la lectura no hay nada que chirríe. Lo que falta no está en el texto de la prueba, está en la **relación entre la prueba y el código**, y eso solo se ve ejecutando las dos juntas en el estado roto.

La herramienta avisa cuando no puede mutar

Cada mutación busca un fragmento **exacto** de código. Si el código cambia y el fragmento ya no existe, la herramienta lo reporta como `NO SE PUDO MUTAR` en vez de saltárselo en silencio.

Es una línea de código y es la diferencia entre una herramienta y un script que dentro de tres meses no encuentra nada sin que nadie sepa por qué. **Un verificador que falla en silencio tiene exactamente el defecto que existe para detectar.**

Coste y límite, dichos entero

Catorce mutaciones tardan unos minutos porque varias arrastran pruebas de integración. **No está en el CI**: se corre a mano al tocar una de las cuatro protecciones, que es cuando importa. Meterlo en cada push multiplicaría el tiempo de la suite por el número de mutaciones a cambio de detectar algo que solo cambia cuando alguien edita esos ficheros.

El límite honesto: esto verifica **las mutaciones que a alguien se le ocurrieron**. Una protección puede romperse de una forma que no está en la lista, y entonces no se entera nadie. No es análisis exhaustivo; es una red contra el error concreto de dar por cubierto algo que no lo está.

10.4 Resultados medidos en el despliegue local

Validado con `docker compose up --build -d`, pasada completa el 18 de septiembre de 2026:

Qué	Medido
Servicios	Los diez de larga vida `healthy`; los tres ControlNodes arrancando por `python -m`
Clúster	4/4 `ALIVE` en cuatro dominios; el balanceador repartiendo entre instancias
`put` de 50 MB en bloques de 1 MB	Commit correcto, `FULLY_REPLICATED`
Reparto de bloques	**11 / 13 / 14 / 12** entre los cuatro nodos
Detección de caída	`SUSPECT` a los **24 s**, `DEAD` a los **37 s**
Relevo del líder	**5,8 s**, época de 2 a 3 — dentro de los 10 s exigidos
Los cuatro guiones de demostración	Pasan los cuatro, con sus controles positivos

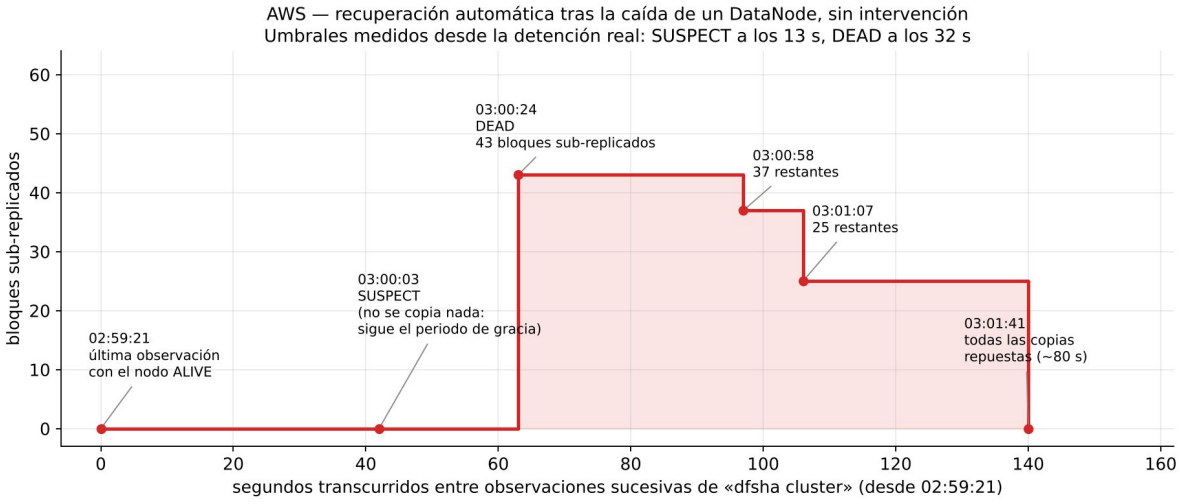
Y con el TLS de cliente encendido (`docker-compose.tls.yml`):

Qué	Medido
Servicios	Los diez `healthy` **con el override**. Que el stack levante ya es la prueba de las sondas: con las de `http://` fijo no habría arrancado
Direcciones	Las cuatro como `https://localhost:800N`; liderazgo en época 3
`put` de 50 MB	`FULLY_REPLICATED`, **3 de 3 copias por bloque**: el pipeline entre nodos cruzó HTTPS verificando la CA propia
Ida y vuelta	SHA-256 idéntico (`6d081597…`), con TLS en todos los saltos y cifrado extremo a extremo por debajo
Apagado	Vuelve a `http://` sin tocar el `.env` ni los volúmenes

10.5 Resultados medidos en AWS

Ejecutado y validado el **9 de octubre de 2026**. Seis `t3.micro` con Ubuntu 24.04 en `us-east-1b` y `us-east-1d`, las ocho migraciones de Alembic sobre PostgreSQL, los tres servicios de `dfsha-db healthy` —incluido `lb`, cuya sonda llega a un ControlNode por HTTPS verificando la CA— y los cuatro DataNodes registrados **al primer intento**, con mTLS y HTTPS.

Qué	Medido
Clúster	4/4 `ALIVE` en **dos dominios de falla reales**, direcciones `https://` con IP privadas
`put` de 50 MB en bloques de 1 MB	`FULLY_REPLICATED (3 de 3)`; **150 réplicas repartidas 36 / 38 / 39 / 37**
Ida y vuelta	SHA-256 idéntico (`6d081597f6bd4b3c6bff8b558683f7ca5ac16d2176cd34f089f45ce9d628f218`), con cifrado extremo a extremo
El balanceador reparte	«líder: otra instancia (`8867fae8`) … te atendió `0a4bf411`»
Caída de un DataNode	`ALIVE` → `SUSPECT` en 13 s → `DEAD` en 32 s con **43 bloques sub-replicados** → **todas las copias repuestas a los ~80 s**, sin intervención
Reincorporación	`FULLY_REPLICATED (3-4 de 3)`: la sobre-replicación documentada en §7.7
Relevo del líder	Se identificó al líder por `is_self` y se mató **a él**: época **1 → 2** en ~10 s, con `previous_leader` llevando el id del muerto
Durante el relevo	`ls` y `get` de 50 MB correctos justo después del `kill`; los cuatro DataNodes siguieron `ALIVE`; y un `put` de 50 MB **con un solo ControlNode vivo** quedó `FULLY_REPLICATED`



Recuperación automática medida en AWS tras detener un DataNode.

El evento del relevo, tal como lo emitió el ControlNode que tomó el lease:

```
{"event": "leadership.acquired", "leader_id": "0a4bf411...",
"epoch": 2,
 "previous_leader": "8867fae8...", "self_recovery": false,
"duration_ms": 10.198}
```

Dos observaciones que valen para la sustentación

La primera. Con el nodo DEAD, `dfsha cluster` mostró `bloq. 82 / repl. 0` para ese nodo: los bloques siguen en su disco, las réplicas disponibles son cero. Esa es exactamente la distinción entre **lo que hay** y **lo que se puede servir** que protege la decisión de no borrar datos por una divergencia (§5.5), hecha visible sin abrir un solo log.

La segunda, y es la que justifica una decisión de topología. El `put` que confirma con un solo ControlNode vivo es lo que **un segundo ControlNode sin balanceador no habría podido enseñar**, porque el aviso síncrono `/stored` siguió llegando por nginx. Era exactamente el argumento de la decisión 1 de §9.5: lo que faltaba no era un ControlNode, era el balanceador.

10.6 Veintidós casos en siete capas: el modo de fallo de este proyecto

Esta sección es, con diferencia, la que más enseñó, y va al informe entera porque un informe que solo cuenta lo que salió bien no dice si el equipo sabe por qué salió bien.

Los fallos serios del proyecto han sido todos el mismo tipo de cosa: algo que aparenta estar puesto y no lo está. Ninguno se detectó leyendo el código.

El inventario

Fallo	Qué parecía	Qué era	Cómo se detectó
Direccionamiento entre nodos	El plan llevaba la dirección de cada réplica	Llevaba la del **cliente** en un camino entre nodos	Validando en **Docker**, no en las pruebas
`cert=` de httpx	El cliente presentaba su certificado	httpx lo **descartaba** sin avisar	Midiendo tres combinaciones, no asumiendo
`blocks.size` vs `files.size`	El metadato describía el bloque almacenado	Describía el **claro**, y en disco había 16 bytes más	Una prueba de ida y vuelta, no de inspección
`details` del error	El cliente imprimía los datos del error	Los **descartaba**; la rama que los imprime no se ejecutó nunca	Una prueba que necesitaba uno de esos datos
El arranque rápido del README	Los comandos levantaban el clúster	Generaba un secreto que ya no se lee y **no mencionaba los certificados**	Actualizando la documentación, no usándola
El punto de entrada	«529 pruebas en verde» decían que el servicio estaba bien	`__main__.py` **no compilaba**: ninguna prueba lo importaba	Levantando el stack

Fallo	Qué parecía	Qué era	Cómo se detectó
`CAPACITY_BYTES=` vacía	Seguir el ejemplo de despliegue levantaba el DataNode	El DataNode **no arrancaba**; el compose local lo tapaba con un default	Ejecutando el punto de entrada, no compilándolo
Los guiones de demostración	Cuatro guiones listos para grabar	Ninguno se había ejecutado; dos **no arrancaban en Windows**	Ejecutándolos en Windows, donde se graban
El cifrado de los usuarios anteriores a la migración 0006	Todo archivo se cifraba; diez pruebas de cifrado en verde	Sin sal no había clave, y el `put` **subía en claro sin avisar**	Un guion con controles positivos fiables encontró la frase en un `.blk`
`verificar_pruebas.py`	La herramienta que verifica las pruebas	Tenía la raíz del repositorio **escrita a mano**: solo corría en una máquina	Leyendo su código al añadirle una mutación
El DataNode con TLS de cliente	El pipeline verificaba a sus pares	Verificaba contra el almacén del **sistema**, sin nuestra CA: cada reenvío fallaba	Escribiendo los pasos para encenderlo, antes de ejecutarlo
Encender el TLS de cliente	Estaba implementado, probado con nueve pruebas y documentado	**No se podía encender**	Lo mismo: intentar escribir los pasos exactos
La rama por defecto de GitHub	Quien clonara obtendría el proyecto	Obtenía **`etapa-1`**: `main` iba cuatro etapas por delante	Al hacer el merge del hito 3, mirando `origin/HEAD`
URL interna del DataNode en AWS	El compose pasaba la URL del plano interno	La interpolación leía otro `.env`: salía `https://:8443` y **pisaba la correcta**	Repasando `deploy/`, confirmado con `docker compose config`
`DFSHA_REPLICATION_FACTOR` en AWS	El ejemplo desplegaba el hito 3	Traía `R=1`: el hito 3 **sin replicación**, funcionando en apariencia bien	Repasando `deploy/`
Python del despliegue en AWS	Ubuntu 22.04 servía	Trae 3.10 y el cliente exige 3.11: su `pip install` **fallaba**	Repasando `deploy/` contra `requires-python`
Permisos de las claves en AWS	`chmod 600` las dejaba legibles por el servicio	Legibles **por coincidencia** entre el uid de `ubuntu` y el del contenedor	Repasando `deploy/` contra el `USER` de los Dockerfiles
El grupo de seguridad	El despliegue permitía	Abría el 8000 **solo a**	**Ejecutando el**

Fallo	Qué parecía	Qué era	Cómo se detectó
de AWS	el escenario que su propio README recomienda	**una IP externa**, y el cliente va dentro de la VPC	**despliegue**: ningún repaso lo vio

Más las **cuatro pruebas que no probaban** (§10.3 y el caso de la migración 0006).

La cuenta, y por qué conviene hacerla bien

Capa	Casos	Cuántos
Código	Direccionamiento, `cert=` de httpx, `blocks.size`, los pares del DataNode sin la CA	4
Cliente	Los `details` descartados desde el hito 1	1
Documentación	El arranque rápido con un secreto muerto y sin certificados; el grupo de seguridad que impedía el escenario que el propio documento recomienda	2
Arranque	El `__main__.py` que no compilaba; el `.env` vacío que tumbaba el DataNode; los guiones que no arrancaban en Windows; `verificar_pruebas.py`; el TLS de cliente que no se podía encender; y los cuatro del despliegue en AWS	9
Migración	La 0006, que dejó sin sal a los usuarios existentes y con ellos el cifrado desactivado en silencio	1
Repositorio	La rama por defecto de GitHub	1
Pruebas	Tres que pasaban por un camino distinto del que su nombre anunciaba, y la del `append` «con cifrado» que no comprobaba el cifrado	4
	Total	**22 en 7 capas**

El caso más grave: una garantía de seguridad que no se cumplía, con todo en verde

La migración 0006 añadió `users.kdf_salt` con `server_default=""` y **no rellenó las filas existentes**. Un usuario sin sal no recibe sal en el login, el cliente no deriva clave maestra, y el `put` tenía esta rama:

```
clave_archivo = new_file_key() if maestra else None   # sin clave: se sube EN CLARO
```

Pensada para compatibilidad con el hito 2, **subía el archivo en claro, con código de salida 0 y sin una sola línea de aviso**. Todo usuario creado antes del Bloque C tomaba esa rama para siempre. En la base del compose de desarrollo, consultada después: **ningún archivo se había cifrado nunca**, con el criterio de cifrado dado por cumplido y 625 pruebas en verde, diez de ellas de cifrado.

Por qué no lo vieron las pruebas. Dos motivos independientes; bastaba con uno:

1. **Las pruebas de cifrado no pasaban por el `put` del cliente.** `_subir_cifrado` reimplementaba «lo que hace `dfsha put`», y era **la prueba** la que decidía cifrar. La decisión real del cliente no la recorría ninguna. Lo demuestra la mutación 11: devolver el cliente a la rama vieja deja en verde las ocho pruebas de cifrado anteriores.
2. **Todas creaban sus usuarios en el momento**, y un usuario recién registrado siempre tiene sal. El estado que deja una migración sobre **filas existentes** no lo montaba nadie.

El «no encontrado» que dio por bueno lo contrario durante dos días. El día anterior se hizo el mismo `grep` a mano, con un archivo equivalente, y salió NO ENCONTRADO. Al ver el fallo, la primera hipótesis fue «algo cambió entre ayer y hoy», y era **falsa**: consultada la base, ningún archivo se había cifrado nunca, así que aquel resultado fue un **falso negativo** — lo más probable, que se buscó en un nodo que no tenía el bloque, cosa que con R=3 y cuatro nodos pasa con uno de cada cuatro.

Lo que va al informe con estas palabras: **un «no encontrado» sin control positivo no es evidencia de nada, y aun así se lee como evidencia de lo que uno espera.** Durante dos días, una comprobación manual certificó que los bloques estaban cifrados cuando ninguno lo estaba. Lo que convierte ese «no encontrado» en evidencia es exactamente lo que se añadió al guion: comprobar antes que la búsqueda **encuentra** la frase cuando está, y que se busca en un nodo que **tiene** el bloque.

El arreglo, en tres capas, porque cada una tapa un modo distinto de volver a caer:

Capa	Qué	Lo fija
Datos	La migración **0008** da una sal propia a cada usuario que no la tenga	`test_la_0008_da_sal_a_los_usuarios_que_la_0006_dejo_sin_ella` (vista caer sin la 0008)
Cliente	`put` **se niega** a subir sin clave. «Sin cifrar» sigue siendo un estado legible, pero ya no algo que el cliente produzca	`test_un_usuario_sin_sal_no_sube_en_claro` (vista caer con la mutación 11)
Pruebas	El criterio comprobado por el **`put` real del CLI**, mirando el `.blk`	`test_el_put_del_CLI_cifra_lo_que_queda_en_disco`

La segunda capa es la que más importa: aunque mañana otra migración vuelva a dejar a alguien sin sal, el síntoma será un error ruidoso en el `put` y no datos en claro.

La lección: *una garantía de seguridad con una rama que la desactiva en silencio no es una garantía.* La rama era razonable —compatibilidad con el hito anterior—, y el defecto no era su existencia sino que

degradaba sin avisar. Y la otra mitad: *una prueba que reimplementa el camino que dice probar prueba la reimplementación.*

El caso límite: implementado, probado, documentado, y no se podía encender

El TLS de cliente estaba **implementado, probado con nueve pruebas y documentado**, y **no se podía encender**. Y con el matiz que lo hace elocuente: las sondas de salud con `http://` fijo no dejaban el stack con mal aspecto, lo dejaban **sin arrancar**, porque el balanceador y los DataNodes esperan a sus dependencias con `depends_on: service_healthy`. Encenderlo no degradaba el sistema: **lo apagaba**.

Es el caso límite del patrón. Los demás son «algo dejó de ser verdad y lo que lo afirmaba no se enteró»: un README que envejeció, una migración que dejó filas atrás. Aquí **nunca llegó a ser verdad**, y todo lo que lo rodeaba decía que sí: el código existía, las nueve pruebas estaban en verde, el README explicaba qué protegía y el `.env.example` documentaba las dos variables. **Nada de eso comprobaba que se pudiera encender, y nadie lo había encendido.**

Las dos decisiones de diseño del arreglo que hay que saber defender:

1. **La sonda decide el esquema con la MISMA variable que decide el servidor**, así que **no pueden discrepar**. Dos variables —una para el servidor y otra para la sonda— habrían reproducido exactamente el fallo, esperando a que alguien olvidara cambiar una.
2. **nginx verifica al ControlNode contra la CA** (`proxy_ssl_verify on`), no se limita a cifrar. Y hay una prueba que lo fija **porque `proxy_ssl_verify off` habría funcionado a la primera**: es la opción que nadie quitaría al ver que todo va bien, y la que convierte el segundo tramo en un TLS que aparenta estar puesto.

La segunda mitad del patrón: el síntoma apunta a otro sitio

Esto es lo que más cuesta en la práctica. **El error de superficie despistaba sobre la causa, y en una dirección concreta**: el sistema tiene relevos y reintentos —réplicas alternativas, otro DataNode, otra instancia del ControlNode—, y **un relevo convierte «esto está mal» en «esto no responde»**.

Fallo	Síntoma que se ve	Categoría a la que apunta	Categoría real
Direccionamiento	`409 no alcanzan el quórum de escritura (W=2)`	Capacidad o carga del clúster	**Red**: una URL válida solo desde el host
`cert=` de httpx	«Server disconnected without sending a response»	Un fallo del **servidor**	Un fallo del **cliente**, que no presentó su certificado
`blocks.size`	«ninguna de las 1 réplicas pudo servir el bloque»	**Disponibilidad**: no hay copias vivas	**Tamaños**: la única copia estaba bien y se rechazaba

Y un cuarto que añade algo: **una explicación falsa que encajaba con todo.** Un guion de demostración falló con el mismo `409` que el direccionamiento, y coincidió con haberlo lanzado segundos después del `up`, con DataNodes aún arrancando. La causa real era otra. Un síntoma con **dos causas plausibles** se cierra

con la primera que aparece, y la prueba que parecía confirmarla —un `put` a mano que sí funcionaba— recorría un camino distinto del del guion.

La contradicción interna: dos partes del mismo documento

El caso del grupo de seguridad es una **variante nueva del patrón**, y la única que apareció **ejecutando** el despliegue y no repasándolo. Las otras son «algo dejó de ser verdad». Aquí nada dejó de ser verdad: **el documento se contradecía consigo mismo desde el primer día.** Una sección abría el 8000 solo a una IP externa; otra dedicaba una sección entera a explicar que la única opción soportada es el cliente corriendo dentro de la VPC.

El agravante dice dónde estaba la atención: el 8001 **sí** tenía las dos reglas. Se pensó en el doble origen para el puerto de los **datos** y no para el del **control**, aunque el puerto de control es el que usa el escenario descrito.

Lo que esto añade: **no basta con que cada afirmación de un documento sea correcta; tienen que ser compatibles entre sí, y nada lo comprueba.** Un README no tiene pruebas, y dos secciones escritas en momentos distintos envejecen por separado.

La forma más externa: la rama por defecto

La rama por defecto del repositorio en GitHub era **`etapa-1`**, y lo era **desde el hito 1**. Quien clonara sin indicar rama —que es lo que hace cualquiera— obtenía el proyecto de la semana 8. Un evaluador que hiciera eso concluiría, con razón según lo que tenía delante, que **falta casi todo**.

El del despliegue en AWS era el peor de los dos, y explica por qué esta capa importa:

1. **Lo que daban las instrucciones**: `etapa-1` **no tiene el directorio `deploy/`**. El primer comando fallaba con «no such file». Ruidoso e inmediato.
2. **Lo que habría venido después, y ese sí es el peligroso**: ante ese error, la salida natural es levantar el `docker-compose.yml` de la raíz, que en `etapa-1` existe y funciona. Eso arranca **un ControlNode y un DataNode, sin replicación, sin certificados, sin liderazgo** — un sistema sano de la semana 8. El síntoma habría sido desconcertante («¿dónde está la replicación?») y la causa, la rama del clon, es de las que nadie mira: todo lo que se está depurando es código, y el código está bien.

La diferencia entre los dos pasos es la lección: el fallo ruidoso no protege si la forma natural de sortearlo lleva al silencioso. El remedio no es confiar en que alguien entienda el primer error, sino que las instrucciones **no puedan** llevar a la rama equivocada: `git clone --branch main`.

El arreglo tiene dos mitades, igual que el resto: cambiar la rama por defecto en GitHub —un ajuste que no se puede fijar desde el repositorio— y **atar la documentación al destino**, que sigue siendo correcto aunque el ajuste de GitHub vuelva a cambiar.

El R=1 que no habría fallado: por qué «funcionar» no es evidencia

De los cuatro fallos del material de AWS que se encontraron repasando, dos habrían costado horas, y por motivos opuestos:

- **La URL interna vacía** habría dado el `409 no alcanzan el quórum` de siempre: el síntoma que ya despistó dos veces hacia la capacidad del clúster, ahora en seis máquinas y con el reloj del laboratorio corriendo.
- **`R=1` no habría fallado en absoluto.** El hito 3 se habría desplegado **sin replicación y funcionando en apariencia bien**: cada `put` confirmado, cada `get` correcto, y el `FULLY_REPLICATED (1 de 1)` diciendo **la verdad sobre una configuración equivocada**.

Eso es peor que fallar: un fallo se investiga; un sistema que funciona se da por bueno, y se habría descubierto, si acaso, al matar un DataNode delante de quien evalúa.

La proporción es el dato: cuatro de cinco fallos del material de AWS se encontraron **leyendo** con la lupa del patrón, y el quinto necesitó **seis máquinas encendidas**. Repasar sirve, y no sustituye a ejecutar.

Dos casos posteriores al cierre de la cuenta

La cifra de veintidós quedó cerrada con la entrega del hito 3. Durante el hito 4, al repasar la documentación contra el código antes de entregar, aparecieron dos más. Van aquí, fuera del conteo y con la fecha, en lugar de renumerar: **el dato interesante no es que ahora sean veinticuatro, es que la lupa siguió produciendo hallazgos después de que el equipo diera el repaso por terminado.**

Uno: una variable de entorno que no existe. `docker-compose.yml` pasaba `DFSHA_CONTROL_URL` a los cuatro DataNodes, y `DataNodeSettings` no tiene ese campo: con `extra="ignore"` de pydantic-settings, la variable entraba y se descartaba en silencio. El plano interno va por `DFSHA_CONTROL_INTERNAL_URL`, que tiene otro puerto, otro esquema y otra autenticación. No rompía nada —por eso llevaba ahí sin que nadie lo notara— pero podía desviar un diagnóstico entero: quien quisiera cambiar a dónde habla el DataNode habría tocado esa línea, no habría visto ningún efecto, y habría concluido algo falso del sistema. La misma variable en el servicio `client` **sí** se lee, así que borrar las dos era el error fácil.

Dos, y es el que importa: una justificación que describía un caso que no es el que ocurre. La decisión de no arreglar la resolución de `upstream` de nginx (§6.6) se apoyaba en tres razones, y la segunda decía que el efecto estaba acotado porque nginx saca de la rotación la IP obsoleta y «se recupera solo». Al levantar el stack tras los cambios, los cuatro DataNodes entraron en bucle de reinicio: el disparador real es `docker compose up -d --build`, que recrea los tres ControlNodes y no el `lb`, de modo que quedan obsoletas **las tres IP a la vez** y no hay ninguna a la que pasar.

Es el patrón otra vez, en un sitio nuevo: **no en una afirmación sobre el sistema, sino en una justificación de una decisión sobre el sistema.** La decisión sigue siendo correcta —el arreglo cuesta justo lo que el balanceador aporta— pero el argumento que la sostenía describía un fallo raro y auto-recuperable, y el real es frecuente y no se recupera. Nadie iba a corregirlo leyendo, porque las tres razones se leen bien; hizo falta que el caso ocurriera.

Y añade un matiz que las veintidós no tenían: **una justificación equivocada es más difícil de detectar que un hecho equivocado**, porque nada la ejercita. Un hecho falso en un README puede chocar con el código; un argumento solo choca con la realidad el día que alguien tropieza con el caso que describe mal.

La conclusión

Veintidós casos en siete capas distintas no es un descuido puntual: es un **modo de fallo del proyecto**. Y tiene una causa común que conviene nombrar: en todos, **algo dejó de ser verdad y lo que lo afirmaba no**

se enteró, porque nada los ataba. El código no comprueba que el README sea cierto, una prueba no comprueba que su nombre describa lo que hace, y un cliente no comprueba que el servidor le mandó datos que él va a imprimir.

Lo único que ha funcionado contra esto es hacer que algo exija el dato: validar en Docker, medir en vez de asumir, romper la protección para ver caer la prueba, o reescribir la documentación con el sistema delante.

Y hay un eco deliberado con el resultado conceptual del hito 3 (§6.4): allí la solución fue *exigir en vez de preguntar*, porque una pregunta se puede ignorar. Aquí la solución es la misma idea aplicada a la verificación: **una afirmación que nada ejercita es una afirmación que nadie comprueba.**

10.7 Un intermitente conocido, declarado

`tests/integration/test_transfer.py::TestIntegridad::test_los_bloques_son_inmutables` falló **una vez** en una pasada completa, el 17 de septiembre. No se ha reproducido: dos pasadas completas posteriores en verde, la prueba aislada en verde y `tests/integration` entero en verde. Se descartó que fuera el TTL del token (el plan se crea dentro de la propia prueba) y el orden de las pruebas (no hay `pytest-randomly` ni `xdist` instalados).

Queda declarado en lugar de omitido porque una prueba intermitente que verifica la **inmutabilidad de los bloques** —sobre la que además descansa la unicidad del nonce del cifrado— no es algo que se pueda dar por resuelto por no haber vuelto a salir. Lo que no se hizo, y es la razón de que siga abierto, fue **capturar el traceback**.

11. Límites conocidos y trabajo futuro

Ninguno de estos es un olvido. Están razonados y se defienden como decisiones.

11.1 Límites del sistema entregado

Límite	Por qué está así	Qué costaría quitarlo
PostgreSQL es punto único de fallo; la promoción de la réplica es manual	Un failover automático correcto necesita árbitro externo, protección contra doble promoción y aviso a los clientes. **Uno a medias es peor que ninguno**	Otro proyecto. El runbook manual está escrito
PBKDF2 en vez de Argon2id	Argon2id resiste mejor el hardware especializado; PBKDF2 está en la biblioteca estándar y no añade dependencia	Cambiar `derive_master_key` y nada más
La clave maestra se guarda en la sesión	El modelo es «el servidor nunca ve la clave», no «la clave nunca toca el disco». Cada `dfsha` es un proceso nuevo: en memoria no serviría	`login --ask-password` ya existe y no la guarda
No hay revocación de tokens de bloque	Retirar un permiso corta la *emisión*; lo ya emitido vale hasta caducar, y por eso la vida son 10 minutos. Consultar al ControlNode en cada petición devolvería el plano de control al camino de los datos	Una lista de revocación compartida, con su propio coste de consistencia
Cambiar la contraseña no re-cifra nada	Las envolturas existentes dejarían de abrirse. Re-envolverlas exige las dos contraseñas a la vez, es decir, hacerlo *durante* el cambio y desde el cliente	El CLI lo dice al fallar en vez de dejar un archivo ilegible sin explicación
La sobre-replicación tras una reincorporación no se limpia sola	Cuesta disco, no corrección, y quitarla automáticamente violaría la regla de que el ControlNode no borra datos por una divergencia	Un mecanismo nuevo y deliberado, con su propia decisión escrita
`append` reescribe el bloque de cola	Amplificación acotada por el tamaño de bloque, no por el del archivo. La alternativa produce miles de bloques diminutos	Un diseño de bloques de tamaño variable
El recolector se corre a mano	Es lo que pide el enunciado, y un demonio que borra datos	Un planificador con su propia política de seguridad

Límite	Por qué está así	Qué costaría quitarlo
	automáticamente es exactamente lo que la regla de §5.5 evita	
Un `data.crt` compartido por los DataNodes	Simplificación de práctica: el SAN cubre los cuatro nombres y el número de nodos es fijo	Un certificado por nodo, y con ello emisión y rotación por nodo
No hay 2FA	No está en el enunciado	—
El contenedor `client` no puede transferir	El plan trae `localhost` y ahí `localhost` es el propio contenedor. El escenario soportado es el cliente del host	Ver §3.3: el precio es la inferencia de direcciones que se rechazó
nginx no re-resuelve los upstreams	Arreglarlo cuesta `least_conn` y el reintento en otra instancia, que es justo lo que el nivel 7 aporta	nginx Plus o un módulo de upstream dinámico

11.2 Trabajo futuro, en orden de valor

1. **Failover automático de PostgreSQL** con un árbitro externo (Patroni o equivalente). Es el único punto único de fallo que queda y el que más valor tiene cerrar.
2. **Recorte de copias sobrantes**, como mecanismo deliberado y no como efecto del recolector, con su propia decisión escrita sobre cuándo es seguro borrar una réplica de un archivo vivo.
3. **Argon2id** para la derivación de clave, y con ello re-envoltura de claves durante el cambio de contraseña.
4. **Rebalanceo al añadir nodos.** Hoy un nodo nuevo solo recibe bloques nuevos; mover bloques existentes hacia él exige una política de rebalanceo con sus propios frenos, de la misma familia que los tres de la re-replicación.
5. **Un certificado por DataNode**, para que revocar a uno no obligue a rotar a todos.
6. **Cifrado por trozos dentro del bloque**, que haría posible pedir rangos parciales al DataNode a cambio de más etiquetas y más nonces que gestionar.
7. **Métricas exportables.** Hoy son logs JSON por decisión explícita —sin Prometheus ni OpenTelemetry—, lo cual es suficiente para medir pero no para alertar.

11.3 Qué se llevaría el equipo a un proyecto siguiente

Tres cosas, y las tres salen de §10.6 más que de cualquier otra parte del trabajo:

1. **Ejecutar lo que se va a desplegar, en el entorno donde se va a desplegar.** Compilar no es arrancar; arrancar en Linux no es arrancar en Windows; y un clúster de pruebas en un solo proceso no distingue dos redes.
2. **Toda comprobación negativa necesita un control positivo al lado.** «No aparece el texto claro» y «la búsqueda estaba mal escrita» se ven exactamente igual, y durante dos días una de ellas se tomó por la otra.
3. **Preferir una API que exige a una que informa.** Una función que responde «¿puedo?» se puede ignorar; una que hace «hazlo si puedes» no. Esa única idea resolvió cuatro problemas que no se

parecían en nada, y su ausencia explica la mayoría de los fallos que el proyecto tuvo que perseguir.

Anexo A — Modelo de metadatos

```
users(id, username UNIQUE, password_hash, kdf_salt, created_at)
    kdf_salt: sal del KDF del CLIENTE. NO es secreta y viaja en el
login; su
    trabajo es que dos usuarios con la misma contraseña tengan
claves distintas.
    NUNCA vacía desde la migración 0008: vacía, el cliente subía EN
CLARO.

directories(id, parent_id NULL, name, owner_id, created_at,
deleted_at NULL)
    UNIQUE(parent_id, name) solo sobre filas con deleted_at NULL
    UNIQUE(owner_id) solo sobre filas con parent_id NULL: una raíz
por usuario

files(id, directory_id, name, owner_id, size, block_size, state,
      created_at, committed_at, expires_at NULL, deleted_at NULL,
      wrapped_key, key_algo)
    state in {WRITING, COMMITTED, DELETED}
    UNIQUE(directory_id, name) solo sobre filas COMMITTED
    size son bytes CLAROS
    wrapped_key: clave del archivo envuelta con la maestra. Se fija
en el COMMIT,
    porque se envuelve con el file_id. Vacía = SIN CIFRAR.

file_locks(id, file_id, holder_id, holder_name, mode, epoch,
acquired_at, expires_at)
    mode in {shared, exclusive}
    UNIQUE(file_id, holder_id): pedir dos veces el mismo lock es
RENOVAR
    holder_id es una SESIÓN, no un usuario
    los VENCIDOS se quedan en la tabla: el vencimiento se evalúa al
consultar

blocks(block_id UUID PK, file_id NULL, index, size, checksum_sha256)
    file_id NULL = bloque DESLIGADO. Lo produce el append al
reescribir la cola.
    UNIQUE(file_id, index) -- además de un invariante, es lo que
garantiza que el
    nonce derivado de (file_id, index) no se repita nunca
    size son los bytes ALMACENADOS: con cifrado, los claros más los
16 de GCM
    checksum_sha256 es el del texto CIFRADO

block_replicas(block_id, data_node_id, state, created_at)
    state in {PENDING, STORED, MISSING};  PK(block_id, data_node_id)
    la re-replicación TAMBIÉN inserta filas aquí al despachar una
```

```
copia

leadership(id=1, leader_id NULL, epoch, acquired_at, renewed_at,
expires_at)
    una sola fila;  epoch solo sube

rereplication_tasks(id, block_id, kind, state, source_node_id,
target_node_id,
                    replicas_at_schedule, created_at, dispatched_at,
sent_at,
                    expires_at, attempts, last_error)
    kind in {REPLICATE, DELETE};  state in {PENDING, IN_FLIGHT,
DONE, FAILED}
    UNIQUE(block_id) solo sobre PENDING o IN_FLIGHT: una tarea viva
por bloque

groups(id, name, owner_id, created_at)        UNIQUE(owner_id, name)
group_members(group_id, user_id, added_at)    grupos PLANOS

acl_entries(id, directory_id, principal_type, principal_id,
permission,
            granted_by, granted_at)
    permission in {1=READ, 2=WRITE, 3=ADMIN}, entero para que el
orden de potencia
    sea el del propio dato.  SOLO concesiones.  Se cuelgan de un
DIRECTORIO.

data_nodes(id, advertise_url UNIQUE, peer_url, capacity_bytes,
used_bytes, state,
           registered_at, fault_domain, boot_id, stat_*)
    state in {ALIVE, DEAD}   (SUSPECT se deriva, no se guarda)
```

Dos desviaciones deliberadas respecto al esquema original, descubiertas al implementar:

1. **`directories.deleted_at`** — los directorios se borran de forma lógica. `rmdir -r` marca `DELETED` los archivos que contiene, y esas filas tienen que sobrevivir hasta que el recolector recoja sus bloques; pero `files.directory_id` apunta a `directories`, así que borrar físicamente la fila violaría la clave foránea.
2. **`GET /gc/orphan-blocks` devuelve también `size`** — sin ese campo, el recolector tendría que hacer una petición extra por bloque al DataNode solo para poder informar cuántos bytes liberó.

Reglas de dominio que `domain/` hace cumplir:

- `rmdir` falla si el directorio no está vacío, salvo `--recursive`.
- Un archivo en estado `WRITING` no aparece en `ls` ni puede abrirse.
- **Reserva vencida**: archivo en `WRITING` con `expires_at` en el pasado = abandonado. No bloquea el nombre. Comprobación **perezosa**, nunca un barrido en segundo plano.

- **Copy-on-write sobre una ruta ocupada**: `create` sobre un nombre ya `COMMITTED` se permite; en el commit, el viejo pasa a `DELETED` y el nuevo a `COMMITTED` **en una sola transacción**. En dos pasos, una caída en medio dejaría la ruta sin archivo visible.
- `mv`: si `dst` es un directorio existente, mueve dentro conservando el nombre; si no existe y su padre sí, renombra; si `dst` es un archivo existente, `409`. El padre nunca se crea implícitamente.
- Las rutas se normalizan y validan en un *value object* `Path`: rechaza `..`, rutas relativas sin cwd, nombres vacíos y caracteres de control.

Anexo B — Variables de configuración

Todo por variables de entorno vía Pydantic Settings; `.env.example` las lista todas. **Una línea vacía significa «no configurada», nunca «cadena vacía».**

Variable	Por defecto	Para qué
`DFSHA_BLOCK_SIZE`	`67108864` (64 MB)	Tamaño de bloque
`DFSHA_CONTROL_URL`	`http://localhost:8000`	Dónde está el ControlNode
`DFSHA_DB_URL`	`sqlite:///./dfsha.db`	Metadato. En Docker, PostgreSQL primario
`DFSHA_DB_REPLICA_URL`	vacío	Réplica de lectura. **Vacío = todo al primario**, modo soportado
`DFSHA_PG_PASSWORD`	— **obligatorio**	Clave del rol de aplicación
`DFSHA_PG_REPLICATION_PASSWORD`	— **obligatorio**	Clave del rol de replicación, que solo puede replicar
`DFSHA_JWT_SECRET`	— **obligatorio**	Firma de los tokens
`DFSHA_JWT_TTL_SECONDS`	`3600`	Vida del token
`DFSHA_TLS_CA_CERT`	— **obligatorio**	La CA contra la que todos se validan
`DFSHA_TLS_CERT` / `DFSHA_TLS_KEY`	— **obligatorio**	El certificado de este rol y su clave
`DFSHA_CLIENT_TLS_CERT` / `_KEY`	vacío	TLS de cara al cliente. **Las dos o ninguna**
`DFSHA_INTERNAL_PORT`	`8443`	Puerto del plano interno
`DFSHA_DATA_DIR`	`/var/lib/dfsha`	Dónde guarda bloques el DataNode
`DFSHA_DATANODE_ADVERTISE_URL`	`http://localhost:8001`	URL **alcanzable por el cliente**
`DFSHA_DATANODE_PEER_URL`	vacío	URL **alcanzable por otros DataNodes**. Vacío = la misma
`DFSHA_DATANODE_FAULT_DOMAIN`	`local-1`	Dominio de falla: cadena opaca
`DFSHA_DATANODE_CAPACITY_BYTES`	libre en disco	Capacidad anunciada
`DFSHA_CONTROL_GRPC_URL`	`lb:9000`	El balanceador, no una instancia
`DFSHA_GRPC_PORT`	`9000`	Puerto gRPC del ControlNode
`DFSHA_HEARTBEAT_INTERVAL_MS`	`3000`	Cada cuánto late un DataNode
`DFSHA_FULL_REPORT_EVERY_N`	`20`	Cada cuántos latidos manda un report completo
`DFSHA_SUSPECT_AFTER_M`	`10000`	Silencio tras el que sale de la

Variable	Por defecto	Para qué
S		colocación
DFSHA_DEAD_AFTER_MS	30000	Silencio tras el que sus réplicas se dan por no disponibles
DFSHA_MEMBERSHIP_INTERVAL_MS	1000	Cada cuánto se evalúan las transiciones
DFSHA_LEASE_TTL_MS	6000	Vida del lease de líder sin renovar
DFSHA_LEASE_RENEW_MS	2000	Debe ser menor que el TTL o el servicio no arranca
DFSHA_WRITE_QUORUM	2	Réplicas confirmadas que exige el commit. No puede ser mayor que R
DFSHA_REPLICATION_FACTOR	3	Copias por bloque
DFSHA_PLACEMENT_D	3	Tamaño de la ventana del *power of d choices*
DFSHA_MIN_FREE_BYTES	134217728	Margen de disco que un nodo conserva para ser candidato
DFSHA_REREPLICATION_GRACE_MS	300000	Espera desde que un nodo entra en DEAD
DFSHA_REREPLICATION_MAX_PER_NODE	2	Copias simultáneas hacia el mismo destino
DFSHA_REREPLICATION_MAX_PER_PASS	8	Copias despachadas por pasada
DFSHA_REREPLICATION_INTERVAL_MS	5000	Cada cuánto corre el planificador
DFSHA_ORDER_WORKERS	2	Copias simultáneas que un DataNode acepta ejecutar
DFSHA_FILE_LOCK_TTL_MS	120000	Vida de un bloqueo de archivo (RF3)
DFSHA_WRITE_TTL_SECONDS	600	Vencimiento de las reservas de escritura
DFSHA_LOG_LEVEL	INFO	Nivel de log

Los secretos y los tres ficheros de TLS **no tienen valor por defecto en el código**. Un plano interno que arranca **sin autenticación** porque se olvidó una variable es peor que uno que no arranca, porque no se nota. El DataNode además comprueba que los ficheros **existen**, para que el fallo sea «no existe `certs/data.crt`» y no un error de *handshake* diez segundos después.

`DFSHA_INTERNAL_SECRET` **ya no existe**: lo sustituyó el mTLS. Un `.env` viejo que todavía lo tenga simplemente lo ignora.

Anexo C — Glosario

Término	Qué significa en DFSha
Bloque	Unidad de distribución y de replicación. Tamaño fijo, inmutable una vez cerrado, identificado por un UUID
`block_id`	Identificador del bloque. Es lo único que el DataNode conoce: no sabe de rutas ni de usuarios
Dominio de falla	Cadena opaca que declara cada DataNode y de la que el sistema solo compara igualdad. Dos réplicas intentan no caer en el mismo
Época	Número que solo sube, asociado a un permiso (liderazgo o lock). Viaja con cada operación y permite rechazar la de un titular que ya no lo es
Fencing / token de cercado	El mecanismo general del que la época es la implementación: una credencial que el receptor puede comparar contra la última que vio
`FULLY_REPLICATED` / **`UNDER_REPLICATED`**	Estado de replicación de un archivo, derivado de su bloque **peor** replicado
Lease	Permiso con vencimiento que hay que renovar. Resuelve al titular que muere; no al que se congela
LSN	*Log Sequence Number* de PostgreSQL. El cliente lo guarda tras escribir y lo reenvía al leer, y es lo que garantiza read-your-writes
Pipeline	La cadena por la que un bloque viaja de un DataNode al siguiente, para que el cliente lo suba una sola vez
Plan	Lo que el ControlNode devuelve al cliente: qué bloques, a qué nodos, con qué tokens. Es el objeto que permite que los bytes no pasen por el plano de control
Quórum (W)	Número de réplicas confirmadas que exige el commit. W=2 con R=3
Réplica	Una copia física de un bloque en un DataNode concreto. Estados: `PENDING`, `STORED`, `MISSING`
Re-replicación	El proceso que repone las copias que faltan, con tres frenos: gracia, tope por destino y prioridad por copias restantes
WORM	*Write Once, Read Many*. Un bloque cerrado no se sobrescribe nunca

Anexo D — Dónde está cada cosa

Qué	Dónde
Código fuente	`https://github.com/tsepulvedf/DFSha-Telematica-` (rama `main`, tag `hito-3`)
Instrucciones de arranque	`README.md`, sección «Arranque rápido», y §9.2 de este informe
Todas las decisiones de diseño con su argumentación	`CLAUDE.md`
Material de despliegue en AWS	`deploy/README.md`
Promoción manual de la réplica de PostgreSQL	`deploy/RUNBOOK-postgres.md`
Guiones de demostración	`scripts/demo/` y su propio `README.md`
Verificación por mutación	`scripts/verificar_pruebas.py`
Contrato del plano de control	`src/dfsha/common/proto/control.proto`
Migraciones del metadato	`alembic/versions/` (ocho revisiones)
Documentación interactiva de las APIs	`http://localhost:8000/docs` y `http://localhost:8001/docs`